Aprendizado de Máquina - Árvore de Decisão

IFSP - PEP

Cláudio M. Zaina

Índice

Árvore de Decisão 2
Medidas de escolha da condição de teste de atributo . . . 3
Impureza . . . 3
Bibliotecas . . . 6
Algoritmo para construção de uma Árvore de Decisão . . . 8
Impureza coletiva dos nós-filhos . . . 8
Identificação da variável . . . 9
Estado civil (`civil`) . . . 9
`sexo` . . . 9
Nível Escolar (`educacao`) . . . 9
`credito` . . . 11
`salario` . . . 12
Escolha da decisão . . . 13
Dados de treino, teste e validação . . . 17
Modelo de Árvore de Decisão . . . 19
Qualidade do modelo: dados de validação . . . 21
Modelo de Árvore de Decisão Podada . . . 23
Caso 1 - Sucesso Acadêmico . . . 28
Dados . . . 28
Preparação das variáveis . . . 31
`graduou` . . . 34
`civil` . . . 35
`inscricao` . . . 36
`posicao` . . . 37
`curso` . . . 37
`periodo` . . . 38
`qualif_ant` . . . 38
`nacion` . . . 39
`qualif_mae` . . . 39
`qualif_pai` . . . 40
`prof_mae` . . . 41
`prof_pai` . . . 42
`relocado`, `nec_especial`, `devedor`, `pagto_em_dia`, `sexo`, `bolsa` e `intercamb` . . . 42
`credit_1sem` . . . 42
`matric_1sem` . . . 44
`cumprid_1sem` . . . 45
`aprov_1sem` . . . 46
`graduad_1sem` . . . 47
`navalid_1sem` . . . 48
`credit_2sem` . . . 49
`matric_2sem` . . . 51
`cumprid_2sem` . . . 52

aprov_2sem . . . 53
graduad_2sem . . . 54
navalid_2sem . . . 55
tx_desemp . . . 56
tx_inflacao . . . 56
tx_pib . . . 56
Separação conjuntos dados: treino, validação e teste . . . 57
Modelo básico . . . 58
Qualidade do modelo: dados de treino . . . 60
Qualidade do modelo: dados de validação . . . 61
Modelo com peso . . . 62
Qualidade do modelo: dados de treino . . . 65
Qualidade do modelo: dados de validação . . . 66
Modelo com custo . . . 67
Qualidade do modelo: dados de treino . . . 70
Qualidade do modelo: dados de validação . . . 71
Modelo com peso, custo e poda . . . 72
Qualidade do modelo: dados de treino . . . 76
Qualidade do modelo: dados de validação . . . 78
Avaliação final - modelo com custo: dados de teste . . . 79

Árvore de Decisão

Árvore de Decisão é uma técnica simples de classificador, bastante útil quando as relações que definem a classe da variável resposta são não-lineares.

Suas qualidades são:

- não é necessário padronização das variáveis pois apenas a posição do valor (posto ou *rank*) é importante;
- é robusta para valores perdidos (`NA`s);
- o modelo é fácil de interpretar;
- variáveis (atributos ou *features*) pouco importantes são automaticamente dispensadas;
- indicadores de impureza são consistentes entre si, fazendo com que todos sejam coerentes com seus resultados;
- seu desempenho é comparável àqueles das demais técnicas.

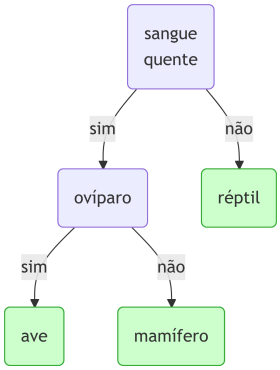
As limitações mais imediatas da técnica da árvore de decisão são:

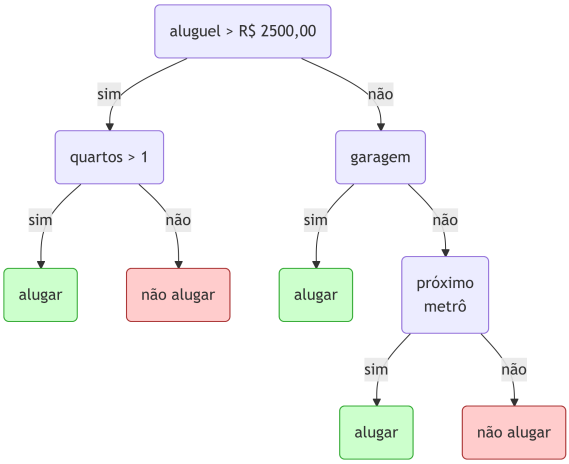
- uma variável categórica com muitos tipos diferentes de resposta (`profissão`, `nacionalidade`, `fabricante`, `sintoma` etc) pode, para árvores não binárias, criar classificações muito específicas para o conjunto de dados;
- tem propensão a superajuste (*overfitting*);
- insensível para variáveis com interação significativa. Se variáveis só são importantes para a definição da classe quando *juntas*, o método guloso de construção da árvore vai decidir por cada uma em separado, construindo árvores complexas demais, desperdiçando a informação que elas só fornecem *juntas*. Por exemplo, Índice de Massa Corpórea e circunferência abdominal. Cada indicador isolado é inconclusivo; juntos indicam risco real

Uma árvore é composta de três tipos de nós (vértices):

- **raiz**: um nó sem anteriores, ligado a nenhum ou mais (normalmente 2, no nosso caso) nós;
- **nó interno**: nó que não é raiz nem folha; possui só um nó anterior e 2 (ou mais) nós seguintes. Também chamado de *nó de decisão*;
- **folha**: nó que não possui nós seguintes; é o resultado da classificação.

Todo nó não-folha contém um teste de condição sobre (geralmente) um atributo. O resultado desse teste tem uma saída para cada resultado do teste. Uma vez atingido uma folha, uma classe é atribuída a ela. Exemplos podem ser observados a seguir, para determinar (grossamente) a classe de um animal ou se um imóvel é um bom negócio para ser alugado ou não:





Medidas de escolha da condição de teste de atributo

Há diversas medidas que podem ser usadas para a adequação da condição de teste de um atributo. O objetivo é obter um teste que obtenha a separação mais *limpa* ou *pura* dentre as classes nos nós filhos. Nós *puros* não precisam mais ser divididos.

Impureza

A impureza de um nó mede quão diferentes são os elementos (segundo a classificação desejada) que "caíram" naquele nó após a separação (do nó anterior). Há diversas medidas para avaliar a impureza de um nó t, sendo c o número de classes e $p_i(t)$ a frequência relativa das instâncias da classe i no nó t.

$$\text{Entropia} = -\sum_{i=0}^{c-1} p_i(t) \cdot log_2(p_i(t))$$

$$\text{Índice Gini} = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

$$\text{Erro de Classificação} = 1 - \max_i[p_i(t)]$$

Observação: consideramos $0 \cdot log_2(0) = 0$.

Vamos supor que tenhamos 4 nós N_1, N_2, N_3 e N_4, cada um com uma quantidade diferente de elementos das classes `0` e `1` mas separando seis observações do nó anterior:

Sendo c o número de classes e $p_i(t)$ a frequência relativa das instâncias da classe i no nó t.

Tabela 1: Nó com 0:0, 1:6

	N1	quantidade
classe	0	0
	1	6

$$\text{Entropia} = -(\tfrac{0}{6} \cdot log_2(\tfrac{0}{6}) + \tfrac{6}{6} \cdot log_2(\tfrac{6}{6})) = 0$$

$$\text{Gini} = 1 - ((\tfrac{0}{6})^2 + (\tfrac{6}{6})^2)) = 0$$

$$\text{Erro clasificação} = 1 - max(\tfrac{0}{6}, \tfrac{6}{6}) = 0$$

As observações foram totalmente separadas e, neste nó, só há uma classe. A impureza portanto é zero.

Tabela 2: Nó com 0:1, 1:5

	N2	quantidade
classe	0	1
	1	5

$$\text{Entropia} = -(\tfrac{1}{6} \cdot log_2(\tfrac{1}{6}) + \tfrac{5}{6} \cdot log_2(\tfrac{5}{6})) = 0{,}650$$

$$\text{Gini} = 1 - ((\tfrac{1}{6})^2 + (\tfrac{5}{6})^2)) = 0{,}278$$

$$\text{Erro clasificação} = 1 - max(\tfrac{1}{6}, \tfrac{5}{6}) = 0{,}167$$

O valor da impureza cresce rápido. Basta uma observação em seis ser de uma classe diferente que a impureza vai para 0,65, no caso da entropia.

Tabela 3: Nó com 0:2, 1:4

	N3	quantidade
classe	0	2
	1	4

$$\text{Entropia} = -(\tfrac{2}{6} \cdot log_2(\tfrac{2}{6}) + \tfrac{4}{6} \cdot log_2(\tfrac{4}{6})) = 0{,}918$$

$$\text{Gini} = 1 - ((\tfrac{2}{6})^2 + (\tfrac{4}{6})^2)) = 0{,}444$$

$$\text{Erro clasificação} = 1 - max(\tfrac{2}{6}, \tfrac{4}{6}) = 0{,}333$$

Com uma observação em três sendo de uma classe diferente, a entropia, por exemplo, se aproxima do máximo.

Tabela 4: Nó com 0:3, 1:3

	N4	quantidade
classe	0	3
	1	3

$$\text{Entropia} = -(\tfrac{3}{6} \cdot log_2(\tfrac{3}{6}) + \tfrac{3}{6} \cdot log_2(\tfrac{3}{6})) = 1$$

$$\text{Gini} = 1 - ((\tfrac{3}{6})^2 + (\tfrac{3}{6})^2)) = 0{,}5$$

$$\text{Erro clasificação} = 1 - max(\tfrac{3}{6}, \tfrac{3}{6}) = 0{,}5$$

Quando, após a separação, a quantidade de elementos de uma classe é igual àquela da outra, a entropia é máxima em 1 (a impureza é total).

Funções para o cálculo da `entropia`, índice de Gini (`gini`) e erro de classificação (`erro_clas`) para casos com apenas 2 classes:

```
rm(list=ls())
```

```
entropia <- function(a0,a1) { # qtd de categoria 0, qtd de categoria 1
    n <- a0 + a1
    if (n == 0) { return (0) }
    p1 <- a0/n
    p2 <- a1/n
    if (p1 == 0) {
        k1 <- 0
    } else {
        k1 <- p1 * log2(p1)
    }
    if (p2 == 0) {
        k2 <- 0
    } else {
        k2 <- p2 * log2(p2)
    }
    return (- (k1 + k2))
}
gini <- function(a0,a1) {
    n <- a0 + a1
    if (n == 0) {
        return (0)
    } else {
        return (1 - a0/n * a0/n - a1/n * a1/n)
    }
}
erro_clas <- function(a0,a1) {
    n <- a0 + a1
    if (n == 0) {
        return (0)
```

```
    } else {
        return (1 - max(a0/n, a1/n))
    }
}
```

Exemplo dos valores para 1 em 6:

```
a0 <- 1;
a1 <- 5
print(paste("Entropia:  ",round(entropia(a0,a1), 4)))
print(paste("Gini:      ",round(gini(a0,a1), 4)))
print(paste("Erro clas.:",round(erro_clas(a0,a1), 4)))
```

```
[1] "Entropia:   0.65"
[1] "Gini:       0.2778"
[1] "Erro clas.: 0.1667"
```

Bibliotecas

```
library(rpart)         # árvores de decisão
library(rpart.plot)    # gráficos de árvores
library(readr)         # leitura de csv
library(repr)          # representações (ajuste gráfico do jupyter)
library(caret)         # matriz de confusão
library(caTools)       # ferramentas em geral; no caso função que divide datasets
library(RColorBrewer)  # gradientes de cores
```

```
grafdim <- function(x=11, y=9) { options(repr.plot.width=x, repr.plot.height=y) }
grafdim()
```

Vamos construir um conjunto de dados sintético (artificial) com regras claras e ver se a técnica da Árvore de Decisão é capaz de recuperá-las.

```
set.seed(997)
```

```
criar_emprestimo <- function(tamanho, semente=997) {
    if (!is.na(semente)) {
        set.seed(semente)
    }
    empr <- data.frame(
        # 30% de solteiros, 70% de casados
        civil    = factor(sample(c('solteiro','casado'), tamanho,
                                 replace=TRUE, prob=c(0.3, 0.7))),
        # 50% para cada sexo
        sexo     = factor(sample(c('M','F'), tamanho,
                                 replace=TRUE, prob=c(0.5, 0.5))),
        # 20% para ensino médio, 60% para nível superior e 20% para pós
        educacao = factor(sample(c("médio", "superior", "pós-graduação"),
                                 tamanho, replace=TRUE,
                                 prob=c(0.2, 0.6, 0.2)),
                                 ordered=TRUE),
        # crédito tem distribuição uniforme em [0,55; 0,95]
        credito  = round(runif(tamanho, 0.55, 0.95) * 10, 1),
```

```
        # salário tem discribuição Wibull com parâmetros 1,6 e 2
        # para uma forma com mais densidade em salários mais baixos
        salario  = round(rweibull(tamanho,1.6,2)*40,4)*100 + 5000
    )
    # as regras "secretas" que motivam a concessão de um empréstimo
    empr$emprest <- factor(as.numeric(
        empr$civil == 'solteiro' & (empr$sexo == 'F' & empr$credito > 7.5 & empr$salario > 15000 |
                                    empr$sexo == 'M' & empr$credito > 9.0) |
        empr$salario > 25000 & empr$credito > 0.7 |
        empr$educacao == 'superior' & empr$salario > 10000 &
                         empr$credito > 0.6 & empr$civil == 'casado' |
        empr$educacao == 'pós-graduação' & empr$credito > 7.0 & empr$salario > 7000
    ))
    return (empr)
}
```

```
exemplo <- criar_emprestimo(10)
```

```
summary(exemplo)
```

```
      civil    sexo           educacao    credito          salario       emprest
 casado  :6   F:4   médio         :3   Min.   :5.600   Min.   : 7416   0:6
 solteiro:4   M:6   pós-graduação:1   1st Qu.:6.975   1st Qu.: 8726   1:4
                    superior      :6   Median :7.700   Median :10891
                                       Mean   :7.660   Mean   :13072
                                       3rd Qu.:8.350   3rd Qu.:15575
                                       Max.   :9.500   Max.   :29346
```

Se a semente for 997 e `tamanho` for 10, os dados gerados serão:

Tabela 5: Tabela com dados para exemplo

civil	sexo	educacao	credito	salario	emprest
casado	F	pós-graduação	9,5	17479,44	1
casado	M	superior	9,3	10546,73	1
solteiro	F	superior	7,8	29345,74	1
casado	F	médio	6,3	12428,51	0
solteiro	M	médio	8,5	9561,46	0
solteiro	M	superior	6,9	16623,75	0
casado	F	médio	7,2	7415,93	0
casado	M	superior	7,6	8446,85	0
casado	M	superior	5,6	11234,57	1
solteiro	M	superior	7,9	7638,89	0

como resultado de:

```
exemplo
```

A data.frame: 10 × 6

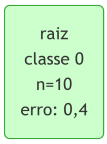
civil <fct>	sexo <fct>	educacao <ord>	credito <dbl>	salario <dbl>	emprest <fct>
casado	F	pós-graduação	9.5	17479.44	1

civil <fct>	sexo <fct>	educacao <ord>	credito <dbl>	salario <dbl>	emprest <fct>
casado	M	superior	9.3	10546.73	1
solteiro	F	superior	7.8	29345.74	1
casado	F	médio	6.3	12428.51	0
solteiro	M	médio	8.5	9561.46	0
solteiro	M	superior	6.9	16623.75	0
casado	F	médio	7.2	7415.93	0
casado	M	superior	7.6	8446.85	0
casado	M	superior	5.6	11234.57	1
solteiro	M	superior	7.9	7638.89	0

Algoritmo para construção de uma Árvore de Decisão

Muitas árvores de decisão podem ser construídas para um mesmo conjunto de dados. Decidir qual a melhor é computacionalmente intensivo, devido à quantidade de decisões possíveis. A abordagem utilizada é *gulosa* (*greedy* - gananciosa) e localmente ótima sobre qual atributos utilizar para decidir o próximo nível.

A base para os algoritmos atuais é o Algoritmo de Hunt. O início é com um nó (raiz) contendo todas as observações e atribuído à classe mais frequente. Assim, para o nosso caso, teremos um primeiro nó, com 10 observações, classificado como `0` (caso *negativo*, não concede empréstimo). O erro desse nó é 0,4, uma vez que há 4 casos (em 10) positivos classificados como negativos. Esse valor de erro é importante, pois normalmente é utilizado como critério de parada.



Uma vez que ainda há elementos de ambas classes no nó, a árvore pode ser expandida pela aplicação de uma decisão sobre um dos seus atributos. A pergunta então é qual atributo e qual valor?

O foco aqui será para a construção de árvores de decisão binárias.

Impureza coletiva dos nós-filhos

Considere uma critério de decisão d que divida um nó w com N observações em k nós-filhos $\{v_1, v_2, v_3, ..., v_k\}$. Vamos focar em árvores binárias, nas quais $k = 2$, com apenas $\{v_1, v_2\}$ mas a teoria é genérica. Vamos chamar de $N(v_i)$ a quantidade de observações que restaram no nó v_i após a divisão. Podemos calcular a impureza $I(v_i)$ desse nó. A impureza ponderada dos filhos de um nó w seria:

$$I(w_d) = \sum_{i=1}^{k} \frac{N(v_i)}{N} I(v_i)$$

```
impcol <- function(a0,a1,b0,b1,funcao=gini) {  # baseada em Gini
    a <- a0 + a1
    b <- b0 + b1
    n <- a + b
    if (a == 0) { ea <- 0 } else { ea <- funcao(a0,a1) }
    if (b == 0) { eb <- 0 } else { eb <- funcao(b0,b1) }
    ic <- (a/n) * ea + (b/n) * eb
    return (ic)
}
```

Identificação da variável

Vamos verificar a impureza coletiva do nó inicial segundo seus atributos pela entropia.

Estado civil (`civil`)

Calculando a impureza coletiva do nó $w = raiz$ para o estado civil.
Qualquer das métricas (Entropia, Gini ou Erro de Classificação) pode ser utilizada para calcular a impureza dos nós-filhos. O Índice de Gini é o mais comum pelo seu baixo custo computacional e é, portanto, padrão nas diversas bibliotecas que modelam árvores de decisão.
A impureza coletiva para a variável `sexo` será calculada com a Entropia e com o Índice de Gini como exemplo; os demais serão calculados utilizando o Índice de Gini.
Observando a tabela de exemplo da Tabela 5 com as 10 observações, há 4 casos com `civil` = `solteiro` e apenas 1 caso com empréstimo. Há 6 casos com casados, sendo que 3 deles conseguiram empréstimo:

Entropia

$$
\begin{aligned}
I(raiz_{\text{civil=solteiro}}) = &\quad -(\tfrac{3}{4} \cdot log_2(\tfrac{3}{4}) + \tfrac{1}{4} \cdot log_2(\tfrac{1}{4})) &&= 0{,}811 \\
I(raiz_{\text{civil=casado}}) = &\quad -(\tfrac{3}{6} \cdot log_2(\tfrac{3}{6}) + \tfrac{3}{6} \cdot log_2(\tfrac{3}{6})) &&= 1 \\
I(raiz_{\text{civil}}) = &\quad \tfrac{4}{10} \cdot 0{,}811 + \tfrac{6}{10} \cdot 1 &&= 0{,}925
\end{aligned}
$$

Índice de Gini

$$
\begin{aligned}
I(raiz_{\text{civil=solteiro}}) = &\quad 1 - ((\tfrac{3}{4})^2 + (\tfrac{1}{4})^2) &&= 0{,}375 \\
I(raiz_{\text{civil=casado}}) = &\quad 1 - ((\tfrac{3}{6})^2 + (\tfrac{3}{6})^2) &&= 0{,}500 \\
I(raiz_{\text{civil}}) = &\quad \tfrac{4}{10} \cdot 0{,}375 + \tfrac{6}{10} \cdot 0{,}5 &&= 0{,}450
\end{aligned}
$$

`sexo`

Há 4 casos com `sexo` = `F`, 2 com empréstimo. Há 6 casos de homens, sendo que 2 deles conseguiram empréstimo:

$$
\begin{aligned}
I(raiz_{\text{sexo=F}}) = &\quad 1 - ((\tfrac{2}{4})^2 + (\tfrac{2}{4})^2)) &&= 0{,}500 \\
I(raiz_{\text{sexo=M}}) = &\quad 1 - ((\tfrac{4}{6})^2 + (\tfrac{2}{6})^2)) &&= 0{,}444 \\
I(raiz_{\text{sexo}}) = &\quad \tfrac{4}{10} \cdot 0{,}5 + \tfrac{6}{10} \cdot 0{,}444 &&= 0{,}467
\end{aligned}
$$

Nível Escolar (`educacao`)

O nível escolar (`educacao`) possui 3 categorias: `médio`, `superior` e `pós-graduação`. O cálculo é feito do mesmo modo:

Tabela 6: Valores para `educacao`

categoria	0	1	total
`médio`	3	0	3
`superior`	3	3	6
`pós-graduação`	0	1	1

$$I(raiz_{\text{educacao=médio}}) = \quad 1 - ((\tfrac{3}{3})^2 + (\tfrac{0}{3})^2) \quad = 0$$

$$I(raiz_{\text{educacao=superior}}) = \quad 1 - ((\tfrac{3}{6})^2 + (\tfrac{3}{6})^2) \quad = 0{,}5$$

$$I(raiz_{\text{educacao=pós-graduação}}) = \quad 1 - ((\tfrac{0}{1})^2 + (\tfrac{1}{1})^2)) \quad = 0$$

$$I(raiz_{\text{educacao}}) = \quad \tfrac{3}{10} \cdot 0 + \tfrac{6}{10} \cdot 0{,}5 + \tfrac{1}{10} \cdot 0 \quad = 0{,}3$$

Porém vamos trabalhar apenas com árvores binárias. A solução seria agrupar categorias de modo sempre ter duas saídas. Assim, as três categorias `médio`, `superior` e `pós-graduação` poderiam ser convertidas para três possibilidades binárias:

- `educacaoA` : `médio` × (`superior` | `pós-graduação`)
- `educacaoB` : `superior`× (`médio` | `pós-graduação`)
- `educacaoC` : `pós-graduação` × (`médio` | `superior`)

`educacaoA` : `médio` × (`superior` | `pós-graduação`)

Temos 3 casos de `médio` e nenhum conseguiu empréstimo; `superior` ou `pós-graduação` são 7 casos com 4 empréstimos:

$$I(raiz_{\text{educacaoA=médio}}) = \quad 1 - ((\tfrac{3}{3})^2 + (\tfrac{0}{3})^2) \quad = 0$$

$$I(raiz_{\text{educacaoA=superior|pós-graduação}}) = \quad 1 - ((\tfrac{3}{7})^2 + (\tfrac{4}{7})^2) \quad = 0{,}490$$

$$I(raiz_{\text{educacaoA}}) = \quad \tfrac{3}{10} \cdot 0 + \tfrac{7}{10} \cdot 0{,}490 \quad = 0{,}343$$

`educacaoB` : `superior` × (`médio` | `pós-graduação`)

Temos 6 casos de `superior` e 3 conseguiram empréstimo; `médio` ou `pós-graduação` são 4 casos com 1 empréstimo:

$$I(raiz_{\text{educacaoB=superior}}) = \quad 1 - ((\tfrac{3}{6})^2 + (\tfrac{3}{6})^2) \quad = 0{,}500$$

$$I(raiz_{\text{educacaoB=médio|pós-graduação}}) = \quad 1 - ((\tfrac{3}{4})^2 + (\tfrac{1}{4})^2)) \quad = 0{,}375$$

$$I(raiz_{\text{educacaoB}}) = \quad \tfrac{6}{10} \cdot 0{,}5 + \tfrac{4}{10} \cdot 0{,}375 \quad = 0{,}450$$

`educacaoC` : `pós-graduação` × (`médio` | `superior`)

Temos 1 caso de `pós-graduação` e obteve empréstimo; `médio` ou `superior` são 9 casos com 3 empréstimo:

$$I(raiz_{\text{educacaoC=pós-graduação}}) = \quad 1 - ((\tfrac{0}{1})^2 + (\tfrac{1}{1})^2) \quad = 0$$

$$I(raiz_{\text{educacaoC=médio|superior}}) = \quad 1 - ((\tfrac{6}{9})^2 + (\tfrac{3}{9})^2)) \quad = 0{,}444$$

$$I(raiz_{\text{educacaoC}}) = \quad \tfrac{1}{10} \cdot 0 + \tfrac{9}{10} \cdot 0{,}444 \quad = 0{,}400$$

Reparar que algumas bibliotecas para gerar árvores de decisão possuem um parâmetro que restringe o tamanho mínimo em uma separação, considerando um *desperdício* de nível uma decisão que separa o nó em x valores, sendo $x < k$, sendo k um valor mínimo. Neste caso, a separação colocou de um lado 1 elemento e do outro 9.

credito

A variável `credito` (assim como `salario`) é contínua, não assumindo valores categóricos ou discretos. A técnica para construir a impureza dos filhos de um dado nó w é avaliar o melhor ponto de corte dentre os valores observados da variável.
A primeira ação é ordenar as observações pelo valor da variável:

```
exemplo[order(exemplo$credito),c(4,6)]
```

A data.frame: 10 × 2

	credito <dbl>	emprest <fct>
9	5.6	1
4	6.3	0
6	6.9	0
7	7.2	0
8	7.6	0
3	7.8	1
10	7.9	0
5	8.5	0
2	9.3	1
1	9.5	1

Em seguida, é construída uma tabela com valores intermediários (e externos, no caso do valor inferior e superior). O primeiro ponto de corte seria antes do primeiro valor de `credito`: 5,6. O valor é arbitrário, qualquer valor menor que 5,6 serve. Foi utilizada a mesma distância entre o primeiro e o segundo valor $(5{,}6-(6{,}3-5{,}6)/2 = 5{,}25)$ para o primeiro e a mesma lógica para o último.
Agora é calculado o valor da impureza - índice de Gini, no nosso caso - para cada limite de corte, inclusive o inferior e superior.

Tabela 7: Separação em classes para cálculo da Impureza coletiva por cada corte em `credito`

`emprest`		**1**		0		**0**		0		**0**		1		**0**		0		**1**		1			
`credito`		**5,6**		6,3		**6,9**		7,2		**7,6**		7,8		**7,9**		8,5		**9,3**		9,5			
		$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$	$\leq$	$>$
lim.		**5,2**	**5,2**	5,9	5,9	**6,6**	**6,6**	7,1	7,1	**7,4**	**7,4**	7,7	7,7	**7,85**	**7,85**	8,2	8,2	**8,9**	**8,9**	9,4	9,4	**9,6**	**9,6**
	0	**0**	**6**	0	6	**1**	**5**	2	4	**3**	**3**	4	2	**4**	**2**	5	1	**6**	**0**	6	0	**6**	**0**
	1	**0**	**4**	1	3	**1**	**3**	1	3	**1**	**3**	1	3	**2**	**2**	2	2	**2**	**2**	3	1	**4**	**0**

$$I(raiz_{\text{credito}\leq 5{,}2}) = \text{ como há uma divisão por 0, convencionou-se ser 0 } = 0$$

$$I(raiz_{\text{credito}>5{,}2}) = 1-((\tfrac{6}{10})^2+(\tfrac{4}{10})^2)) = 0{,}480$$

$$I(raiz_{\text{credito_5,2}}) = \tfrac{0}{10}\cdot 0+\tfrac{10}{10}\cdot 0{,}48 = 0{,}480$$

Observação: a primeira e a última supõe a criação de um nó com 0 observações. A convenção para esses casos é o índice ser 0. Na prática, os algoritmos de construção de árvores de decisão alteram os valores para que nunca ocorra essa situação.

$$
\begin{aligned}
I(raiz_{\text{credito}\leq 5{,}9}) = &\quad 1-((\tfrac{0}{1})^2+(\tfrac{1}{1})^2) &= 0\\
I(raiz_{\text{credito}>5{,}9}) = &\quad 1-((\tfrac{6}{9})^2+(\tfrac{3}{9})^2)) &= 0{,}444\\
I(raiz_{\text{credito_}5{,}9}) = &\quad \tfrac{1}{10}\cdot 0+\tfrac{9}{10}\cdot 0{,}444 &= 0{,}400
\end{aligned}
$$

$$
\begin{aligned}
I(raiz_{\text{credito}\leq 6{,}6}) = &\quad 1-((\tfrac{1}{2})^2+(\tfrac{1}{2})^2) &= 0{,}5\\
I(raiz_{\text{credito}>6{,}6}) = &\quad 1-((\tfrac{5}{8})^2+(\tfrac{3}{8})^2)) &= 0{,}469\\
I(raiz_{\text{credito_}6{,}6}) = &\quad \tfrac{2}{10}\cdot 0{,}5+\tfrac{8}{10}\cdot 0{,}469 &= 0{,}475
\end{aligned}
$$

$$
\begin{aligned}
I(raiz_{\text{credito}\leq 7{,}1}) = &\quad 1-((\tfrac{2}{3})^2+(\tfrac{1}{3})^2)) &= 0{,}444\\
I(raiz_{\text{credito}>7{,}1}) = &\quad 1-((\tfrac{4}{7})^2+(\tfrac{3}{7})^2) &= 0{,}490\\
I(raiz_{\text{credito_}7{,}1}) = &\quad \tfrac{3}{10}\cdot 0{,}444+\tfrac{7}{10}\cdot 0{,}490 &= 0{,}476
\end{aligned}
$$

...

$$
\begin{aligned}
I(raiz_{\text{credito}\leq 9{,}6}) = &\quad 1-((\tfrac{6}{10})^2+(\tfrac{4}{10})^2)) &= 0{,}480\\
I(raiz_{\text{credito}>9{,}6}) = &\quad \text{como há uma divisão por 0, convencionou-se ser 0} &= 0\\
I(raiz_{\text{credito_}9{,}6}) = &\quad \tfrac{10}{10}\cdot 0{,}480+\tfrac{0}{10}\cdot 0 &= 0{,}480
\end{aligned}
$$

É possível então construir uma tabela com a impureza coletiva (ic) dos filhos para cada corte em `credito`:

Tabela 8: Impureza coletiva em cortes de `credito`

cortes	5,2	5,9	6,6	7,1	7,4	7,7	7,85	8,2	**8,9**	9,4	9,6
ic	0,480	0,400	0,475	0,476	0,450	0,400	0,467	0,419	**0,300**	0,400	0,480

Podemos observar que o corte que produz a menor impureza coletiva dos filhos do nó raiz é `credito` $\leq 8{,}9$ com impureza coletiva de 0,300.

$$I(raiz_{\text{credito}}) = 0{,}300$$

`salario`

A variável `salario` é contínua e o procedimento é idêntico àquele aplicado ao `credito`.
A primeira ação é ordenar as observações pelo valor da variável:

```
exemplo[order(exemplo$salario),c("salario","emprest")]
```

A data.frame: 10 × 2

	salario <dbl>	emprest <fct>
7	7415.93	0
10	7638.89	0
8	8446.85	0
5	9561.46	0
2	10546.73	1
9	11234.57	1
4	12428.51	0
6	16623.75	0
1	17479.44	1
3	29345.74	1

É possível então construir uma tabela com a impureza coletiva (ic) dos filhos para cada corte em `salario`:

Tabela 9: Cortes `salario`

cortes	7.300	7.500	8.000	9.000	**10.000**	11.000	12.000	14.000	17.000	23.000	31.000
ic	0,480	0,444	0,400	0,343	**0,267**	0,400	0,467	0,419	0,300	0,400	0,480

Podemos observar que o corte que produz a menor impureza coletiva dos filhos do nó raiz é `salario` ≤ 10000 com impureza coletiva de 0,267.

$$I(raiz_{\text{salario}}) = 0{,}267$$

Escolha da decisão

Vamos analisar as impurezas coletivas e seus valores:

Tabela 10: Compilação das Impurezas coletivas das variáveis

variável	$I(raiz_{\text{variavel}})$
$raiz_{\text{civil}}$	0,450
$raiz_{\text{sexo}}$	0,467
$raiz_{\text{educacaoA}}$	0,343
$raiz_{\text{credito}<8{,}9}$	0,300
$raiz_{\text{salario}<10.000}$	0,267

A impureza da raiz era:

$$I(raiz) = 1 - ((\frac{6}{10})^2 + (\frac{4}{10})^2)) = 0{,}480$$

O melhor teste é aquele que maximiza a redução da impureza do nó-pai (o raiz, neste exemplo).
Podemos chamar de Δ a diferença entre a impureza original (pai) e a atual (filhos) e é, portanto, um ganho em pureza pela divisão.

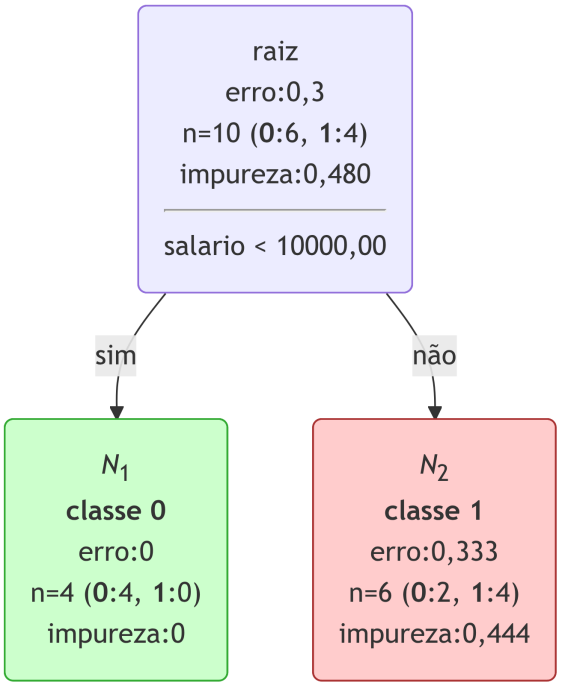
$$\Delta = I(pai) - I(filhos)$$

Para cada uma das variáveis podemos calcular a diferença:

Tabela 11: Diferença (ganho) entre a impureza do pai e dos filhos

variável	$I(raiz_{\text{variavel}})$	Δ
$raiz_{\text{civil}}$	0,450	0,480 - 0,450 = 0,030
$raiz_{\text{sexo}}$	0,467	0,013
$raiz_{\text{educacaoA}}$	0,343	0,137
$raiz_{\text{credito}<8,9}$	0,300	0,180
$raiz_{\text{salario}<10.000}$	0,267	0,213

A escolha então que aumenta a pureza daquele nó (raiz) é `salario` < 10.000



A impureza de N_1 é 0 e, portanto, não há mais o que fazer. A impureza de N_2 é 0,444 e está passível de todo o processo ser refeito agora considerando N_2 o nó-pai e procurando a melhor divisão para os seus nós-filhos.

```
a0 <- 4; a1 <- 2
b0 <- 0; b1 <- 1
round(gini(a0,a1),3)
round(gini(b0,b1),3)
round(impcol(a0,a1,b0,b1),3)
```

0.444

0

0.381

Utilizando a biblioteca `rpart` que implementa em R a construção recursiva de árvores de decisão:

```
modelo <- rpart(emprest ~ civil + sexo + educacao + credito + salario, # modelo
                data=exemplo,                                          # dados
                method="class",                                        # classificação
                control=rpart.control(minsplit=2, minbucket=0))        # parâmetros de controle
```

Algund parâmetros da criação da árvore de `rpart` também permitem fazer uma *pré-poda* na árvore:

- `minsplit`: especifica o mínimo de observações em um nó para que haja uma uma divisão. Por exemplo, se `minsplit` $= 5$, um nó com 5 ou menos observações automaticamente está finalizado;
- `maxdepth`: especifica a profundidade máxima da árvore. A árvore pára de crescer ao atingir `maxdepth`;
- `minbucket`: define a quantidade mínima de observações em uma folha. Se for tentada uma divisão e aparecer uma folha com menos que `minbucket`, a decisão é cancelada.

```
modelo
```

```
n= 10

node), split, n, loss, yval, (yprob)
      * denotes terminal node

 1) root 10 4 0 (0.6000000 0.4000000)
   2) salario< 10054.09 4 0 0 (1.0000000 0.0000000) *
   3) salario≥10054.09 6 2 1 (0.3333333 0.6666667)
     6) credito< 7.35 3 1 0 (0.6666667 0.3333333)
      12) credito≥5.95 2 0 0 (1.0000000 0.0000000) *
      13) credito< 5.95 1 0 1 (0.0000000 1.0000000) *
     7) credito≥7.35 3 0 1 (0.0000000 1.0000000) *
```

É possível ver até onde chegamos com os cálculos manuais: quando o nó raiz se dividiu nos nós 2 e 3 em função do `salario` < 10000. O nó 2 tem um asterisco `*` no final, indicando ser uma folha com probabilidades da classe 0 ser 1, e da classe 1, ser 0.

Vamos analisar o resultado dos passos do processo:

```
printcp(modelo)
```

```
Classification tree:
rpart(formula = emprest ~ civil + sexo + educacao + credito +
    salario, data = exemplo, method = "class", control = rpart.control(minsplit = 2,
    minbucket = 0))

Variables actually used in tree construction:
[1] credito salario

Root node error: 4/10 = 0.4

n= 10

    CP nsplit rel error xerror    xstd
1 0.50      0       1.0   1.00 0.38730
2 0.25      1       0.5   2.25 0.23717
3 0.01      3       0.0   2.00 0.31623
```

O erro da raiz é 0,4, pois considerou todas observações da classe 0 e há 4 da classe 1.
Se for desejado estudar cada nó e sua divisão, podemos pedir o resumo do modelo:

```
summary(modelo)
```

```
Call:
rpart(formula = emprest ~ civil + sexo + educacao + credito +
    salario, data = exemplo, method = "class", control = rpart.control(minsplit = 2,
```

```
    minbucket = 0))
  n= 10

    CP nsplit rel error xerror      xstd
1 0.50      0       1.0   1.00 0.3872983
2 0.25      1       0.5   2.25 0.2371708
3 0.01      3       0.0   2.00 0.3162278

Variable importance
 credito  salario educacao     sexo
      44       42        7        6

Node number 1: 10 observations,    complexity param=0.5
  predicted class=0  expected loss=0.4  P(node) =1
    class counts:     6     4
   probabilities: 0.600 0.400
  left son=2 (4 obs) right son=3 (6 obs)
  Primary splits:
      salario  < 10054.09 to the left,  improve=2.1333330, (0 missing)
      credito  < 8.9      to the left,  improve=1.8000000, (0 missing)
      educacao splits as  LRR,          improve=1.3714290, (0 missing)
      civil    splits as  RL,           improve=0.3000000, (0 missing)
      sexo     splits as  RL,           improve=0.1333333, (0 missing)
  Surrogate splits:
      educacao splits as  LRR,          agree=0.7, adj=0.25, (0 split)
      credito  < 7.05     to the right, agree=0.7, adj=0.25, (0 split)

Node number 2: 4 observations
  predicted class=0  expected loss=0  P(node) =0.4
    class counts:     4     0
   probabilities: 1.000 0.000

Node number 3: 6 observations,    complexity param=0.25
  predicted class=1  expected loss=0.3333333  P(node) =0.6
    class counts:     2     4
   probabilities: 0.333 0.667
  left son=6 (3 obs) right son=7 (3 obs)
  Primary splits:
      credito  < 7.35     to the left,  improve=1.3333330, (0 missing)
      educacao splits as  LRR,          improve=1.0666670, (0 missing)
      salario  < 11831.54 to the right, improve=0.6666667, (0 missing)
      civil    splits as  RL,           improve=0.1666667, (0 missing)
  Surrogate splits:
      salario < 17051.6  to the left,  agree=0.833, adj=0.667, (0 split)
      sexo    splits as  RL,           agree=0.667, adj=0.333, (0 split)

Node number 6: 3 observations,    complexity param=0.25
  predicted class=0  expected loss=0.3333333  P(node) =0.3
    class counts:     2     1
   probabilities: 0.667 0.333
  left son=12 (2 obs) right son=13 (1 obs)
  Primary splits:
      credito  < 5.95     to the right, improve=1.3333330, (0 missing)
      salario  < 11831.54 to the right, improve=1.3333330, (0 missing)
      civil    splits as  RL,           improve=0.3333333, (0 missing)
      sexo     splits as  LR,           improve=0.3333333, (0 missing)
      educacao splits as  LLR,          improve=0.3333333, (0 missing)
```

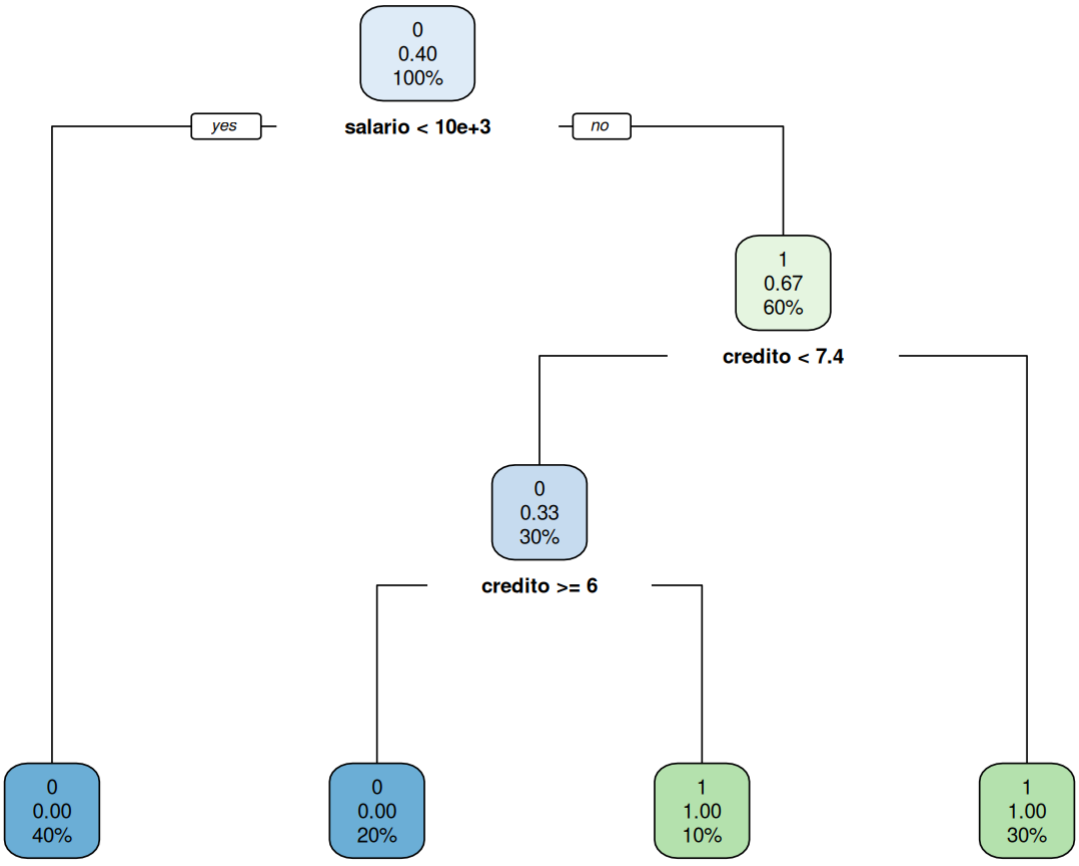
```
Node number 7: 3 observations
  predicted class=1  expected loss=0  P(node) =0.3
    class counts:     0     3
   probabilities: 0.000 1.000

Node number 12: 2 observations
  predicted class=0  expected loss=0  P(node) =0.2
    class counts:     2     0
   probabilities: 1.000 0.000

Node number 13: 1 observations
  predicted class=1  expected loss=0  P(node) =0.1
    class counts:     0     1
   probabilities: 0.000 1.000
```

Por fim, é possível pedir a representação gráfica da árvore:

```
grafdim(10,8)
rpart.plot(modelo)
```



Dados de treino, teste e validação

Construção dos conjuntos de treino, teste e validação

Construindo um conjunto de dados (*dataset*) com 1000 observações.

```
empr <- criar_emprestimo(1000)
```

```
summary(empr)
```

```
      civil     sexo             educacao      credito         salario
 casado  :686   F:504   médio         :211   Min.   :5.500   Min.   : 5053
 solteiro:314   M:496   pós-graduação:183   1st Qu.:6.575   1st Qu.: 8597
                        superior      :606   Median :7.600   Median :11318
                                             Mean   :7.549   Mean   :11985
                                             3rd Qu.:8.600   3rd Qu.:14487
                                             Max.   :9.500   Max.   :29250
 emprest
 0:607
 1:393
```

Especificação da quantidade de observações no conjunto de treino, validação e teste.

```
(num_obs   <- nrow(empr))
tx_valid   <- 0.17      # 20% para validação
tx_teste   <- tx_valid  # mesma porcentagem da validação
(num_teste <- trunc(num_obs * tx_teste))
(num_valid <- trunc(num_obs * tx_valid))
(num_trein <- num_obs - num_teste - num_valid)
```

1000

170

170

660

```
# treino começa com todas observações
treino <- empr
# sorteio da amostra
amostra  <- sample(1:num_obs, num_teste, replace=FALSE)
# separação do conjunto de teste
teste  <- treino[ amostra, ]
# exclusão dos dados do teste do treino
treino <- treino[-amostra, ]
```

```
# sorteio de total - quantidade do teste para validação
amostra  <- sample(1:(num_obs - num_teste), num_valid, replace=FALSE)
# separação do conjunto de validação
valid  <- treino[ amostra, ]
# exclusão dos dados da validação do treino
treino <- treino[-amostra, ]
```

```
nrow(treino)
nrow(valid)
nrow(teste)
```

660

170

170

Modelo de Árvore de Decisão

```
mod_bas <- rpart(emprest ~ civil + sexo + educacao + credito + salario,
                 data=treino,
                 method="class",
                 control=rpart.control(minsplit=2, minbucket=0))  # vamos permitir a árvore
                                                                  # crescer até o fim. Isso
                                                                  # dá chance de overfitting
```

```
mod_bas
```

```
n= 660

node), split, n, loss, yval, (yprob)
      * denotes terminal node

  1) root 660 259 0 (0.60757576 0.39242424)
    2) salario< 10415.8 288  36 0 (0.87500000 0.12500000)
      4) educacao=superior 177   7 0 (0.96045198 0.03954802) *
      5) educacao=médio,pós-graduação 111  29 0 (0.73873874 0.26126126)
       10) educacao=médio 57   1 0 (0.98245614 0.01754386) *
       11) educacao=pós-graduação,superior 54  26 1 (0.48148148 0.51851852)
         22) credito< 7.05 22   0 0 (1.00000000 0.00000000) *
         23) credito≥7.05 32   4 1 (0.12500000 0.87500000)
           46) salario< 6949.345 5   1 0 (0.80000000 0.20000000) *
           47) salario≥6949.345 27   0 1 (0.00000000 1.00000000) *
    3) salario≥10415.8 372 149 1 (0.40053763 0.59946237)
      6) educacao=médio 82   8 0 (0.90243902 0.09756098)
       12) salario< 25587.56 79   5 0 (0.93670886 0.06329114) *
       13) salario≥25587.56 3   0 1 (0.00000000 1.00000000) *
      7) educacao=pós-graduação,superior 290  75 1 (0.25862069 0.74137931)
       14) civil=solteiro 86  28 0 (0.67441860 0.32558140)
         28) credito< 7.25 32   0 0 (1.00000000 0.00000000) *
         29) credito≥7.25 54  26 1 (0.48148148 0.51851852)
           58) educacao=superior 40  14 0 (0.65000000 0.35000000)
            116) salario< 15669.66 28   4 0 (0.85714286 0.14285714) *
            117) salario≥15669.66 12   2 1 (0.16666667 0.83333333) *
           59) educacao=médio,pós-graduação 14   0 1 (0.00000000 1.00000000) *
       15) civil=casado 204  17 1 (0.08333333 0.91666667)
         30) educacao=médio,pós-graduação 46  17 1 (0.36956522 0.63043478)
           60) credito< 7.05 18   1 0 (0.94444444 0.05555556) *
           61) credito≥7.05 28   0 1 (0.00000000 1.00000000) *
         31) educacao=superior 158   0 1 (0.00000000 1.00000000) *
```

```
printcp(mod_bas)
```

```
Classification tree:
```

```
rpart(formula = emprest ~ civil + sexo + educacao + credito +
    salario, data = treino, method = "class", control = rpart.control(minsplit = 2,
    minbucket = 0))

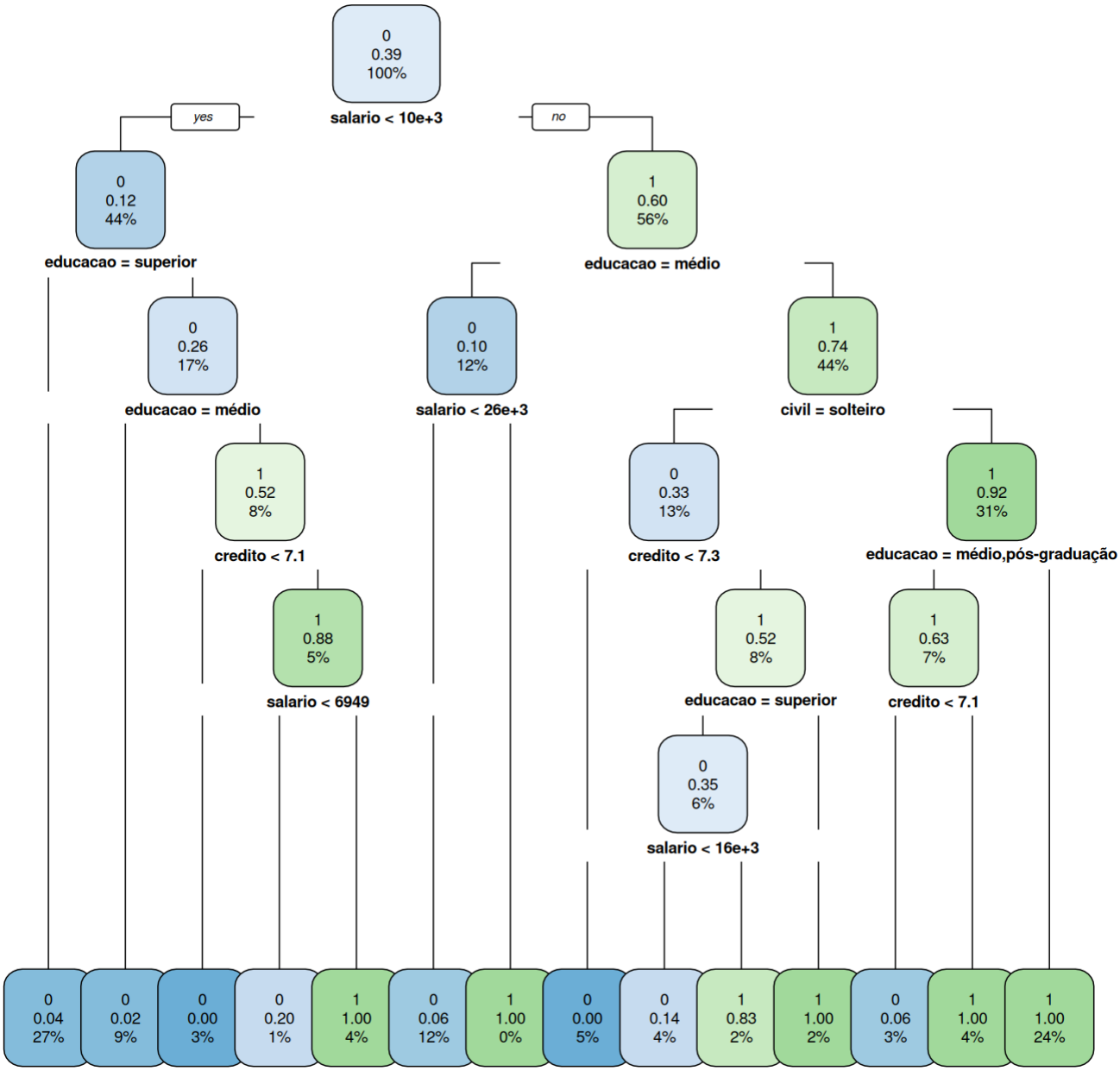
Variables actually used in tree construction:
[1] civil    credito  educacao salario

Root node error: 259/660 = 0.39242

n= 660

        CP nsplit rel error  xerror     xstd
1 0.285714      0  1.000000 1.00000 0.048434
2 0.254826      1  0.714286 0.89575 0.047358
3 0.115830      2  0.459459 0.48263 0.038865
4 0.030888      3  0.343629 0.35135 0.034198
5 0.027027      8  0.189189 0.25483 0.029757
6 0.011583     11  0.104247 0.20463 0.026956
7 0.010000     13  0.081081 0.15444 0.023668
```

```
grafdim(12,11)
rpart.plot(mod_bas, cex=0.9)
```




```
predito <- predict(mod_bas, valid, type="class")
print(summary(predito))
```

```
  0   1
110  60
```

Qualidade do modelo: dados de validação

Vamos avaliar a qualidade do modelo utilizando o conjunto de dados de validação:

```
matconf <- confusionMatrix(reference = valid$emprest, data=predito, positive="1" )
matconf
```

```
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 102   8
```

```
         1   1  59

               Accuracy : 0.9471
                 95% CI : (0.9019, 0.9755)
    No Information Rate : 0.6059
    P-Value [Acc > NIR] : <2e-16

                  Kappa : 0.8871

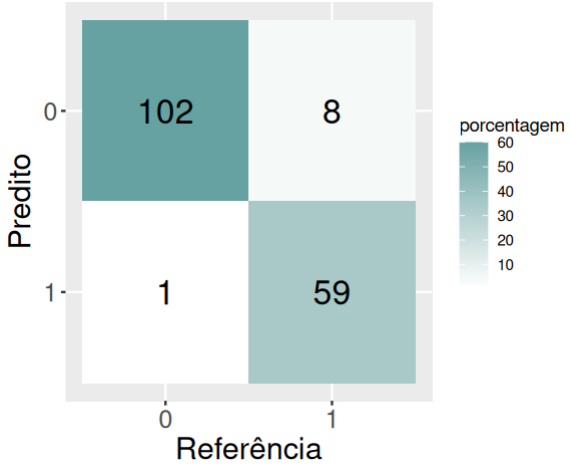
 Mcnemar's Test P-Value : 0.0455

            Sensitivity : 0.8806
            Specificity : 0.9903
         Pos Pred Value : 0.9833
         Neg Pred Value : 0.9273
             Prevalence : 0.3941
         Detection Rate : 0.3471
   Detection Prevalence : 0.3529
      Balanced Accuracy : 0.9354

       'Positive' Class : 1
```

```r
grafico_matriz_confusao <- function (matriz, cor_min="white", cor_max="#006464") {
    tabela <- as.data.frame(matriz$table)
    tabela$Prediction <- factor(tabela$Prediction, levels=rev(levels(tabela$Prediction)))
    total             <- sum(tabela$Freq)
    tabela$porcentagem<- round(tabela$Freq / total * 100,0)
    colfunc <- colorRampPalette(c(cor_min,cor_max))(100)
    cmin    <- colfunc[1]
    cmax    <- colfunc[max(tabela$porcentagem)]
    grafico <- ggplot(tabela, aes(Reference, Prediction, fill=porcentagem)) +
                      geom_tile() +
                      geom_text(aes(label=Freq), size=7) +
                      scale_fill_gradient(low=cmin, high=cmax)  +
                      labs(x="Referência", y="Predito") +
                      theme(axis.text=element_text(size=15), axis.title=element_text(size=18))
    return (grafico)
}
```

```r
grafdim(5,4)
grafico_matriz_confusao(matconf)
```



Modelo de Árvore de Decisão Podada

Árvores de decisão são competentes em capturar relacionamentos não lineares entre as variáveis porém o resultado pode ser desnecessariamente complexo. É possível então "podar" a árvore em um nível a partir do qual o ganho em precisão não é significativo, diminuindo a complexidade final da árvore.

```
printcp(mod_bas)
```

```
Classification tree:
rpart(formula = emprest ~ civil + sexo + educacao + credito +
    salario, data = treino, method = "class", control = rpart.control(minsplit = 2,
    minbucket = 0))

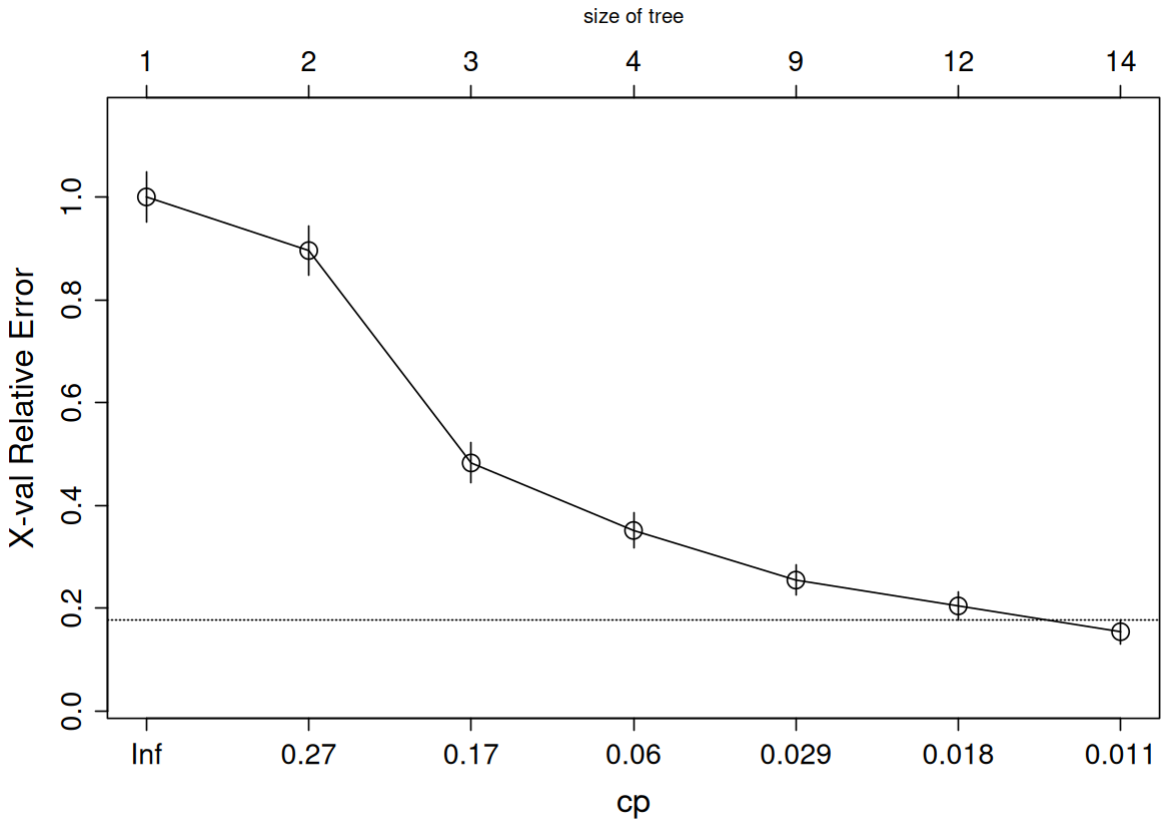
Variables actually used in tree construction:
[1] civil    credito  educacao salario

Root node error: 259/660 = 0.39242

n= 660

        CP nsplit rel error  xerror     xstd
1 0.285714      0  1.000000 1.00000 0.048434
2 0.254826      1  0.714286 0.89575 0.047358
3 0.115830      2  0.459459 0.48263 0.038865
4 0.030888      3  0.343629 0.35135 0.034198
5 0.027027      8  0.189189 0.25483 0.029757
6 0.011583     11  0.104247 0.20463 0.026956
7 0.010000     13  0.081081 0.15444 0.023668
```

```
grafdim(10,7)
plotcp(mod_bas, cex=1.8, cex.lab=1.6, cex.axis=1.4, sub="")
```



É costume podar a árvore no nível em que o "ganho" de mais um nível não é significativamente maior que o anterior (dentro do seu desvio padrão), ou seja, é o nível com o menor `xerror` para o qual `xerror` ± `xstd` não tem intersecção com o anterior.

```
cp     <- mod_bas$cptable[,1]
xerror <- mod_bas$cptable[,4]
xstd   <- mod_bas$cptable[,5]
tem    <- character(length(cp))
tem[1] <- "-"
for (linha in 2:length(cp)) {
    if ( (xerror[linha]+xstd[linha]) < (xerror[linha-1]-xstd[linha-1]) ) {
        tem[linha] <- "___não_tem___"
    } else { tem[linha] <- "tem     "}
}
print(data.frame(cp, "xerr_menos_xstd"=xerror-xstd, xerror, "xerr_mais_xstd"=xerror+xstd, tem_intersecção=
```

```
          cp xerr_menos_xstd    xerror xerr_mais_xstd tem_intersecção
1 0.28571429       0.9515660 1.0000000      1.0484340               -
2 0.25482625       0.8483948 0.8957529      0.9431110   ___não_tem___
3 0.11583012       0.4437603 0.4826255      0.5214906   ___não_tem___
4 0.03088803       0.3171530 0.3513514      0.3855497   ___não_tem___
5 0.02702703       0.2250690 0.2548263      0.2845836   ___não_tem___
6 0.01158301       0.1776769 0.2046332      0.2315895        tem
7 0.01000000       0.1307726 0.1544402      0.1781077        tem
```

A linha 7 diz que o limite superior `xerror`+`xstd`= 0,1781 é maior (possui intesercção) com o limite inferior da linha anterior (linha 6, na qual `xerror`-`xstd`= 0,1777). A linha 6 tem intersecção com a 5, pois `xerror`+`xstd`= 0,2316 e a linha 5 diz que `xerror`-`xstd`= 0,2251. A linha 5 tem limite superior em 0,2846 mas a anterior (linha 4), tem limite em 0,3172. Vamos estabelecer o valor de corte do `cp` abaixo de 0,027. O gráfico sugere entre 0,011 e 0,018. Estabeleceremos o corte em 0,015.

```
mod_bas_poda <- prune(mod_bas, cp=0.015)
mod_bas_poda
```

```
n= 660

node), split, n, loss, yval, (yprob)
      * denotes terminal node

  1) root 660 259 0 (0.60757576 0.39242424)
    2) salario< 10415.8 288  36 0 (0.87500000 0.12500000)
      4) educacao=superior 177   7 0 (0.96045198 0.03954802) *
      5) educacao=médio,pós-graduação 111  29 0 (0.73873874 0.26126126)
       10) educacao=médio 57   1 0 (0.98245614 0.01754386) *
       11) educacao=pós-graduação,superior 54  26 1 (0.48148148 0.51851852)
         22) credito< 7.05 22   0 0 (1.00000000 0.00000000) *
         23) credito≥7.05 32   4 1 (0.12500000 0.87500000) *
    3) salario≥10415.8 372 149 1 (0.40053763 0.59946237)
      6) educacao=médio 82   8 0 (0.90243902 0.09756098) *
      7) educacao=pós-graduação,superior 290  75 1 (0.25862069 0.74137931)
       14) civil=solteiro 86  28 0 (0.67441860 0.32558140)
         28) credito< 7.25 32   0 0 (1.00000000 0.00000000) *
         29) credito≥7.25 54  26 1 (0.48148148 0.51851852)
           58) educacao=superior 40  14 0 (0.65000000 0.35000000)
            116) salario< 15669.66 28   4 0 (0.85714286 0.14285714) *
```

```
            117) salario≥15669.66 12   2 1 (0.16666667 0.83333333) *
           59) educacao=médio,pós-graduação 14   0 1 (0.00000000 1.00000000) *
       15) civil=casado 204  17 1 (0.08333333 0.91666667)
         30) educacao=médio,pós-graduação 46  17 1 (0.36956522 0.63043478)
           60) credito< 7.05 18   1 0 (0.94444444 0.05555556) *
           61) credito≥7.05 28   0 1 (0.00000000 1.00000000) *
         31) educacao=superior 158   0 1 (0.00000000 1.00000000) *
```

```
printcp(mod_bas_poda)
```

```
Classification tree:
rpart(formula = emprest ~ civil + sexo + educacao + credito +
    salario, data = treino, method = "class", control = rpart.control(minsplit = 2,
    minbucket = 0))

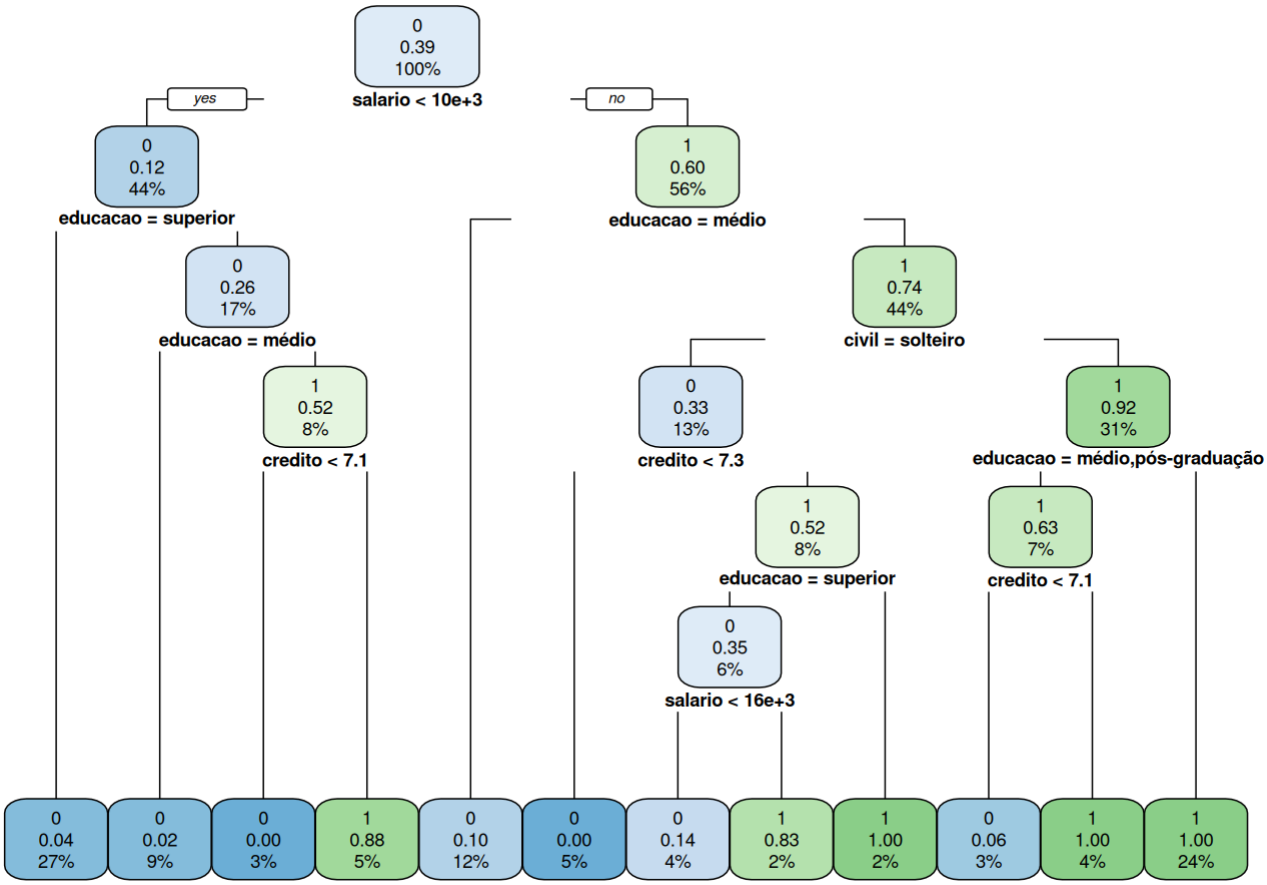
Variables actually used in tree construction:
[1] civil    credito  educacao salario

Root node error: 259/660 = 0.39242

n= 660

        CP nsplit rel error  xerror     xstd
1 0.285714      0   1.00000 1.00000 0.048434
2 0.254826      1   0.71429 0.89575 0.047358
3 0.115830      2   0.45946 0.48263 0.038865
4 0.030888      3   0.34363 0.35135 0.034198
5 0.027027      8   0.18919 0.25483 0.029757
6 0.015000     11   0.10425 0.20463 0.026956
```

```
grafdim(12,8)
rpart.plot(mod_bas_poda, cex=0.9)
```




```
predito <- predict(mod_bas_poda, valid, type="class")
print(summary(predito))
```

```
  0   1
108  62
```

Qualidade do modelo podado

Vamos avaliar a qualidade do modelo podado utilizando o conjunto de dados de validação:

```
matconf <- confusionMatrix(reference = valid$emprest, data=predito, positive="1" )
matconf
```

```
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 101   7
         1   2  60

               Accuracy : 0.9471
                 95% CI : (0.9019, 0.9755)
    No Information Rate : 0.6059
    P-Value [Acc > NIR] : <2e-16

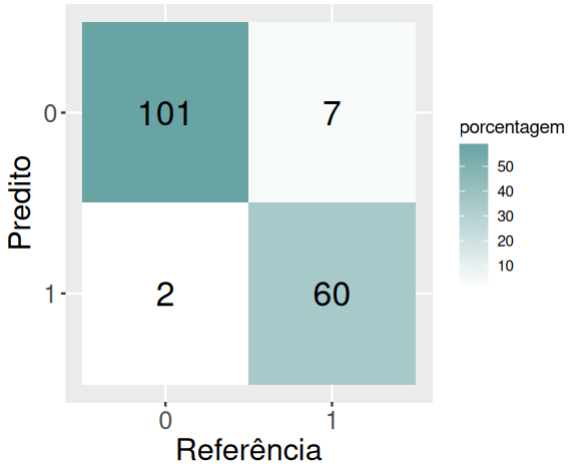
                  Kappa : 0.8877

 Mcnemar's Test P-Value : 0.1824
```

```
            Sensitivity : 0.8955
            Specificity : 0.9806
         Pos Pred Value : 0.9677
         Neg Pred Value : 0.9352
             Prevalence : 0.3941
         Detection Rate : 0.3529
   Detection Prevalence : 0.3647
      Balanced Accuracy : 0.9381

       'Positive' Class : 1
```

```
grafdim(5,4)
grafico_matriz_confusao(matconf)
```



A árvore passou de 13 divisões para 11 (ficou mais simples) e, para os dados de validação, apresentou uma acurácia ligeiramente (0,9381/0,9354 = 0,29%) maior.

Considerando então este o melhor modelo, vamos checar seu desempenho com o conjunto de teste:

```
predito <- predict(mod_bas_poda, teste, type="class")
print(summary(predito))
```

```
  0   1
109  61
```

```
matconf <- confusionMatrix(reference = teste$emprest, data=predito, positive="1" )
matconf
```

```
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 100   9
         1   3  58

               Accuracy : 0.9294
                 95% CI : (0.8799, 0.963)
    No Information Rate : 0.6059
    P-Value [Acc > NIR] : <2e-16
```

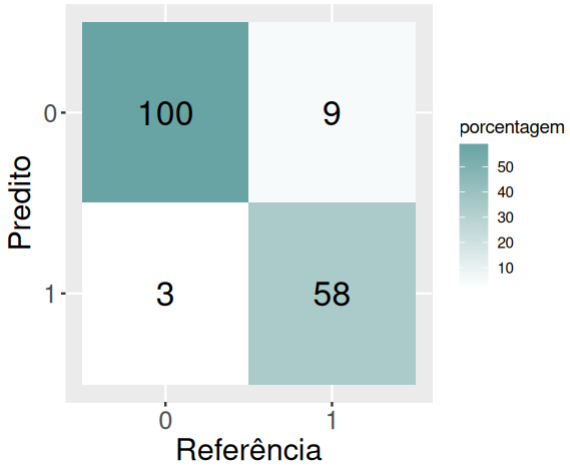
```
                 Kappa : 0.8498

 Mcnemar's Test P-Value : 0.1489

            Sensitivity : 0.8657
            Specificity : 0.9709
         Pos Pred Value : 0.9508
         Neg Pred Value : 0.9174
             Prevalence : 0.3941
         Detection Rate : 0.3412
   Detection Prevalence : 0.3588
      Balanced Accuracy : 0.9183

       'Positive' Class : 1
```

```
grafdim(5,4)
grafico_matriz_confusao(matconf)
```



A acurácia balanceada para os dados desconhecidos foi de 0,9183. Inferior àquelas apresentadas em treino ou validação, mas ainda razoável devido à flutuações dos casos.

Caso 1 - Sucesso Acadêmico

Referência: Predict students' dropout and academic success

```
apagar <- ls()
apagar <- apagar[! apagar %in% c('grafdim','grafico_matriz_confusao')]
rm(list=apagar)
```

```
library(rpart)  # árvores de decisão
library(repr)   # representações (ajuste gráfico do jupyter)
library(caret)  # matriz de confusão
library(rpart.plot)
library(RColorBrewer)
```

Dados

```
st <- read.csv(unzip("dad/Student_dropout_academic_success.zip","Student_dropout_academic_success.csv"),
                sep=",", header=TRUE)
file.remove("Student_dropout_academic_success.csv")
```

TRUE

```
print(head(st,10))
```

```
   Marital.status Application.mode Application.order Course
1               1                8                 5      2
2               1                6                 1     11
3               1                1                 5      5
4               1                8                 2     15
5               2               12                 1      3
6               2               12                 1     17
7               1                1                 1     12
8               1                9                 4     11
9               1                1                 3     10
10              1                1                 1     10
   Daytime.evening.attendance Previous.qualification Nacionality
1                           1                      1           1
2                           1                      1           1
3                           1                      1           1
4                           1                      1           1
5                           0                      1           1
6                           0                     12           1
7                           1                      1           1
8                           1                      1           1
9                           1                      1          15
10                          1                      1           1
   Mother.s.qualification Father.s.qualification Mother.s.occupation
1                      13                     10                    6
2                       1                      3                    4
3                      22                     27                   10
4                      23                     27                    6
5                      22                     28                   10
6                      22                     27                   10
7                      13                     28                    8
8                      22                     27                   10
9                       1                      1                   10
10                      1                     14                    5
   Father.s.occupation Displaced Educational.special.needs Debtor
1                   10         1                         0      0
2                    4         1                         0      0
3                   10         1                         0      0
4                    4         1                         0      0
5                   10         0                         0      0
6                    8         0                         0      1
7                   11         1                         0      0
8                   10         1                         0      0
9                   10         0                         0      0
10                   8         1                         0      1
   Tuition.fees.up.to.date Gender Scholarship.holder Age.at.enrollment
1                        1      1                  0                20
2                        0      1                  0                19
```

```
3                       0      1                 0                19
4                       1      0                 0                20
5                       1      0                 0                45
6                       1      1                 0                50
7                       1      0                 1                18
8                       0      1                 0                22
9                       1      0                 1                21
10                      0      0                 0                18
   International Curricular.units.1st.sem..credited.
1              0                                  0
2              0                                  0
3              0                                  0
4              0                                  0
5              0                                  0
6              0                                  0
7              0                                  0
8              0                                  0
9              1                                  0
10             0                                  0
   Curricular.units.1st.sem..enrolled. Curricular.units.1st.sem..evaluations.
1                                    0                                      0
2                                    6                                      6
3                                    6                                      0
4                                    6                                      8
5                                    6                                      9
6                                    5                                     10
7                                    7                                      9
8                                    5                                      5
9                                    6                                      8
10                                   6                                      9
   Curricular.units.1st.sem..approved. Curricular.units.1st.sem..grade.
1                                    0                          0.00000
2                                    6                         14.00000
3                                    0                          0.00000
4                                    6                         13.42857
5                                    5                         12.33333
6                                    5                         11.85714
7                                    7                         13.30000
8                                    0                          0.00000
9                                    6                         13.87500
10                                   5                         11.40000
   Curricular.units.1st.sem..without.evaluations.
1                                               0
2                                               0
3                                               0
4                                               0
5                                               0
6                                               0
7                                               0
8                                               0
9                                               0
10                                              0
   Curricular.units.2nd.sem..credited. Curricular.units.2nd.sem..enrolled.
1                                    0                                   0
2                                    0                                   6
3                                    0                                   6
4                                    0                                   6
```

```
5                                   0                                   6
6                                   0                                   5
7                                   0                                   8
8                                   0                                   5
9                                   0                                   6
10                                  0                                   6
   Curricular.units.2nd.sem..evaluations. Curricular.units.2nd.sem..approved.
1                                       0                                   0
2                                       6                                   6
3                                       0                                   0
4                                      10                                   5
5                                       6                                   6
6                                      17                                   5
7                                       8                                   8
8                                       5                                   0
9                                       7                                   6
10                                     14                                   2
   Curricular.units.2nd.sem..grade.
1                           0.00000
2                          13.66667
3                           0.00000
4                          12.40000
5                          13.00000
6                          11.50000
7                          14.34500
8                           0.00000
9                          14.14286
10                         13.50000
   Curricular.units.2nd.sem..without.evaluations. Unemployment.rate
1                                               0              10.8
2                                               0              13.9
3                                               0              10.8
4                                               0               9.4
5                                               0              13.9
6                                               5              16.2
7                                               0              15.5
8                                               0              15.5
9                                               0              16.2
10                                              0               8.9
   Inflation.rate   GDP   Target
1             1.4  1.74  Dropout
2            -0.3  0.79 Graduate
3             1.4  1.74  Dropout
4            -0.8 -3.12 Graduate
5            -0.3  0.79 Graduate
6             0.3 -0.92 Graduate
7             2.8 -4.06 Graduate
8             2.8 -4.06  Dropout
9             0.3 -0.92 Graduate
10            1.4  3.51  Dropout
```

Preparação das variáveis

Tabela 12: Variáveis do conjunto de dados *Student dropout academic success*

nome original	nome	descrição	tipo
Marital status	civil	estado civil do estudante	categórica
Application mode	inscricao	modo de inscrição	categórica
Application order	posicao	sequência de inscrição	numérica
Course	curso	curso escolhido	categórica
Daytime/evening attendance	periodo	diurno ou noturno	categórica
Previous qualification	qualif_ant	qualificação anterior ao ensiono superior	categórica
Nacionality	nacion	nacionalidade do estudante	categórica
Mother's qualification	qualif_mae	qualificação da mãe do estudante	categórica
Father's qualification	qualif_pai	qualificação do pai do estudante	categórica
Mother's occupation	prof_mae	profissão da mãe	categórica
Father's occupation	prof_pai	profissão do pai	categórica
Displaced	relocado	se o aluno vem de outra localidade	categórica
Educational special needs	nec_especial	se o estudante possui necessidades especiais	categórica
Debtor	devedor	se o estudante está em débito	categórica
Tuition fees up to date	pagto_em_dia	se as mensalidades estão em dia	categórica
Gender	sexo	sexo	categórica
Scholarship holder	bolsa	se o estudante possui bolsa	categórica
Age at enrollment	idade_matric	idade no ato da matrícula	numérica
International	intercamb	se o estudante é intercâmbio	categórica
Curricular units 1st sem (credited)	credit_1sem	créditos requisitados no 1º semestre	numérica
Curricular units 1st sem (enrolled)	matric_1sem	créditos matriculados no 1º semestre	numérica
Curricular units 1st sem (evaluations)	cumprid_1sem	créditos avaliados no 1º semestre	numérica
Curricular units 1st sem (approved)	aprov_1sem	créditos aprovados no 1º semestre	numérica
Curricular units 1st sem (grade)	graduad_1sem	créditos graduados no 1º semestre	numérica
Curricular units 1st sem (without evaluations)	navalid_1sem	unidades não avaliadas no 1º semestre	numérica
Curricular units 2nd sem (credited)	credit_2sem	créditos requisitados no 2º semestre	numérica
Curricular units 2nd sem (enrolled)	matric_2sem	créditos matriculados no 2º semestre	numérica
Curricular units 2nd sem (evaluations)	cumprid_2sem	créditos avaliados no 2º semestre	numérica
Curricular units 2nd sem (approved)	aprov_2sem	créditos aprovados no 2º semestre	numérica
Curricular units 2nd sem (grade)	graduad_2sem	créditos graduados no 2º semestre	numérica
Curricular units 2nd sem (without evaluations)	navalid_2sem	unidades não avaliadas no 2º semestre	numérica
Unemployment rate	tx_desemp	taxa de desemprego	numérica
Inflation rate	tx_inflacao	taxa de inflação	numérica
GDP	pib	produto interno bruto	numérica
target	graduou	terminou a graduação	categórica

Renomeando as variáveis:

```
names(st) <- c('civil','inscricao','posicao','curso',
        'periodo','qualif_ant','nacion',
        'qualif_mae','qualif_pai','prof_mae','prof_pai','relocado',
        'nec_especial','devedor','pagto_em_dia','sexo', 'bolsa','idade_matric','intercamb',
        'credit_1sem','matric_1sem','cumprid_1sem','aprov_1sem','graduad_1sem','navalid_1sem',
        'credit_2sem','matric_2sem','cumprid_2sem','aprov_2sem','graduad_2sem','navalid_2sem',
        'tx_desemp','tx_inflacao','pib','graduou')
```

Transformando em fatores e inteiros as variáveis categóricas e numéricas (respectivamente):

```r
grad <- within(st, {
    civil        <- as.integer(civil)
    inscricao    <- as.integer(inscricao)
    posicao      <- as.integer(posicao)
    curso        <- factor(curso)
    periodo      <- factor(periodo)
    qualif_ant   <- as.integer(qualif_ant)
    nacion       <- as.integer(nacion)
    qualif_mae   <- as.integer(qualif_mae)
    qualif_pai   <- as.integer(qualif_pai)
    prof_mae     <- as.integer(prof_mae)
    prof_pai     <- as.integer(prof_pai)
    relocado     <- factor(relocado)
    nec_especial <- factor(nec_especial)
    devedor      <- factor(devedor)
    pagto_em_dia <- factor(pagto_em_dia)
    sexo         <- factor(sexo)
    bolsa        <- factor(bolsa)
    idade_matric <- as.integer(idade_matric)
    intercamb    <- factor(intercamb)
    credit_1sem  <- as.integer(credit_1sem)
    matric_1sem  <- as.integer(matric_1sem)
    cumprid_1sem <- as.integer(cumprid_1sem)
    aprov_1sem   <- as.integer(aprov_1sem)
    graduad_1sem <- as.integer(graduad_1sem)
    navalid_1sem <- as.integer(navalid_1sem)
    credit_2sem  <- as.integer(credit_2sem)
    matric_2sem  <- as.integer(matric_2sem)
    cumprid_2sem <- as.integer(cumprid_2sem)
    aprov_2sem   <- as.integer(aprov_2sem)
    graduad_2sem <- as.integer(graduad_2sem)
    navalid_2sem <- as.integer(navalid_2sem)
    graduou[graduou == 'Dropout']  <- "evadiu"
    graduou[graduou == 'Graduate'] <- "graduou"
    graduou[graduou == 'Enrolled'] <- "matriculado"
    })
```

```r
summary(grad)
```

```
     civil         inscricao         posicao          curso      periodo
 Min.   :1.000   Min.   : 1.000   Min.   :0.000   12     : 766   0: 483
 1st Qu.:1.000   1st Qu.: 1.000   1st Qu.:1.000   9      : 380   1:3941
 Median :1.000   Median : 8.000   Median :1.000   10     : 355
 Mean   :1.179   Mean   : 6.887   Mean   :1.728   6      : 337
 3rd Qu.:1.000   3rd Qu.:12.000   3rd Qu.:2.000   15     : 331
 Max.   :6.000   Max.   :18.000   Max.   :9.000   14     : 268
                                                  (Other):1987
   qualif_ant         nacion         qualif_mae      qualif_pai
 Min.   : 1.000   Min.   : 1.000   Min.   : 1.00   Min.   : 1.00
 1st Qu.: 1.000   1st Qu.: 1.000   1st Qu.: 2.00   1st Qu.: 3.00
 Median : 1.000   Median : 1.000   Median :13.00   Median :14.00
 Mean   : 2.531   Mean   : 1.255   Mean   :12.32   Mean   :16.46
 3rd Qu.: 1.000   3rd Qu.: 1.000   3rd Qu.:22.00   3rd Qu.:27.00
 Max.   :17.000   Max.   :21.000   Max.   :29.00   Max.   :34.00

    prof_mae         prof_pai      relocado nec_especial devedor  pagto_em_dia
```

```
Min.   : 1.000   Min.   : 1.000   0:1998   0:4373       0:3921   0: 528
1st Qu.: 5.000   1st Qu.: 5.000   1:2426   1:  51       1: 503   1:3896
Median : 6.000   Median : 8.000
Mean   : 7.318   Mean   : 7.819
3rd Qu.:10.000   3rd Qu.:10.000
Max.   :32.000   Max.   :46.000

sexo     bolsa     idade_matric   intercamb  credit_1sem     matric_1sem
0:2868   0:3325   Min.   :17.00   0:4314    Min.   : 0.00   Min.   : 0.000
1:1556   1:1099   1st Qu.:19.00   1: 110    1st Qu.: 0.00   1st Qu.: 5.000
                  Median :20.00             Median : 0.00   Median : 6.000
                  Mean   :23.27             Mean   : 0.71   Mean   : 6.271
                  3rd Qu.:25.00             3rd Qu.: 0.00   3rd Qu.: 7.000
                  Max.   :70.00             Max.   :20.00   Max.   :26.000

 cumprid_1sem      aprov_1sem      graduad_1sem   navalid_1sem
Min.   : 0.000   Min.   : 0.000   Min.   : 0.0   Min.   : 0.0000
1st Qu.: 6.000   1st Qu.: 3.000   1st Qu.:11.0   1st Qu.: 0.0000
Median : 8.000   Median : 5.000   Median :12.0   Median : 0.0000
Mean   : 8.299   Mean   : 4.707   Mean   :10.3   Mean   : 0.1377
3rd Qu.:10.000   3rd Qu.: 6.000   3rd Qu.:13.0   3rd Qu.: 0.0000
Max.   :45.000   Max.   :26.000   Max.   :18.0   Max.   :12.0000

 credit_2sem       matric_2sem      cumprid_2sem      aprov_2sem
Min.   : 0.0000   Min.   : 0.000   Min.   : 0.000   Min.   : 0.000
1st Qu.: 0.0000   1st Qu.: 5.000   1st Qu.: 6.000   1st Qu.: 2.000
Median : 0.0000   Median : 6.000   Median : 8.000   Median : 5.000
Mean   : 0.5418   Mean   : 6.232   Mean   : 8.063   Mean   : 4.436
3rd Qu.: 0.0000   3rd Qu.: 7.000   3rd Qu.:10.000   3rd Qu.: 6.000
Max.   :19.0000   Max.   :23.000   Max.   :33.000   Max.   :20.000

 graduad_2sem     navalid_2sem       tx_desemp      tx_inflacao
Min.   : 0.000   Min.   : 0.0000   Min.   : 7.60   Min.   :-0.800
1st Qu.:10.000   1st Qu.: 0.0000   1st Qu.: 9.40   1st Qu.: 0.300
Median :12.000   Median : 0.0000   Median :11.10   Median : 1.400
Mean   : 9.909   Mean   : 0.1503   Mean   :11.57   Mean   : 1.228
3rd Qu.:13.000   3rd Qu.: 0.0000   3rd Qu.:13.90   3rd Qu.: 2.600
Max.   :18.000   Max.   :12.0000   Max.   :16.20   Max.   : 3.700

     pib                graduou
Min.   :-4.060000   Length   :4424
1st Qu.:-1.700000   N.unique :   3
Median : 0.320000   N.blank  :   0
Mean   : 0.001969   Min.nchar:   6
3rd Qu.: 1.790000   Max.nchar:  11
Max.   : 3.510000
```

Sendo a variável resposta (*target*), vamos prepará-la antes das demais, para que a avaliação das variáveis independentes já reflitam as operações.

`graduou`

Uma vez que deseja-se modelar se o estudante graduou ou não, aqueles que trancaram a matrículas (`graduou` = matriculado) não interessam, pois a situação não se resolveu para um caso ou outro. Vamos eliminá-los então:

```
unique(grad$graduou)
```

1. 'evadiu'
2. 'graduou'
3. 'matriculado'

```
grad <- grad[grad$graduou != 'matriculado', ]
unique(grad$graduou)
```

1. 'evadiu'
2. 'graduou'

```
grad$graduou <- factor(as.numeric(grad$graduou == 'graduou'))
print(summary(grad$graduou))
```

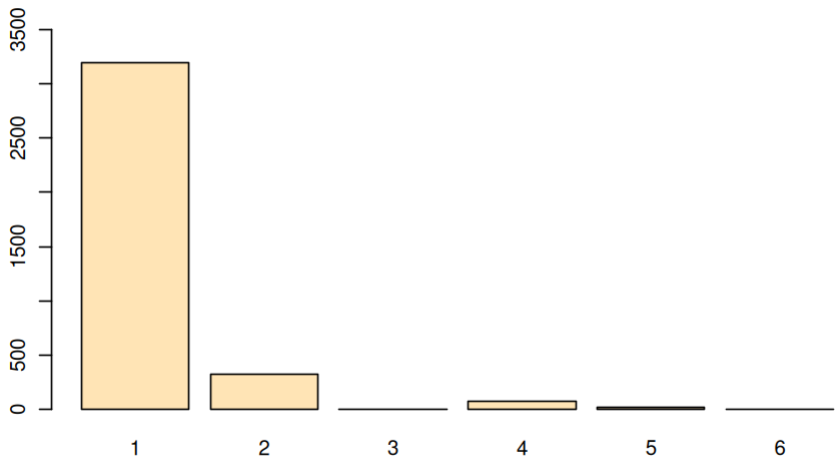
```
   0    1
1421 2209
```

civil

Vamos estudar a variável `civil`.

```
grafdim(8,5)
(tab <- table(grad$civil))
barplot(table(grad$civil), col="moccasin", ylim=c(0,3500))
```

```
   1    2    3    4    5    6
3199  327    2   75   22    5
```




```
3199/nrow(grad) * 100
```

88.1267217630854

Mais que 88% das observações de `civil` são do tipo 1; podemos supor que seja "solteiro/a".
Vamos considerar então apenas se é solteiro ou não.

```r
grad$civil <- factor(as.numeric(grad$civil == 1))
print(summary(grad$civil))
```

```
   0    1
 431 3199
```

```r
(cruzamento <- xtabs(~graduou + civil, data=grad))
```

```
       civil
graduou    0    1
      0  237 1184
      1  194 2015
```

inscricao

Analisando a `inscricao`:

```r
tab <- table(grad$inscricao)
print(tab)
df <- data.frame(tab)
names(df) <- c("tipo", "freq")
df$porc <- round(df$freq / nrow(grad) * 100, 1)
df <- df[order(-df$freq),]
head(df)
```

```
   1    2    3    4    5    6    7    8    9   10   11   12   13   14   15   16
1408    3    8  132   10   20   30  713  108    1    1  664   66  237  157   46
  17   18
  25    1
```

A data.frame: 6 × 3

	tipo <fct>	freq <int>	porc <dbl>
1	1	1408	38.8
8	8	713	19.6
12	12	664	18.3
14	14	237	6.5
15	15	157	4.3
4	4	132	3.6

O ideal seria a árvore avaliar variáveis categóricas com a menor quantidade de níveis possível. Vamos, como fizemos com `civil`, manter os mais frequentes e agrupar os demais.
Todos os métodos que não sejam 1, 8 ou 12 serão considerados como pertencentes da nova categoria 0.

```r
grad$inscricao[grad$inscricao != 1 &
               grad$inscricao != 8 &
               grad$inscricao != 12 ] <- 0
grad$inscricao <- factor(grad$inscricao, levels=c(1, 8, 12, 0), ordered=TRUE)
print(summary(grad$inscricao))
```

```
   1    8   12    0
1408  713  664  845
```

posicao

Vamos analisar a `posicao`:

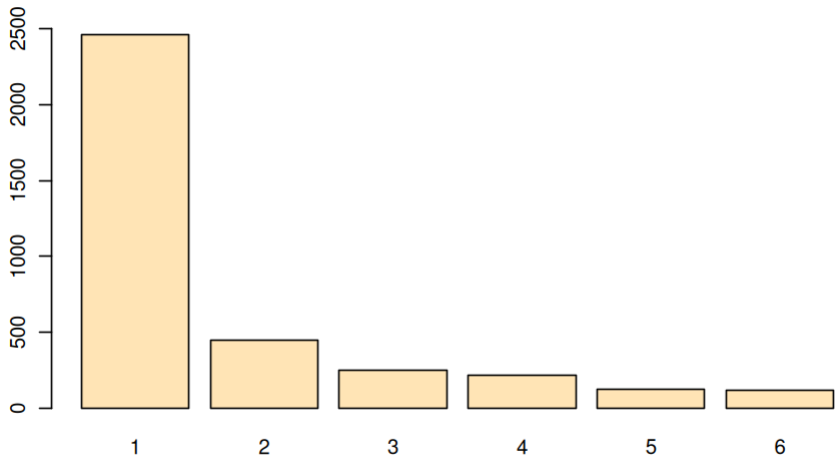
```
tab <- table(grad$posicao)
print(tab)
```

```
   0    1    2    3    4    5    6
   1 2461  451  249  218  129  121
```

Os valores parecem ser algum tipo de classificação, exceto o 0. O valor de 9 (uma observação) também parece anormal, não existindo 7 ou 8.
As observações para `posicao` 0 e 9 serão excluídas.

```
grad <- grad[grad$posicao !=  0 &
             grad$posicao !=  9, ]
```

```
barplot(table(grad$posicao), col="moccasin", ylim=c(0,2500))
```



curso

Analisando o comportamento dos valores para a variável `curso` conforme a aprovação ou não:

```
(cruzamento <- xtabs(~graduou + curso, data=grad[grad$graduou != "matriculado",]))
```

```
       curso
graduou   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17
      0   8  82  71  86  51  90  92  78 134  65  96 118  33  95 101  85 136
      1   1  96 123  86 133 172  14  42 138 248 115 548  36 125 196  57  78
```

Parece haver um problema de coleta de observações pois alguns cursos (3, 10, 11, 12 etc) possuem uma proporção de evadidos, o que é razoável, porém outros cursos (8, 16 e 17) possuem mais evadidos que graduados. Uma vez que a variável é categórica e não há explicação para seus valores - a fim de algum tipo de consolidação (exatas / biológicas / humanas, por exemplo) - será descartada.

```
grad$curso <- NULL
```

periodo

```
(cruzamento <- xtabs(~graduou + periodo, data=grad[grad$graduou =/= "matriculado",]))
chisq.test(cruzamento)
```

```
       periodo
graduou    0    1
      0  207 1214
      1  201 2007


    Pearson's Chi-squared test with Yates' continuity correction

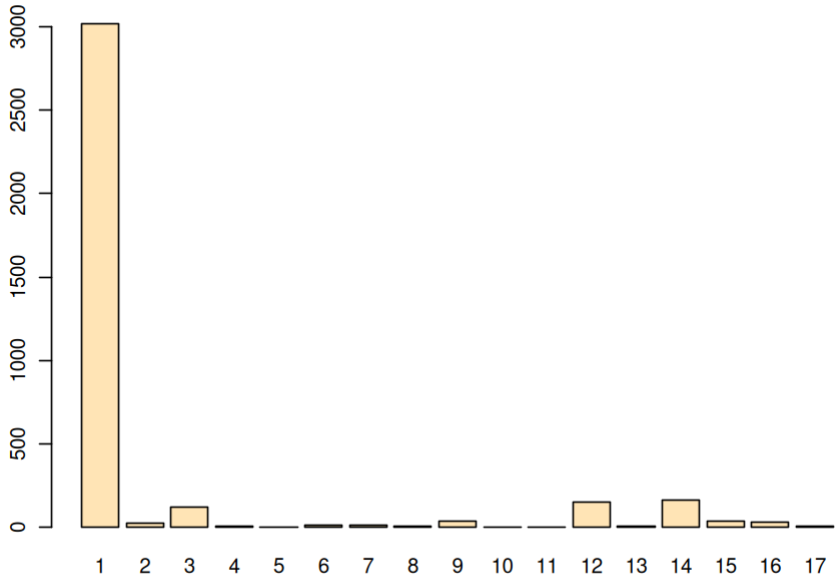
data:  cruzamento
X-squared = 25.322, df = 1, p-value = 4.851e-07
```

A variável `periodo` já está adequada e há signficância estatística para afirmar que a relação dos níveis de uma variável não é independente (proporcional) dentre os níveis da outra. Levando em consideração apenas `periodo` e `graduou`, o comportamento entre elas não é proporcional (seria improvável ser proporcional e obter esses valores).

qualif_ant

```
grafdim(8,6)
(tab <- table(grad$qualif_ant))
barplot(table(grad$qualif_ant), col="moccasin", ylim=c(0,3000))
```

```
   1    2    3    4    5    6    7    8    9   10   11   12   13   14   15   16
3019   22  121    7    1   15   11    4   39    1    2  149    6  164   34   28
  17
   6
```



```
3019/nrow(grad) * 100
```

83.1909616974373

83% de `qualif_ant` é `1`. Vamos considerar a `qualif_ant` como `1` ou não `1`:

```
grad$qualif_ant <- factor(as.numeric(grad$qualif_ant == 1))
```

```
(cruzamento <- xtabs(~graduou + qualif_ant, data=grad[grad$graduou != "matriculado",]))
chisq.test(cruzamento)
```

```
       qualif_ant
graduou    0    1
      0  343 1078
      1  267 1941
```

```
    Pearson's Chi-squared test with Yates' continuity correction

data:  cruzamento
X-squared = 88.85, df = 1, p-value < 2.2e-16
```

nacion

```
(cruzamento <- xtabs(~graduou + nacion, data=grad[grad$graduou != "matriculado",]))
```

```
       nacion
graduou    1    2    3    4    5    6    7    8    9   10   11   12   14   15
      0 1389    0    4    0    0    0    1    1    4    1    0    1   14    0
      1 2154    2    4    3    1    1    0    0    8    4    1    8   18    1
       nacion
graduou   16   17   18   19   21
      0    2    1    1    1    1
      1    0    1    2    0    0
```

```
3543/nrow(grad) * 100
```

97.6302011573436

97% de `nacion` é `1`, que deve ser nativo. Vamos considerar a `nacion` como `1` ou não `1`:

```
grad$nacion <- factor(as.numeric(grad$nacion == 1))
```

```
print(summary(grad$nacion))
```

```
   0    1
  86 3543
```

qualif_mae

```
tab <- table(grad$qualif_mae)
print(tab)
df <- data.frame(tab)
names(df) <- c("tipo", "freq")
df$porc <- round(df$freq / nrow(grad) * 100, 1)
df <- df[order(-df$freq),]
head(df)
```

```
  1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20
864  60 337  34  16   4   8   3   3  39   2   1 777   1   1   1   3   3 127   3
 21  22  23  24  25  26  27  28  29
  3 840 471   6   8   6   4   3   1
```

A data.frame: 6 × 3

	tipo <fct>	freq <int>	porc <dbl>
1	1	864	23.8
22	22	840	23.1
13	13	777	21.4
23	23	471	13.0
3	3	337	9.3
19	19	127	3.5

```
qm <- grad$qualif_mae
outros <- qm != 1 & qm != 22 & qm != 13 & qm != 23
qm[outros] <- 0
qm <- factor(qm, levels=c(1,13,22,23,0), ordered=TRUE)
grad$qualif_mae <- qm
rm(qm)
print(summary(grad$qualif_mae))
```

```
  1  13  22  23   0
864 777 840 471 677
```

qualif_pai

```
tab <- table(grad$qualif_pai)
print(tab)
df <- data.frame(tab)
names(df) <- c("tipo", "freq")
df$porc <- round(df$freq / nrow(grad) * 100, 1)
df <- df[order(-df$freq),]
head(df)
```

```
   1    2    3    4    5    6    7    8    9   10   11   12   13   14   15   16
 732   43  220   32   15    2    5    2   10   34    1    4    1  785    1    4
  17   18   19   20   21   22   23   24   25   26   27   28   29   30   31   32
   1    2    1    3    3    1    1  109    2    8 1009  575   13    5    1    1
  33   34
   2    1
```

A data.frame: 6 × 3

	tipo <fct>	freq <int>	porc <dbl>
27	27	1009	27.8
14	14	785	21.6
1	1	732	20.2
28	28	575	15.8
3	3	220	6.1
24	24	109	3.0

```
qp <- grad$qualif_pai
outros <- qp != 27 & qp != 14 & qp != 1 & qp != 28
qp[outros] <- 0
qp <- factor(qp, levels=c(1,14,27,28,0), ordered=TRUE)
grad$qualif_pai <- qp
rm(qp)
print(summary(grad$qualif_pai))
```

```
   1   14   27   28    0
 732  785 1009  575  528
```

prof_mae

```
tab <- table(grad$prof_mae)
print(tab)
df <- data.frame(tab)
names(df) <- c("tipo", "freq")
df$porc <- round(df$freq / nrow(grad) * 100, 1)
df <- df[order(-df$freq),]
head(df)
```

```
   1    2    3    4    5    6    7    8    9   10   11   12   13   15   16   18
 143   87  240  272  669  436   77  224   29 1313    2   70   15    5    1    1
  19   20   21   22   23   24   25   26   28   29   30   31   32
   2    4    1    4    2    1    2    1    3   15    2    4    4
```

A data.frame: 6 × 3

	tipo <fct>	freq <int>	porc <dbl>
10	10	1313	36.2
5	5	669	18.4
6	6	436	12.0
4	4	272	7.5
3	3	240	6.6
8	8	224	6.2

```
pmae <- grad$prof_mae
outros <- pmae != 10 & pmae != 5 & pmae != 6
pmae[outros] <- 0
pmae <- factor(pmae, levels=c(5,6,10,0), ordered=TRUE)
grad$prof_mae <- pmae
rm(pmae)
print(summary(grad$prof_mae))
```

```
   5    6   10    0
 669  436 1313 1211
```

prof_pai

```
tab <- table(grad$prof_pai)
print(tab)
df <- data.frame(tab)
names(df) <- c("tipo", "freq")
df$porc <- round(df$freq / nrow(grad) * 100, 1)
df <- df[order(-df$freq),]
head(df,10)
```

```
  1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  20  21
125 111 147 321 312 409 194 555 263 846 212  65  17   1   1   3   1   1   1   1
 22  23  26  27  28  29  30  31  32  33  35  36  37  38  39  40  41  42  43  44
  1   1   2   1   1   5   2   1   1   1   2   3   1   1   2   2   1   1   4   7
 45  46
  2   1
```

A data.frame: 10 × 3

	tipo <fct>	freq <int>	porc <dbl>
10	10	846	23.3
8	8	555	15.3
6	6	409	11.3
4	4	321	8.8
5	5	312	8.6
9	9	263	7.2
11	11	212	5.8
7	7	194	5.3
3	3	147	4.1
1	1	125	3.4

```
pp <- grad$prof_pai
outros <- pp != 10 & pp != 8 & pp != 6 & pp != 5 & pp != 4
pp[outros] <- 0
pp <- factor(pp, levels=c(4,5,6,8,10,0), ordered=TRUE)
grad$prof_pai <- pp
rm(pp)
print(summary(grad$prof_pai))
```

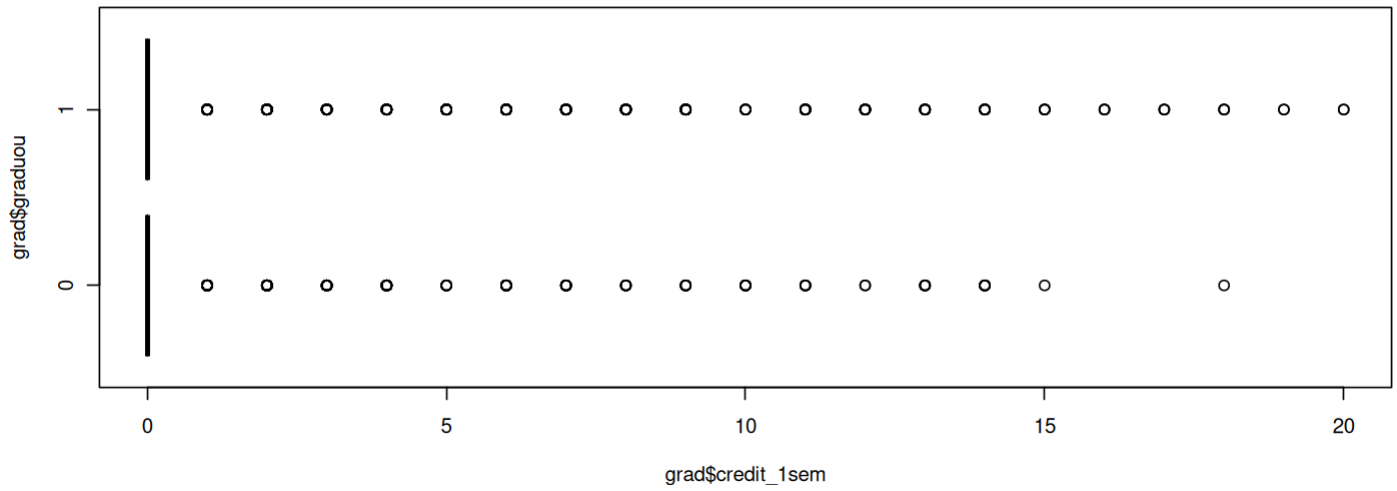
```
   4    5    6    8   10    0
 321  312  409  555  846 1186
```

relocado, nec_especial, devedor, pagto_em_dia, sexo, bolsa e intercamb

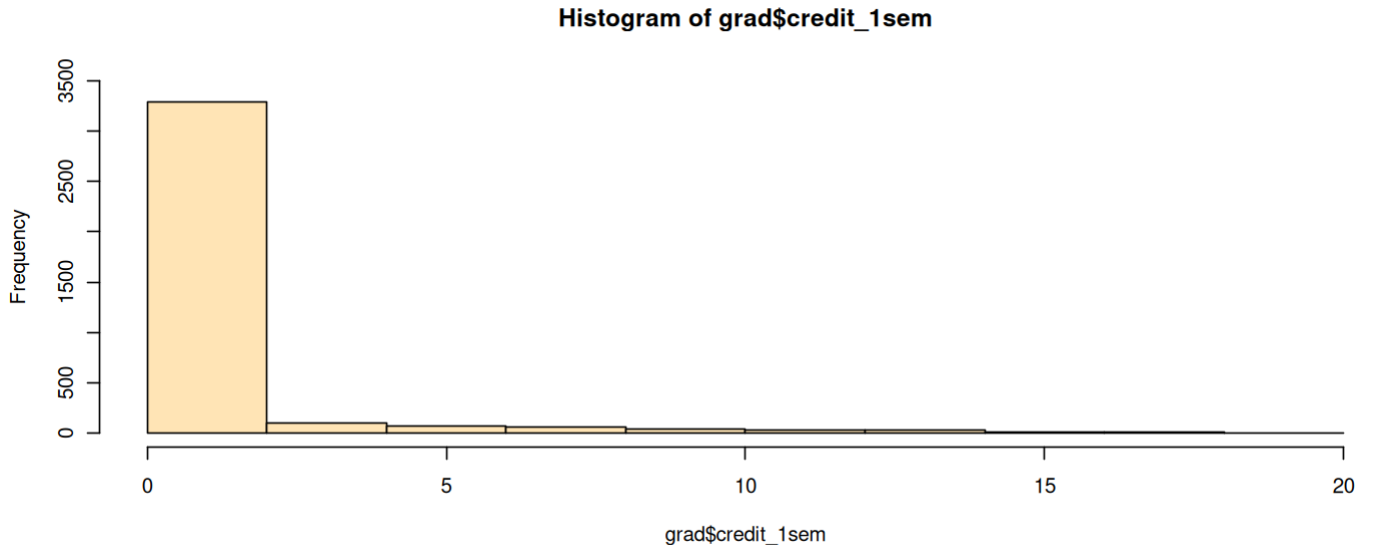
As variáveis `relocado`, `nec_especial`, `devedor`, `pagto_em_dia`, `sexo`, `bolsa` e `intercamb` são bastante claras e já estão codificadas como 0 ou 1.

credit_1sem

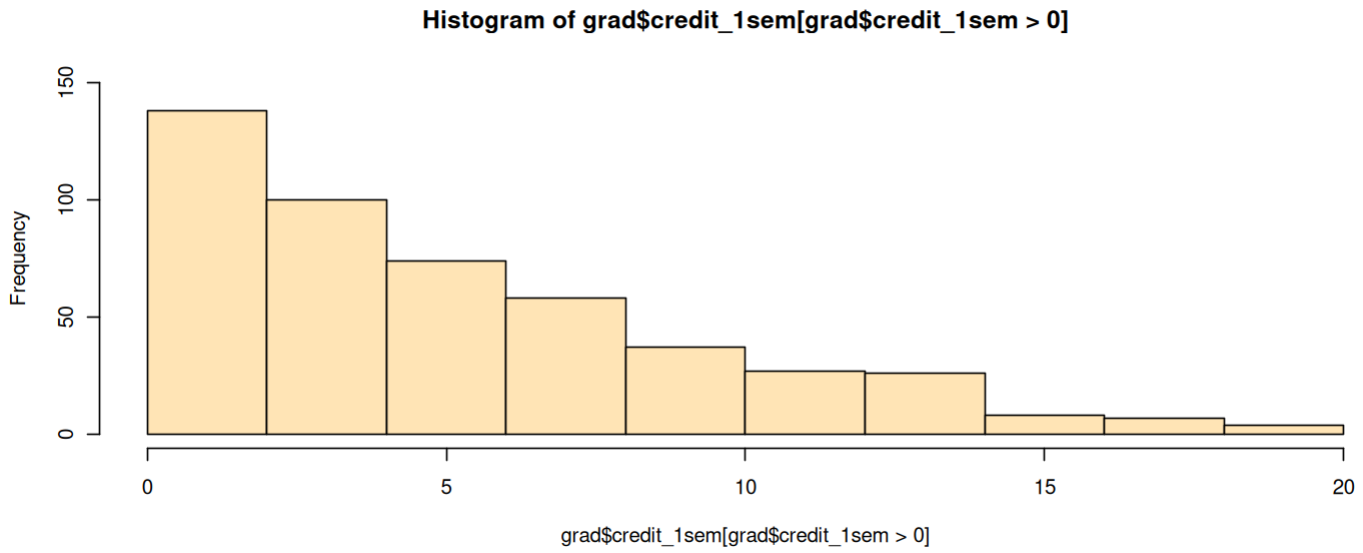
```r
grafdim(12,5)
boxplot(grad$credit_1sem ~ grad$graduou, horizontal=TRUE)
```




```r
hist(grad$credit_1sem, ylim=c(0,3500), col="moccasin")
```




```r
hist(grad$credit_1sem[grad$credit_1sem>0], ylim=c(0,150), col="moccasin")
```




```
sum(grad$credit_1sem == 0)
```

3150

A grande maioria dos valores de `credit_1sem` é 0. Há uma distribuição que vai decaindo de 170 a partir de 1 até 20.

```
table(grad$graduou,grad$credit_1sem)
```

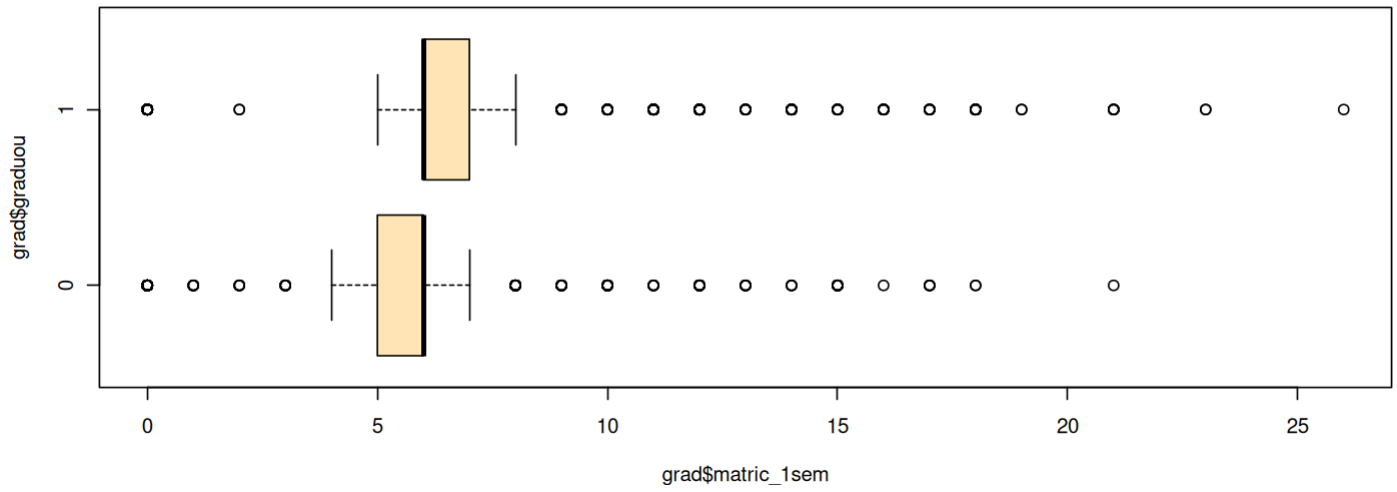
```
     0    1    2    3    4    5    6    7    8    9   10   11   12   13   14
  0 1242   29   38   21   24    6   14    9    5    6    7    5    2    5    6
  1 1908   38   33   36   19   25   29   22   22   18    6   11    9    7    8

     15   16   17   18   19   20
  0    1    0    0    1    0    0
  1    4    3    3    3    2    2
```

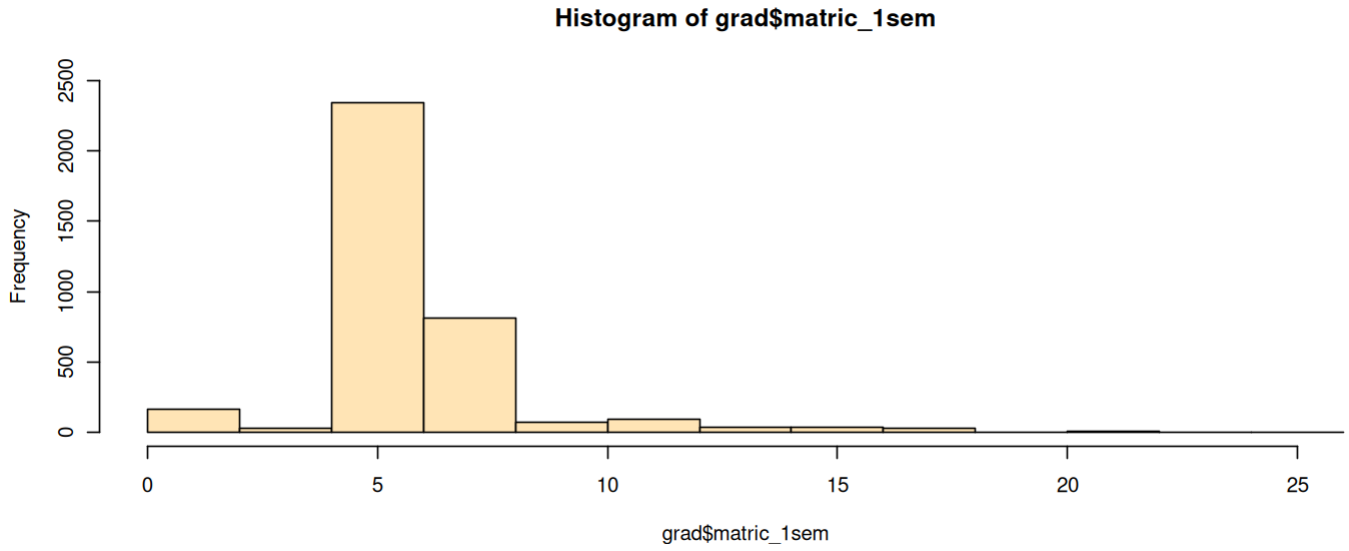
Parece haver uma prevalência em haver mais graduado que evadido conforme a quantidade de créditos aumenta. Poderia ser apenas flutuação.

matric_1sem

```
grafdim(12,5)
boxplot(grad$matric_1sem ~ grad$graduou, horizontal=TRUE, col="moccasin")
```




```
hist(grad$matric_1sem, ylim=c(0,2500), col="moccasin")
```



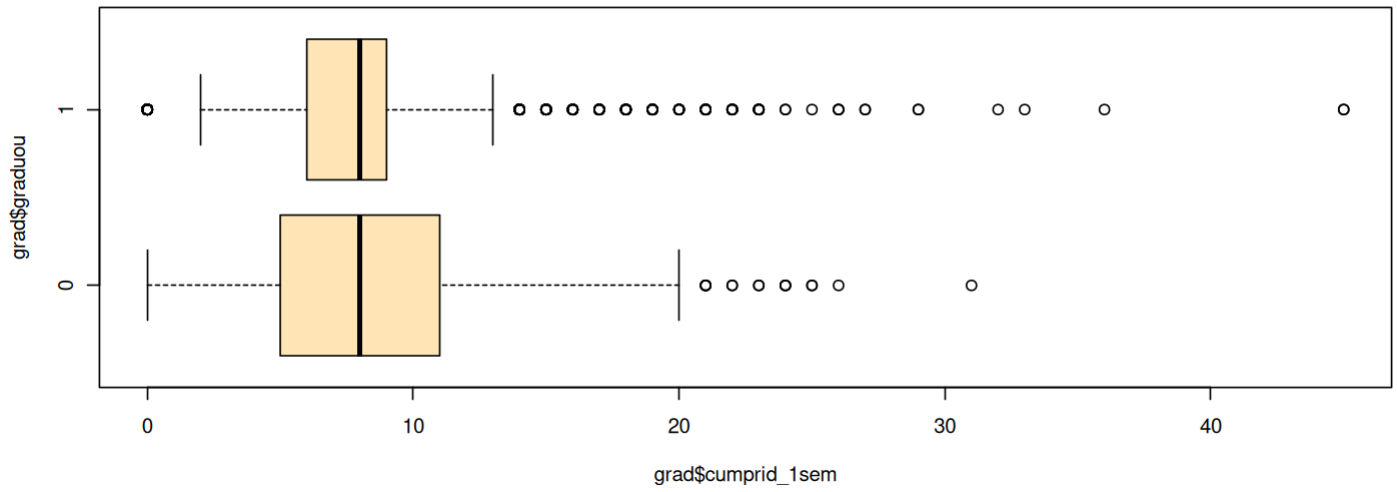

```
table(grad$graduou,grad$matric_1sem)
```

```
   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
0 77   6   5   9  21 421 605 164  26  12  24   5  15   6   3  14   1   4   2
1 75   0   2   0   0 328 989 382 238  19  19  32  39  10  19  10  11   8  17

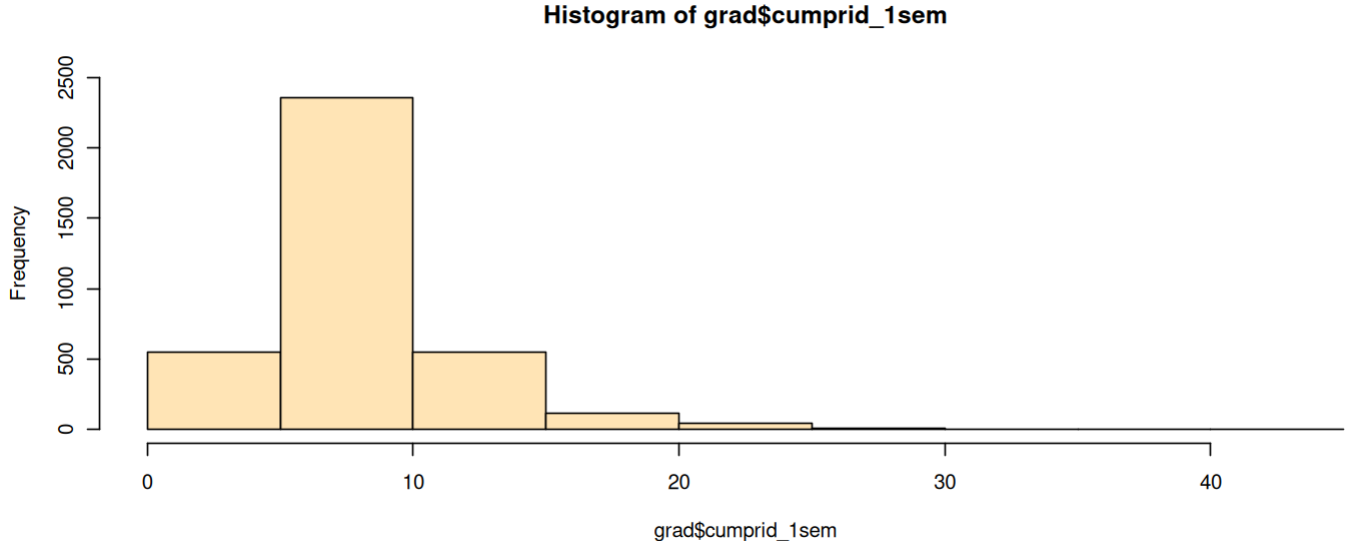
   19  21  23  26
0   0   1   0   0
1   2   5   2   1
```

cumprid_1sem

```
grafdim(12,5)
boxplot(grad$cumprid_1sem ~ grad$graduou, horizontal=TRUE, col="moccasin")
```




```
hist(grad$cumprid_1sem, ylim=c(0,2500), col="moccasin")
```



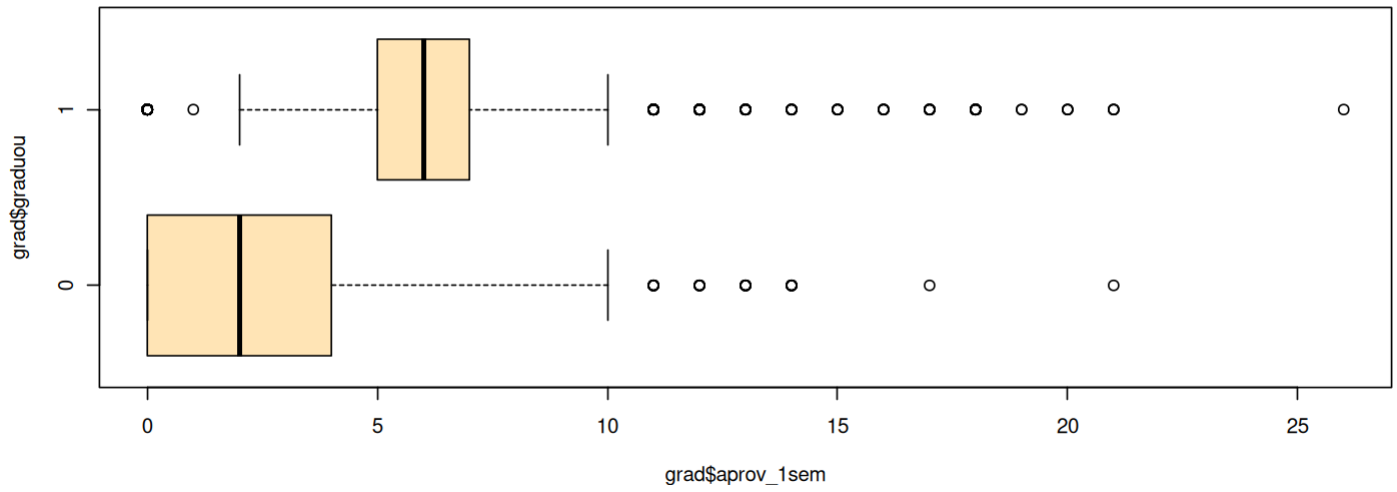

```
table(grad$graduou,grad$cumprid_1sem)
```

```
     0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
  0 246   5   5   6  17 110 137 135 150 119 105  79  94  63  46  29  25  15  11
  1  75   0   1   0   1  83 393 486 524 177 130  83  66  41  31  20  12  14  11

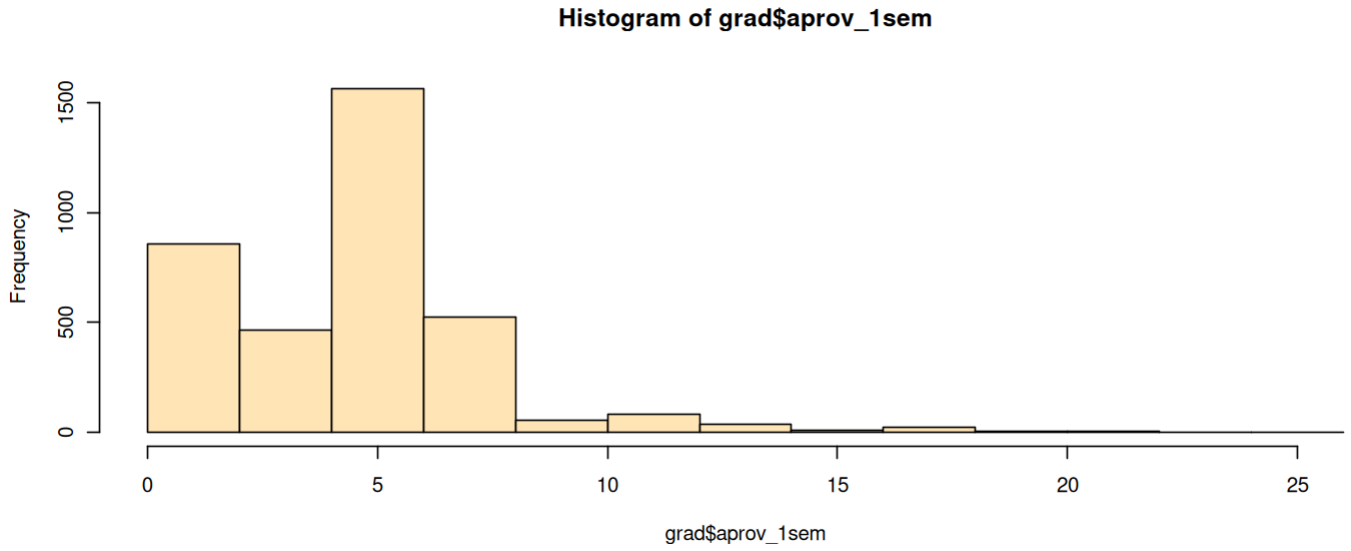
    19  20  21  22  23  24  25  26  27  29  31  32  33  36  45
  0   5   4   4   2   2   3   2   1   0   0   1   0   0   0   0
  1  13   6  11   8   7   2   1   3   2   2   0   1   1   1   2
```

aprov_1sem

```
grafdim(12,5)
boxplot(grad$aprov_1sem ~ grad$graduou, horizontal=TRUE, col="moccasin")
```




```
hist(grad$aprov_1sem, ylim=c(0,1600), col="moccasin")
```



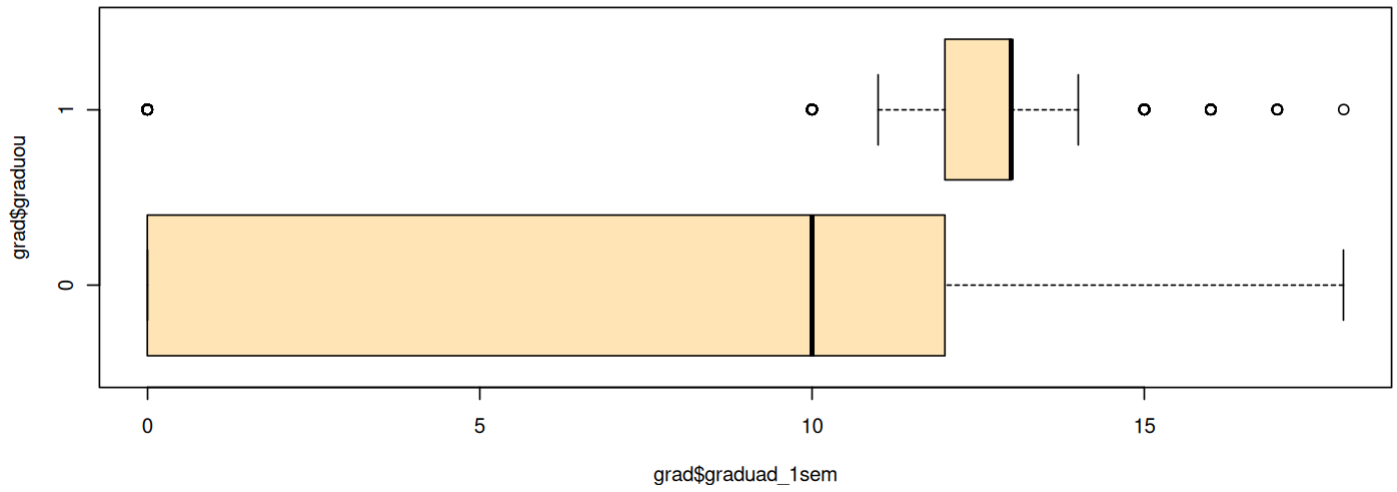

```
table(grad$graduou,grad$aprov_1sem)
```

```
     0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
  0 570  93 112 139 158 145 117  33  10   8   5   8   6   8   7   0   0   1   0
  1  77   1   6  37 130 385 916 396  84  27  16  34  34  15   7   6   5   8  15

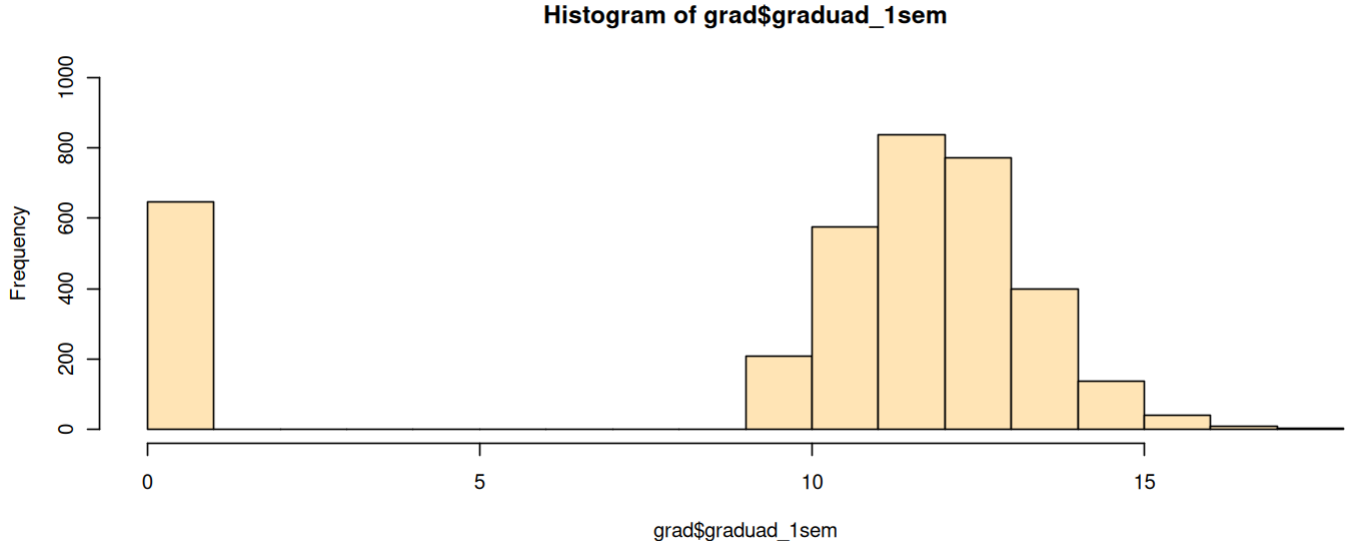
    19  20  21  26
  0   0   0   1   0
  1   2   3   3   1
```

graduad_1sem

```
grafdim(12,5)
boxplot(grad$graduad_1sem ~ grad$graduou, horizontal=TRUE, col="moccasin")
```




```
hist(grad$graduad_1sem, ylim=c(0,1000), col="moccasin")
```



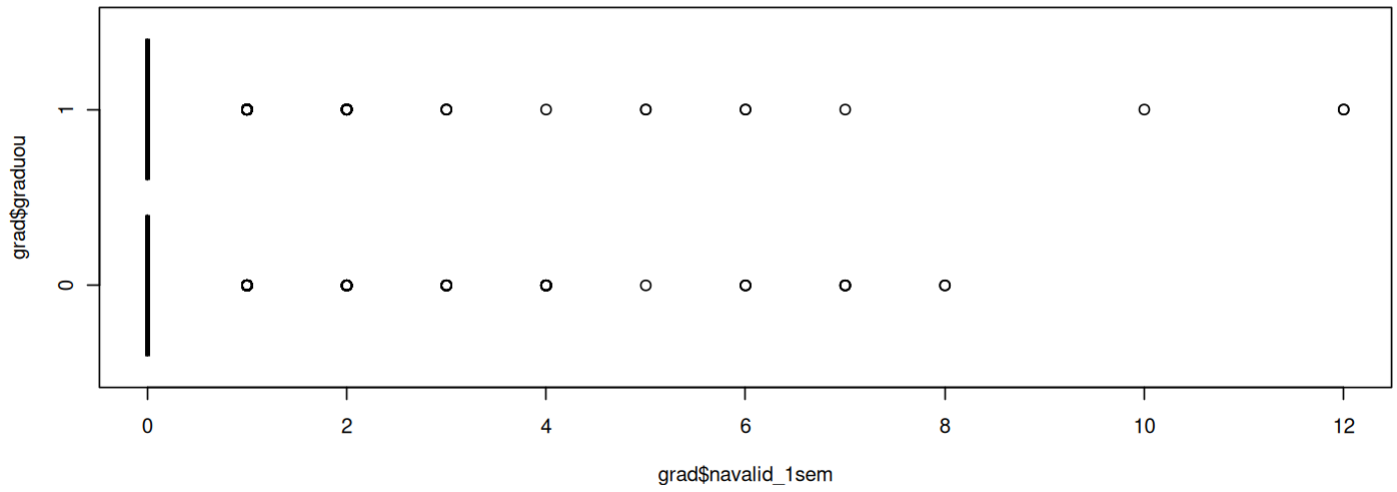

```
table(grad$graduou,grad$graduad_1sem)
```

```
     0   9  10  11  12  13  14  15  16  17  18
  0 570   1 140 254 240 138  46  17  12   1   2
  1  77   0  69 322 597 634 353 120  28   7   1
```

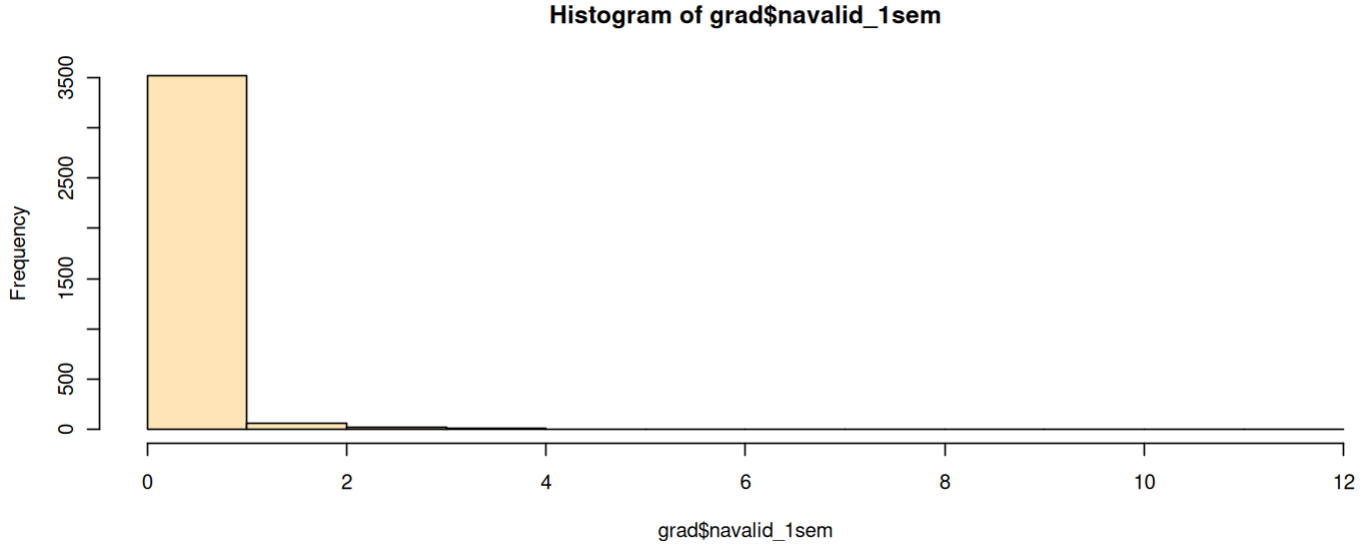
Um grande indicador se graduou ou evadiu parece ser `graduad_1sem`, na qual a maioria dos valores abaixo de 12 evadiram.

navalid_1sem

```
grafdim(12,5)
boxplot(grad$navalid_1sem ~ grad$graduou, horizontal=TRUE, col="moccasin")
```




```
hist(grad$navalid_1sem, ylim=c(0,3500), col="moccasin")
```

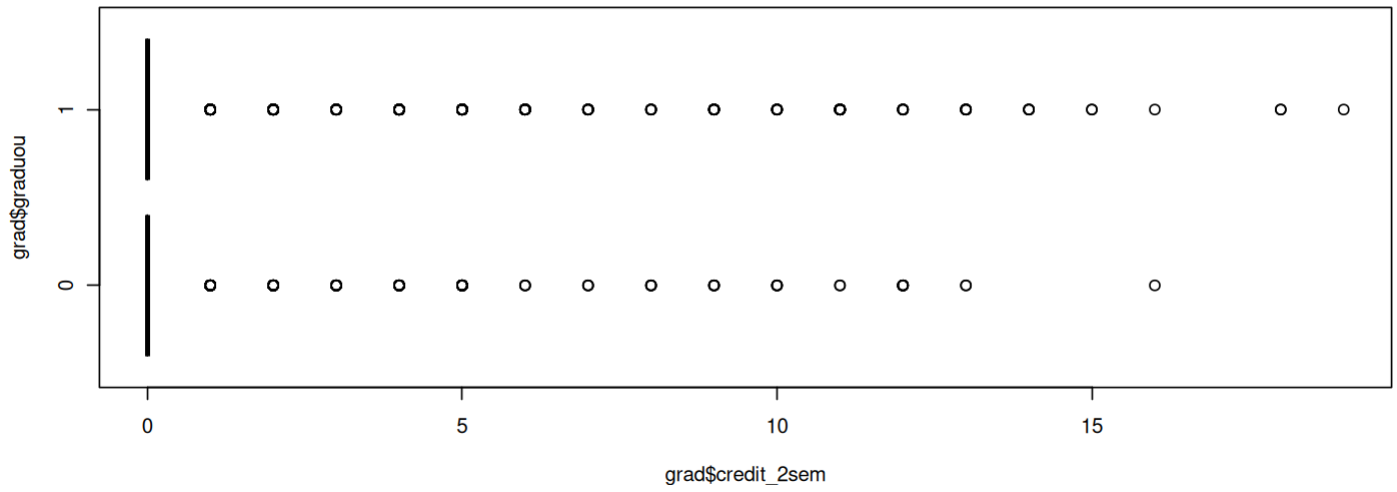



```
table(grad$graduou,grad$navalid_1sem)
```

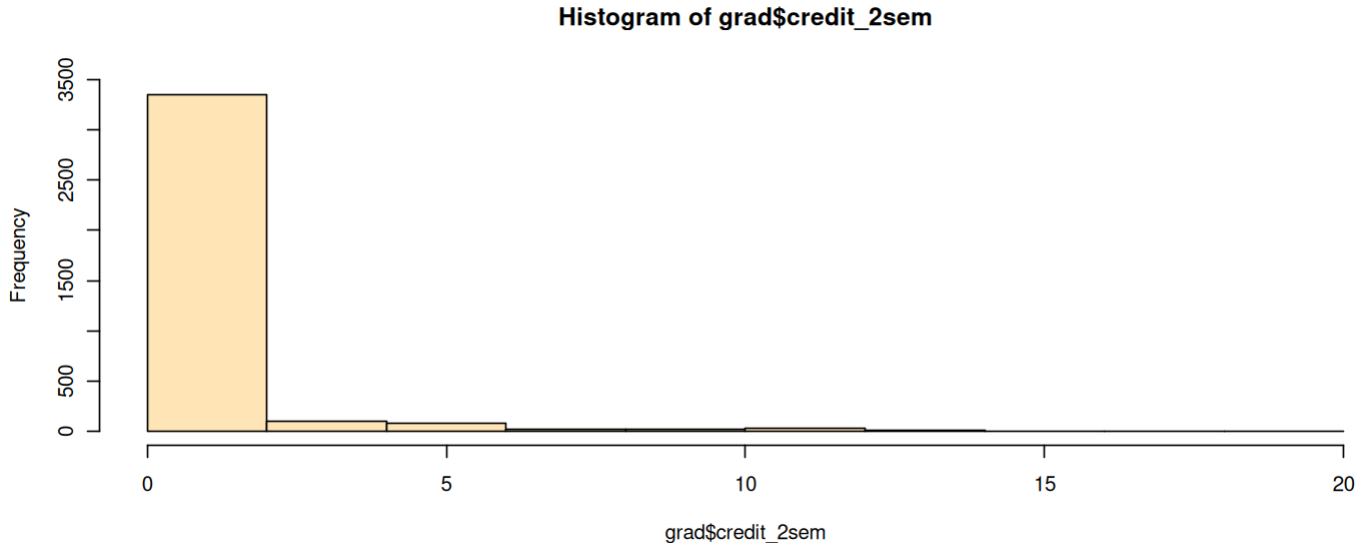
```
      0    1    2    3    4    5    6    7    8   10   12
  0 1297   60   29   12   13    1    3    4    2    0    0
  1 2109   58   28    4    1    2    2    1    0    1    2
```

credit_2sem

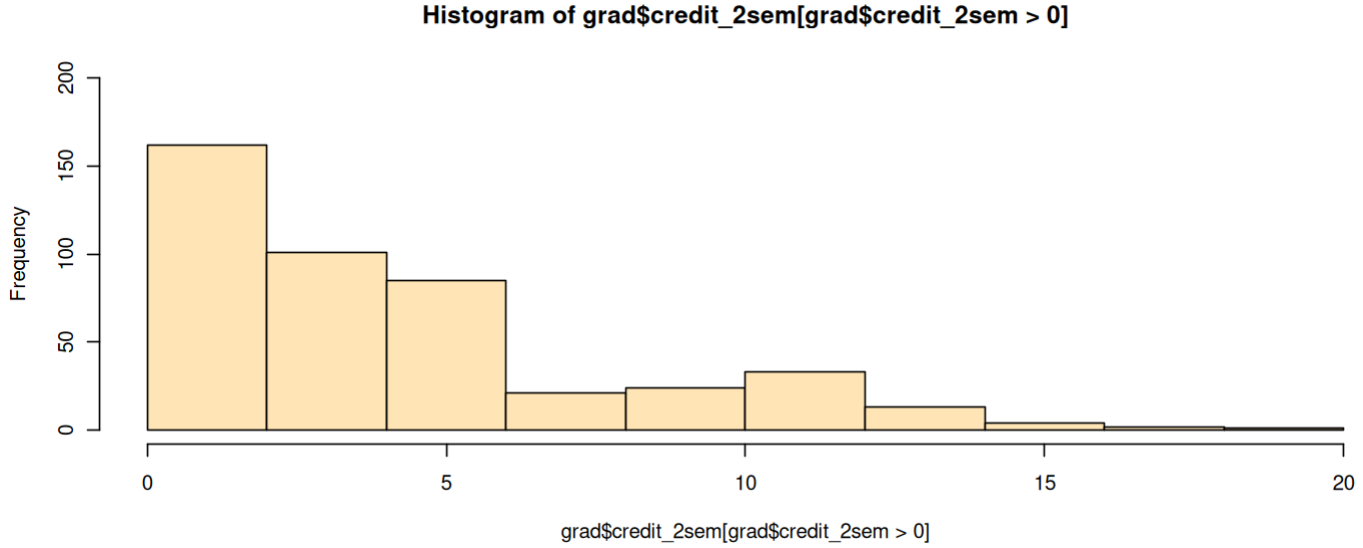
```
grafdim(12,5)
boxplot(grad$credit_2sem ~ grad$graduou, horizontal=TRUE, col="moccasin")
```




```
hist(grad$credit_2sem, ylim=c(0,3500), col="moccasin")
```




```
hist(grad$credit_2sem[grad$credit_2sem>0], ylim=c(0,200), col="moccasin")
```



```
sum(grad$credit_2sem == 0)
```

3183

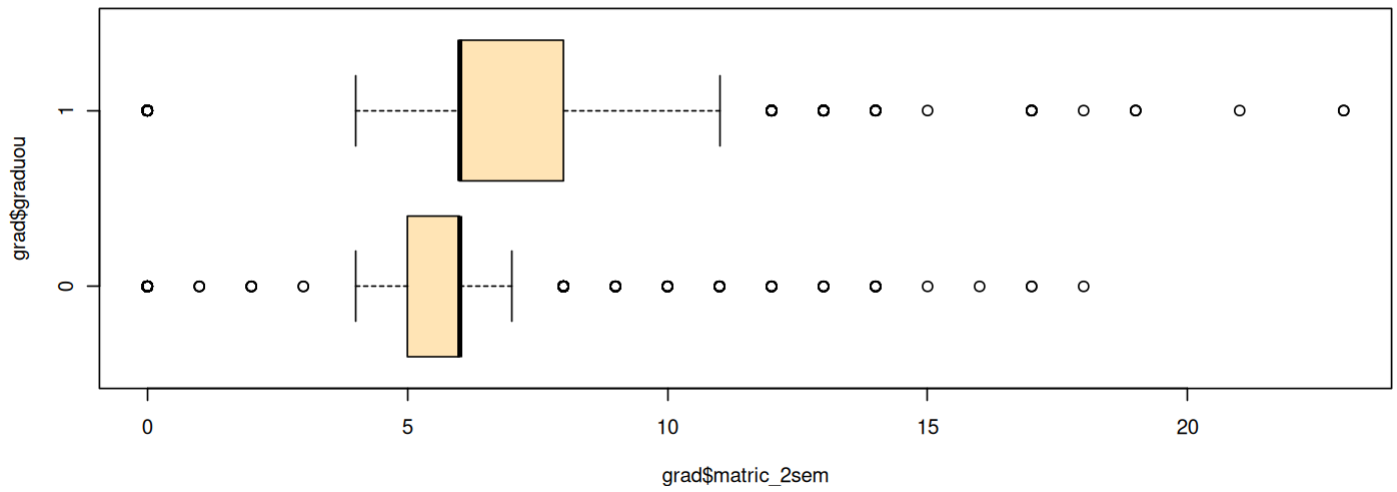
```
table(grad$graduou,grad$credit_2sem)
```

```
     0    1    2    3    4    5    6    7    8    9   10   11   12   13   14
  0 1264   37   30   17   19   24    3    3    4    4    4    2    7    2    0
  1 1919   45   50   23   42   37   21    9    5    9    7   18    6    7    4

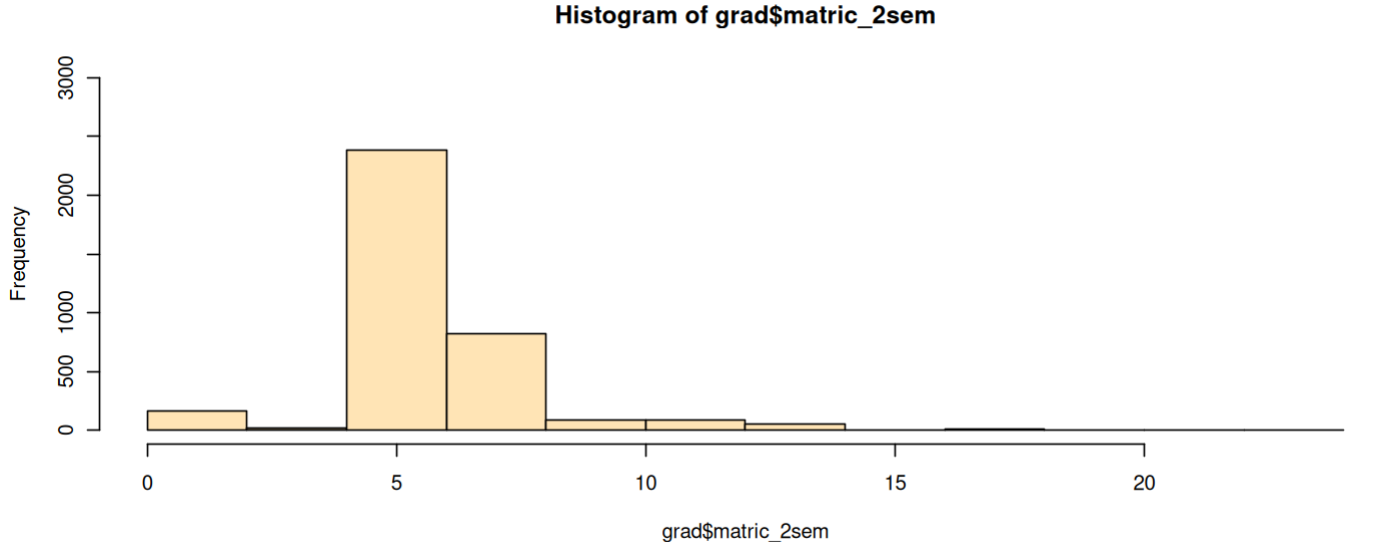
     15   16   18   19
  0    0    1    0    0
  1    2    1    2    1
```

matric_2sem

```
grafdim(12,5)
boxplot(grad$matric_2sem ~ grad$graduou, horizontal=TRUE, col="moccasin")
```




```
hist(grad$matric_2sem, ylim=c(0,3000), col="moccasin")
```



```
table(grad$graduou,grad$matric_2sem)
```

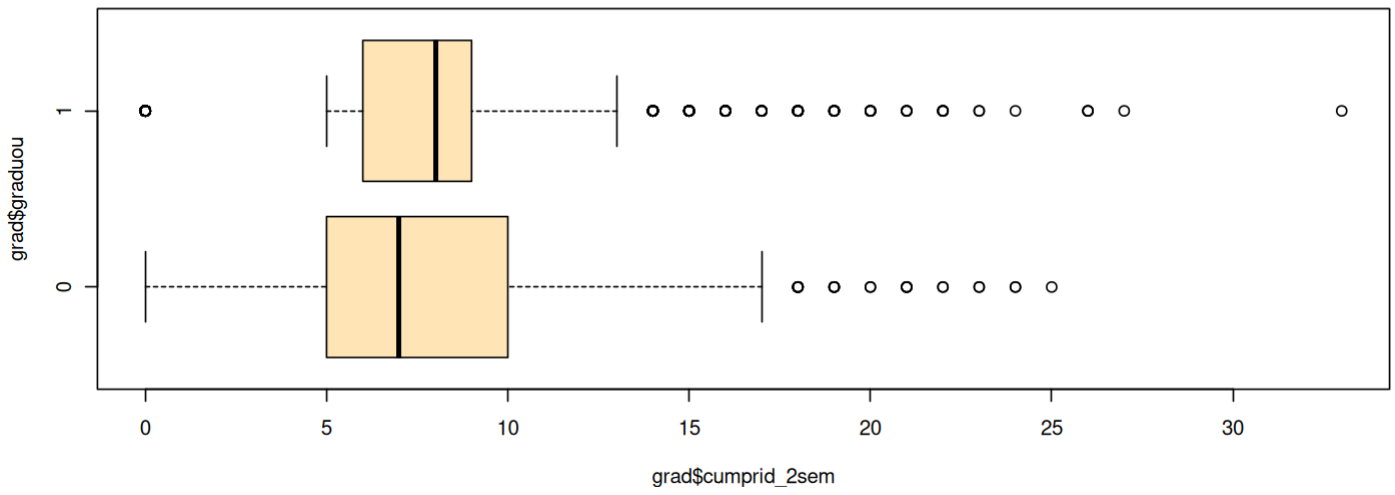
```
   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
0 77   3   5   2  14 454 602  99  95  16  12  12   9   7   9   1   1   2   1
1 75   0   0   0   1 335 994 125 501  28  26  38  29  27  13   1   0   8   1

   19  21  23
0   0   0   0
1   3   1   2
```

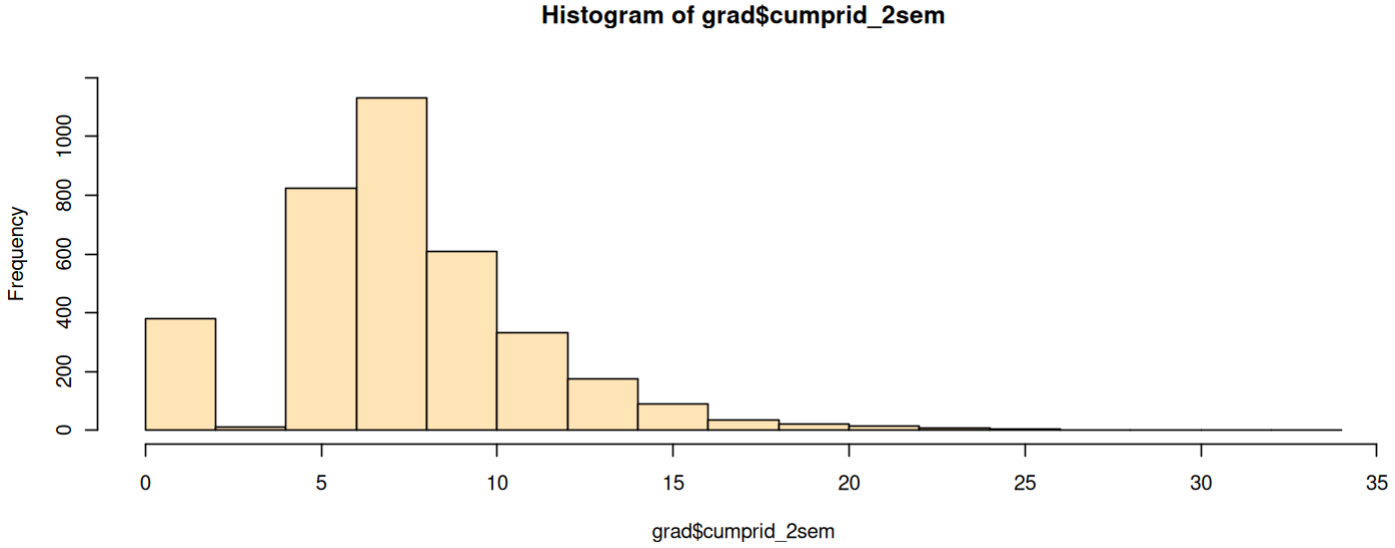
Outro indicador de evasão ou não é `matric_2sem`, para a qual, a maioria abaixo de 6 evadiu.

cumprid_2sem

```
grafdim(12,5)
boxplot(grad$cumprid_2sem ~ grad$graduou, horizontal=TRUE, col="moccasin")
```




```
hist(grad$cumprid_2sem, ylim=c(0,1200), col="moccasin")
```



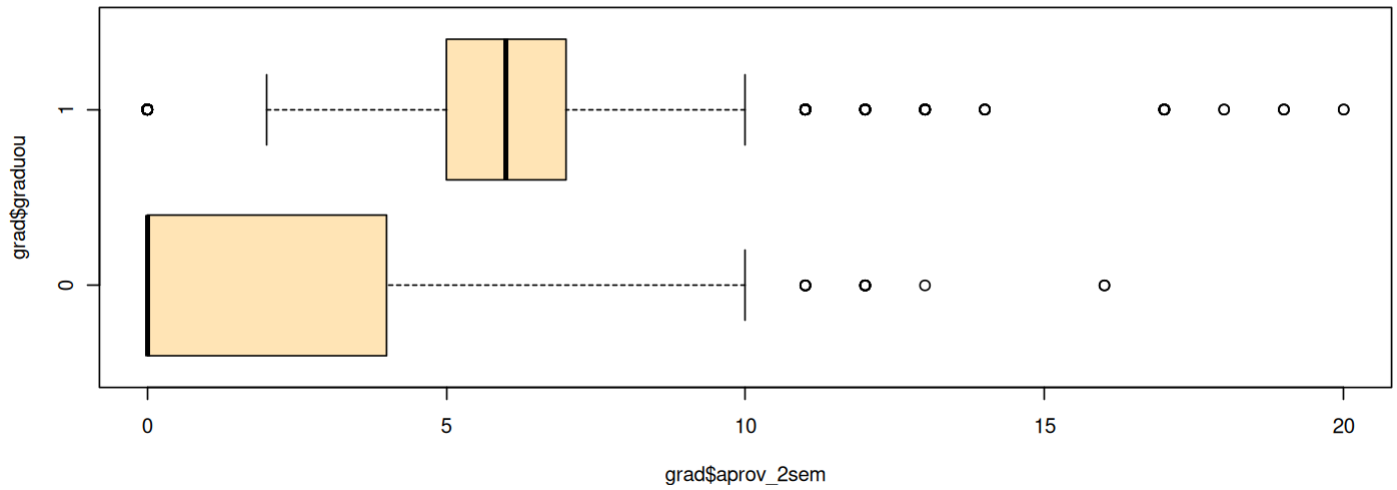
```
table(grad$graduou,grad$cumprid_2sem)
```

```
   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
0 297   3   4   1   8 161 135 115 127 110 122  80  88  50  43  27  21   6   6
1  75   0   0   0   0 107 421 363 527 243 135 101  62  48  34  26  15   9  12

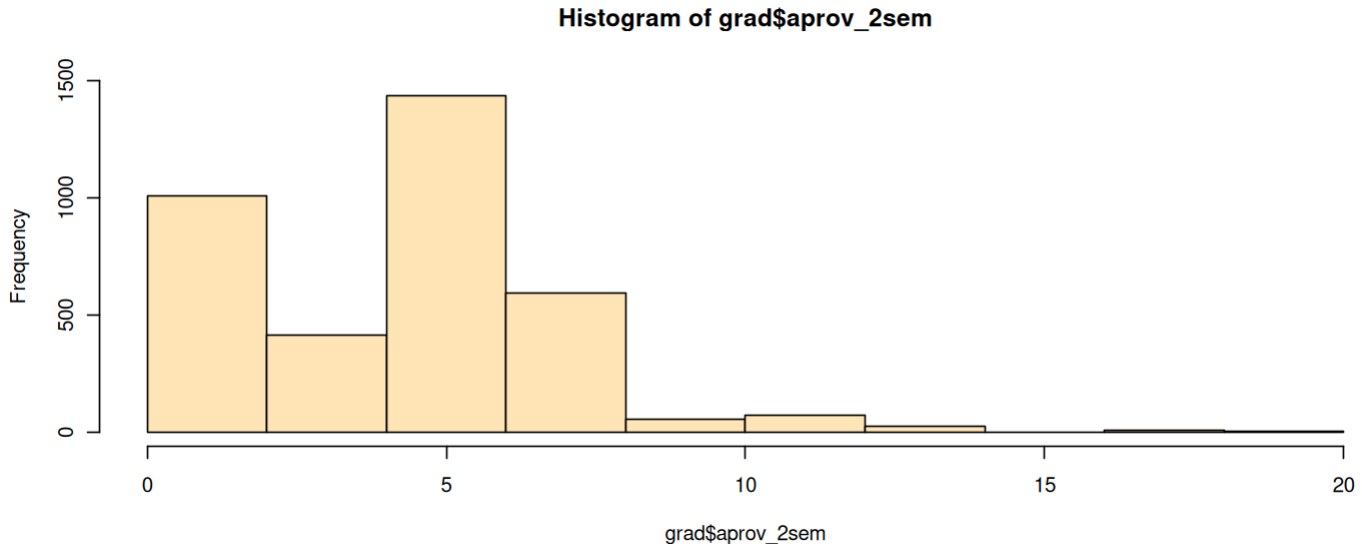
   19  20  21  22  23  24  25  26  27  33
0   4   2   4   2   2   2   1   0   0   0
1   7   6   4   5   2   1   0   3   1   1
```

aprov_2sem

```
grafdim(12,5)
boxplot(grad$aprov_2sem ~ grad$graduou, horizontal=TRUE, col="moccasin")
```




```
hist(grad$aprov_2sem, ylim=c(0,1500), col="moccasin")
```



```
table(grad$graduou,grad$aprov_2sem)
```

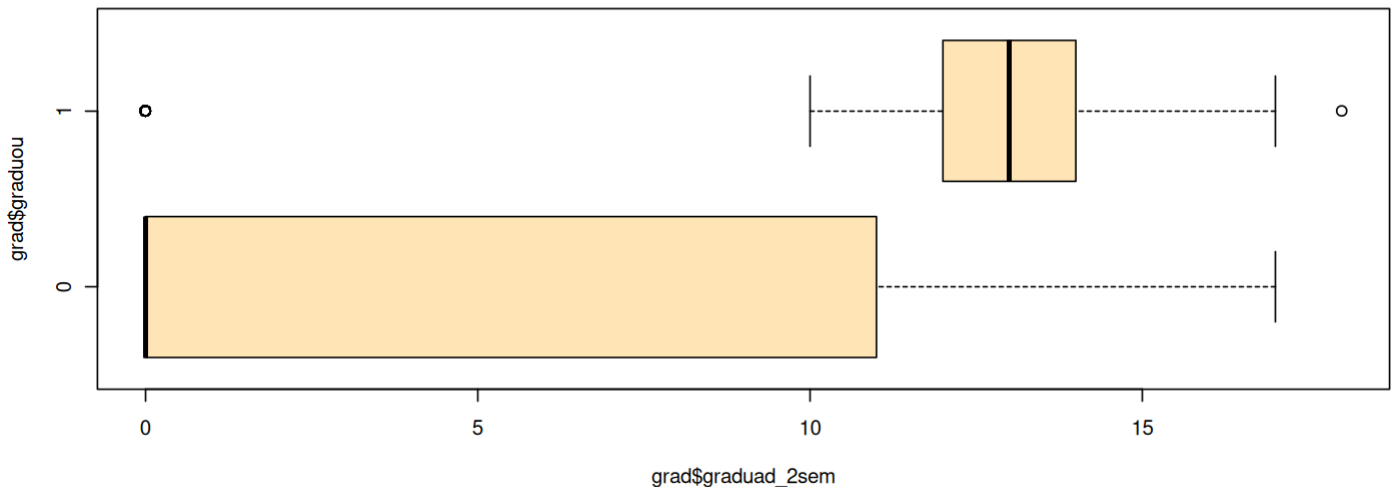
```
   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  16  17  18  19
0 727  86 116 127 112 119  64  19  20   6  10   5   7   1   0   2   0   0   0
1  75   0   5  34 142 450 803 268 286  19  22  39  24  20   6   0   8   2   3

   20
0   0
1   2
```

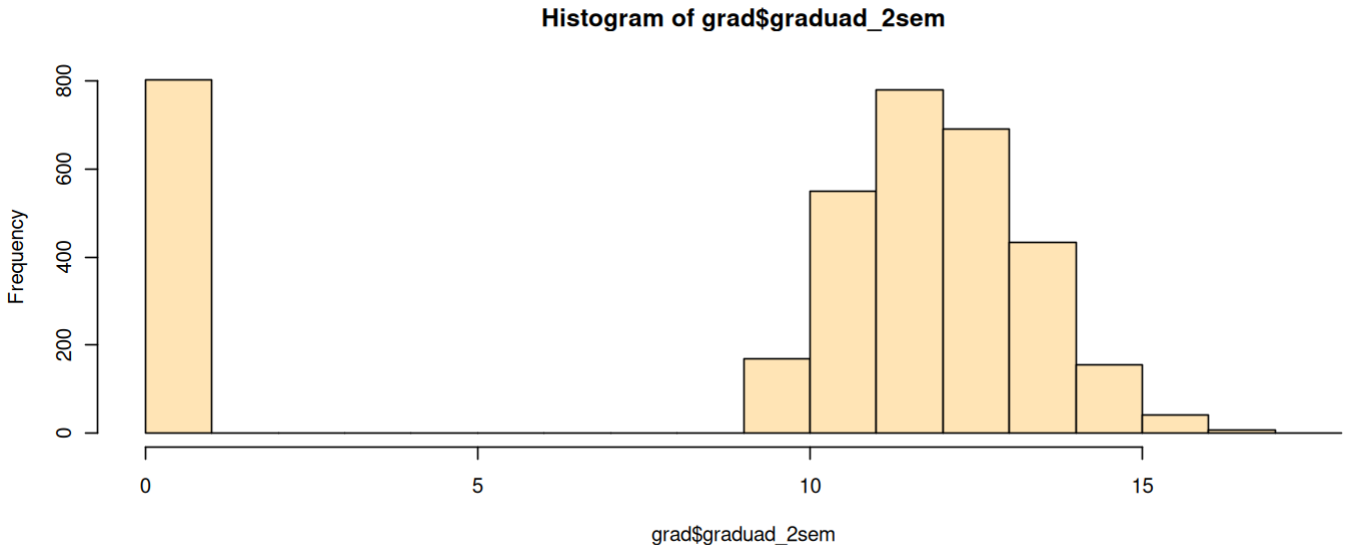
Há uma grande quantidade de evadidos abaixo de 5.

graduad_2sem

```
grafdim(12,5)
boxplot(grad$graduad_2sem ~ grad$graduou, horizontal=TRUE, col="moccasin")
```




```
hist(grad$graduad_2sem, ylim=c(0,800), col="moccasin")
```

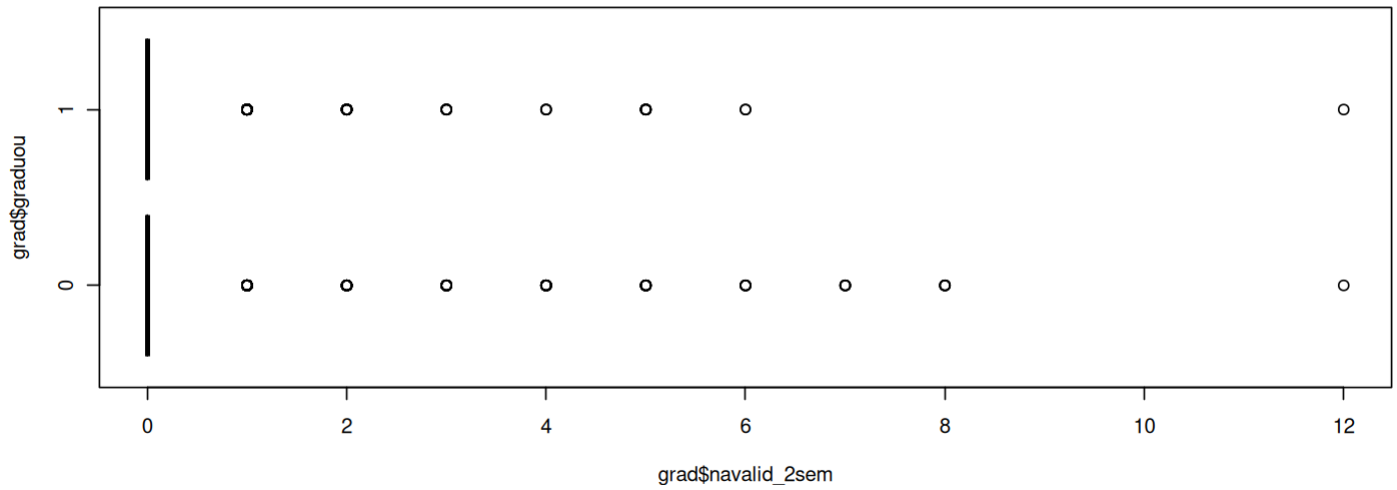


```
table(grad$graduou,grad$graduad_2sem)
```

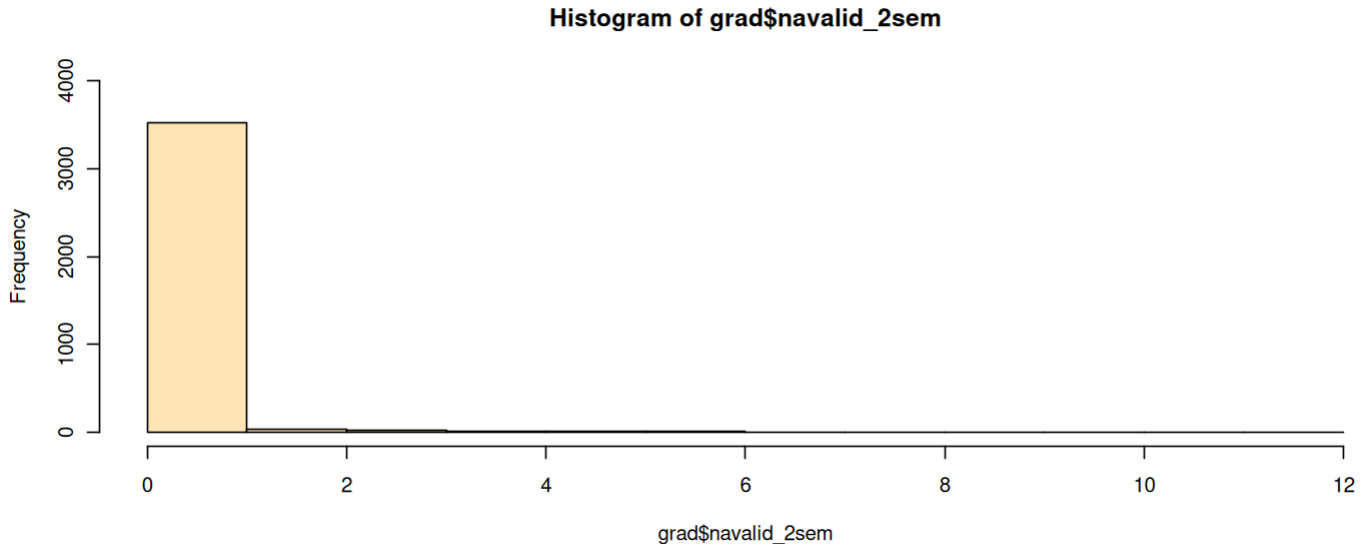
```
     0  10  11  12  13  14  15  16  17  18
  0 727 124 219 169 110  44  18   7   3   0
  1  75  44 331 611 580 390 137  35   4   1
```

navalid_2sem

```
grafdim(12,5)
boxplot(grad$navalid_2sem ~ grad$graduou, horizontal=TRUE, col="moccasin")
```




```
hist(grad$navalid_2sem, ylim=c(0,4000), col="moccasin")
```

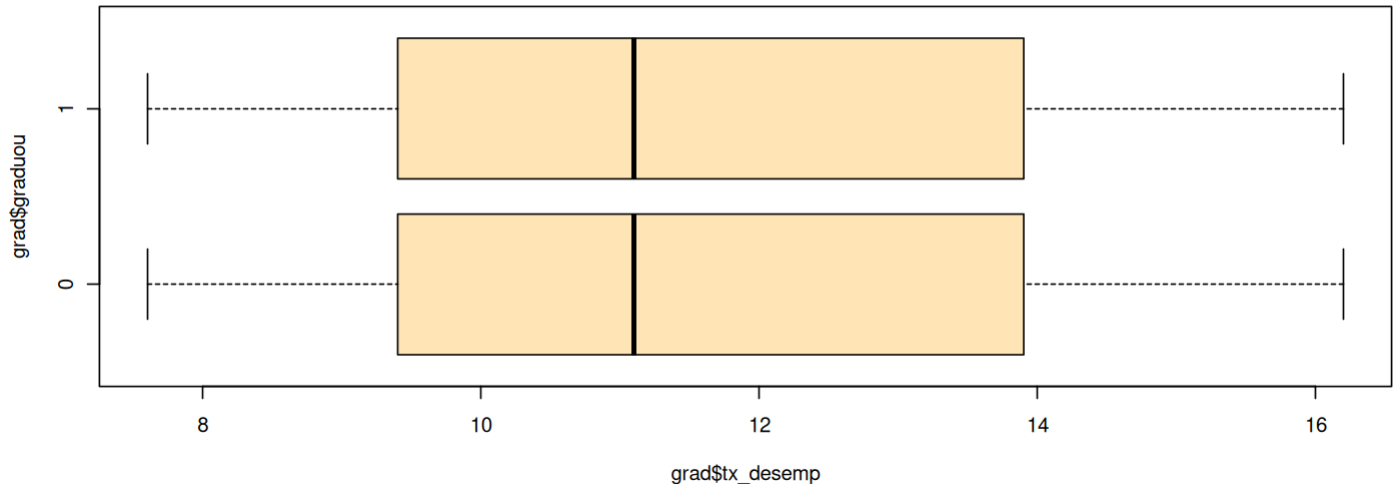



```
table(grad$graduou,grad$navalid_2sem)
```

```
     0    1    2    3    4    5    6    7    8   12
  0 1298   51   20   16   13    9    5    4    4    1
  1 2117   56   15    7    3    7    2    0    0    1
```

tx_desemp

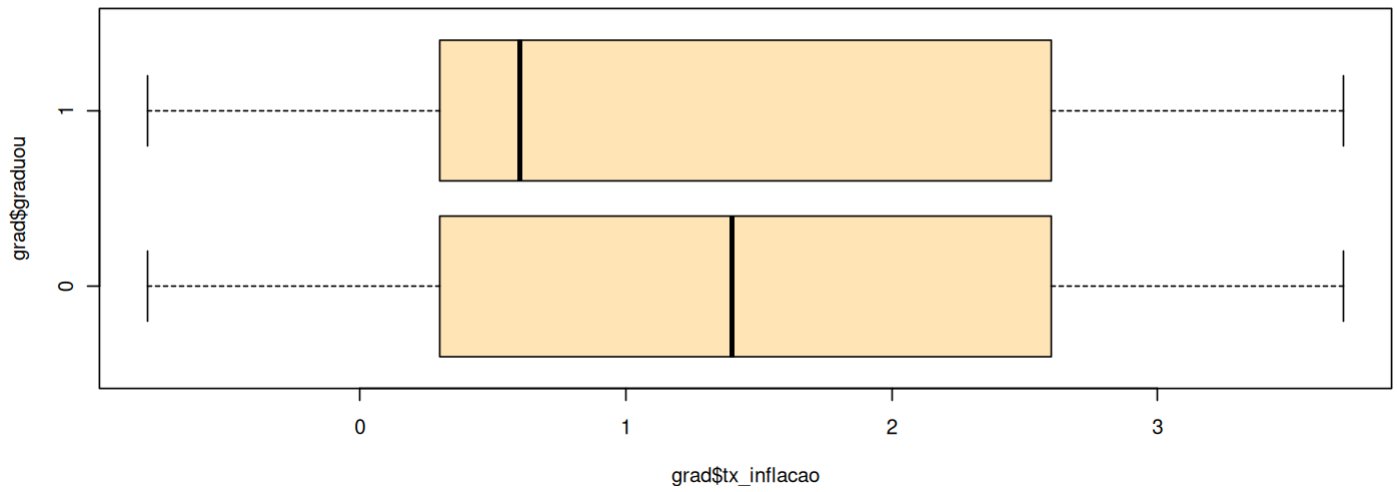
```
grafdim(12,5)
boxplot(grad$tx_desemp ~ grad$graduou, horizontal=TRUE, col="moccasin")
```



O comportamento para `tx_desemp` é exatamente o mesmo para ambos valores de `graduou`.

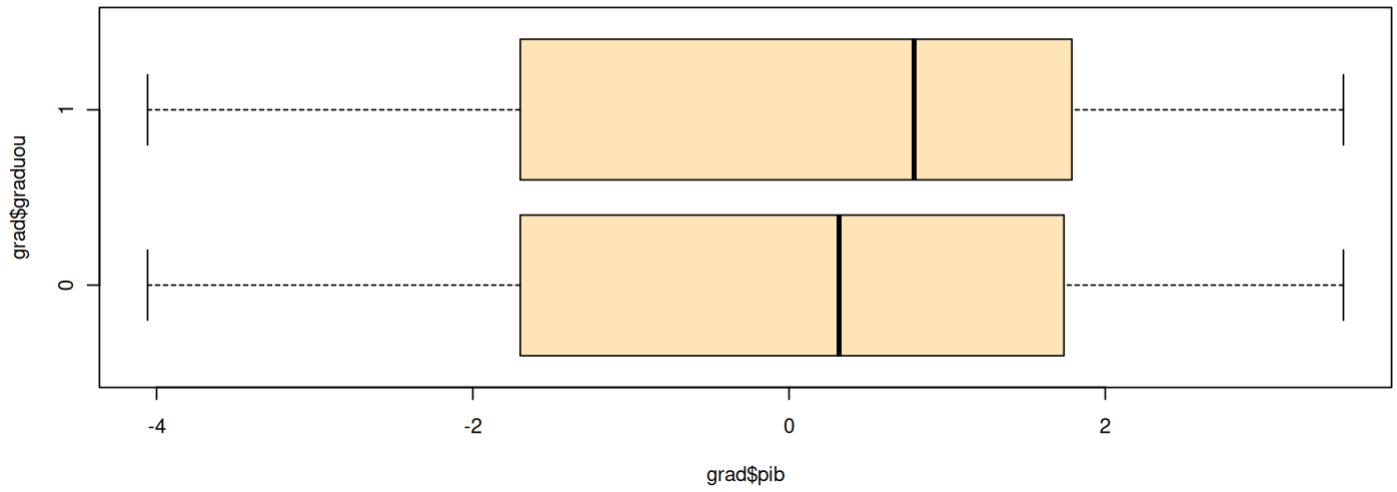
tx_inflacao

```
grafdim(12,5)
boxplot(grad$tx_inflacao ~ grad$graduou, horizontal=TRUE, col="moccasin")
```



tx_pib

```
grafdim(12,5)
boxplot(grad$pib ~ grad$graduou, horizontal=TRUE, col="moccasin")
```



Separação conjuntos dados: treino, validação e teste

```
save(grad, grafdim, grafico_matriz_confusao ,
     file="dad/sucesso_academico.RData")
```

```
rm(list=ls())
```

```
load("dad/sucesso_academico.RData")
```

```
(num_obs    <- nrow(grad))
tx_teste    <- 0.15
tx_valida   <- 0.15
(num_teste  <- trunc(num_obs * tx_teste))
(num_valida <- trunc(num_obs * tx_valida))
(num_treino <- num_obs - num_teste - num_valida)
```

3629

544

544

2541

```
set.seed(997)
```

```
grad_treino <- grad
amostra     <- sample(1:num_obs, num_teste, replace=FALSE)
grad_teste  <- grad_treino[ amostra, ]
grad_treino <- grad_treino[-amostra, ]
amostra     <- sample(1:(num_obs - num_teste), num_valida, replace=FALSE)
grad_valid  <- grad_treino[ amostra, ]
grad_treino <- grad_treino[-amostra, ]
```

A proporção de positivos deve ser próxima nos três conjuntos.

```r
(sum(grad_treino$graduou == 1) / nrow(grad_treino))
(sum(grad_valid$graduou  == 1) / nrow(grad_valid))
(sum(grad_teste$graduou  == 1) / nrow(grad_teste))
```

0.615112160566706

0.602941176470588

0.582720588235294

O conjunto de teste possui um proporção ligeiramente menor de graduados que os demais, mas não parece comprometedor para a análise.

Modelo básico

```r
mod_base <- rpart(graduou ~ ., data=grad_treino, method="class",
                  control=rpart.control(cp=0.0025,minsplit=2, minbucket=1))
```

```r
printcp(mod_base)
```

```
Classification tree:
rpart(formula = graduou ~ ., data = grad_treino, method = "class",
    control = rpart.control(cp = 0.0025, minsplit = 2, minbucket = 1))

Variables actually used in tree construction:
 [1] aprov_1sem   aprov_2sem   bolsa        cumprid_1sem cumprid_2sem
 [6] devedor      graduad_2sem idade_matric inscricao    matric_1sem
[11] pagto_em_dia qualif_pai   relocado     tx_desemp

Root node error: 978/2541 = 0.38489

n= 2541

         CP nsplit rel error  xerror     xstd
1 0.6595092      0   1.00000 1.00000 0.025079
2 0.0470348      1   0.34049 0.34049 0.017393
3 0.0076687      2   0.29346 0.29346 0.016315
4 0.0051125      4   0.27812 0.29346 0.016315
5 0.0044308      5   0.27301 0.29346 0.016315
6 0.0030675      8   0.25971 0.29652 0.016389
7 0.0027267     10   0.25358 0.30470 0.016584
8 0.0025562     13   0.24540 0.31391 0.016798
9 0.0025000     19   0.23006 0.31391 0.016798
```

```r
mod_base
```

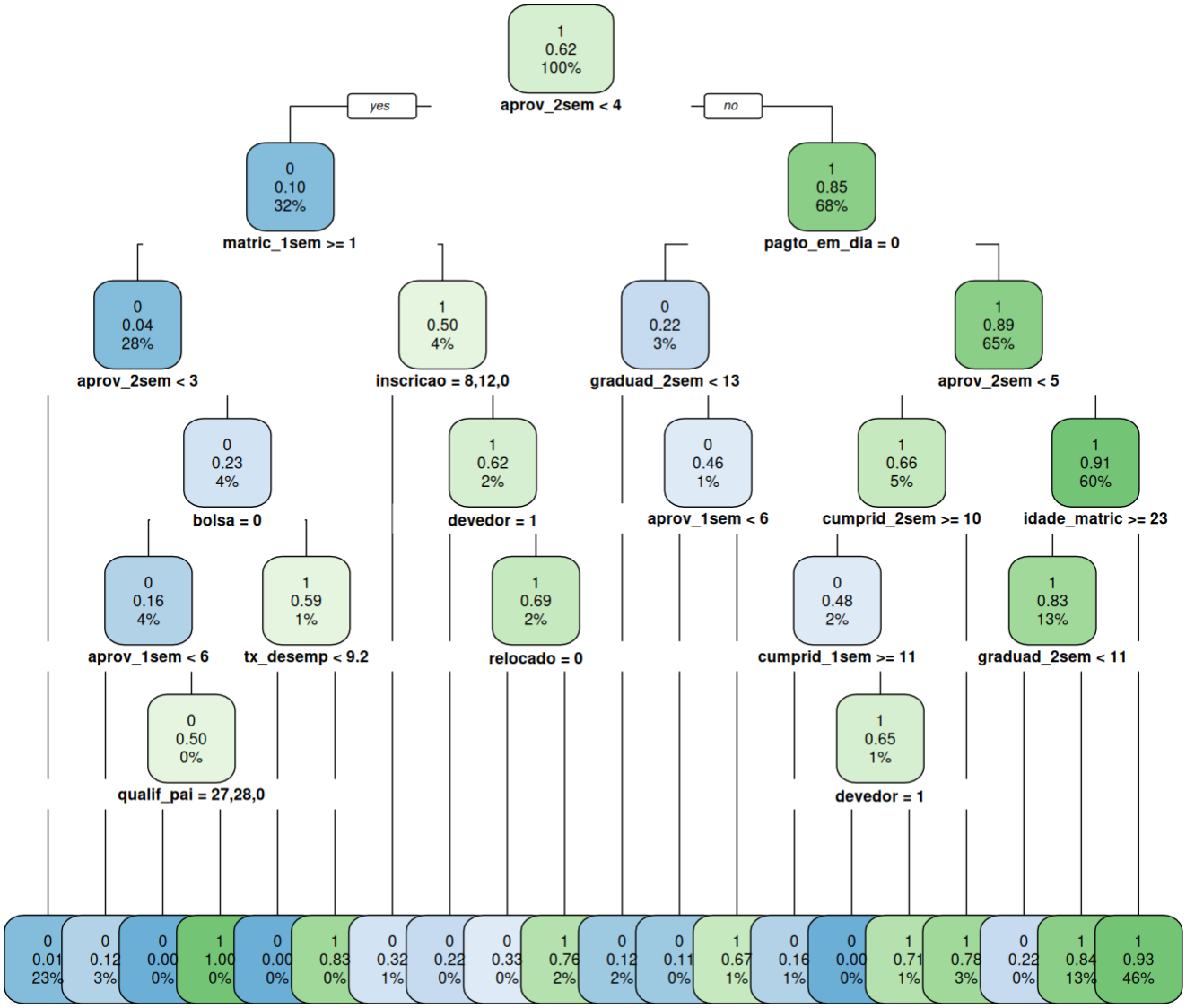
```
n= 2541

node), split, n, loss, yval, (yprob)
      * denotes terminal node

  1) root 2541 978 1 (0.384887839 0.615112161)
    2) aprov_2sem< 3.5 805  80 0 (0.900621118 0.099378882)
```

```
    4) matric_1sem ≥ 0.5 704  29 0 (0.958806818 0.041193182)
      8) aprov_2sem< 2.5 595   4 0 (0.993277311 0.006722689) *
      9) aprov_2sem ≥ 2.5 109  25 0 (0.770642202 0.229357798)
       18) bolsa=0 92  15 0 (0.836956522 0.163043478)
         36) aprov_1sem< 5.5 82  10 0 (0.878048780 0.121951220) *
         37) aprov_1sem ≥ 5.5 10   5 0 (0.500000000 0.500000000)
           74) qualif_pai=27,28,0 5   0 0 (1.000000000 0.000000000) *
           75) qualif_pai=1,14 5   0 1 (0.000000000 1.000000000) *
       19) bolsa=1 17   7 1 (0.411764706 0.588235294)
         38) tx_desemp< 9.15 5   0 0 (1.000000000 0.000000000) *
         39) tx_desemp ≥ 9.15 12   2 1 (0.166666667 0.833333333) *
    5) matric_1sem< 0.5 101  50 1 (0.495049505 0.504950495)
      10) inscricao=8,12,0 38  12 0 (0.684210526 0.315789474) *
      11) inscricao=1 63  24 1 (0.380952381 0.619047619)
        22) devedor=1 9   2 0 (0.777777778 0.222222222) *
        23) devedor=0 54  17 1 (0.314814815 0.685185185)
          46) relocado=0 9   3 0 (0.666666667 0.333333333) *
          47) relocado=1 45  11 1 (0.244444444 0.755555556) *
  3) aprov_2sem ≥ 3.5 1736 253 1 (0.145737327 0.854262673)
    6) pagto_em_dia=0 82  18 0 (0.780487805 0.219512195)
      12) graduad_2sem< 12.5 58   7 0 (0.879310345 0.120689655) *
      13) graduad_2sem ≥ 12.5 24  11 0 (0.541666667 0.458333333)
        26) aprov_1sem< 5.5 9   1 0 (0.888888889 0.111111111) *
        27) aprov_1sem ≥ 5.5 15   5 1 (0.333333333 0.666666667) *
    7) pagto_em_dia=1 1654 189 1 (0.114268440 0.885731560)
      14) aprov_2sem< 4.5 139  47 1 (0.338129496 0.661870504)
        28) cumprid_2sem ≥ 9.5 56  27 0 (0.517857143 0.482142857)
          56) cumprid_1sem ≥ 10.5 19   3 0 (0.842105263 0.157894737) *
          57) cumprid_1sem< 10.5 37  13 1 (0.351351351 0.648648649)
            114) devedor=1 3   0 0 (1.000000000 0.000000000) *
            115) devedor=0 34  10 1 (0.294117647 0.705882353) *
        29) cumprid_2sem< 9.5 83  18 1 (0.216867470 0.783132530) *
      15) aprov_2sem ≥ 4.5 1515 142 1 (0.093729373 0.906270627)
        30) idade_matric ≥ 22.5 341  59 1 (0.173020528 0.826979472)
          60) graduad_2sem< 10.5 9   2 0 (0.777777778 0.222222222) *
          61) graduad_2sem ≥ 10.5 332  52 1 (0.156626506 0.843373494) *
        31) idade_matric< 22.5 1174  83 1 (0.070698467 0.929301533) *
```

```
grafdim(12,10)
rpart.plot(mod_base, cex=0.85)
```




```
predito <- predict(mod_base, grad_treino, type="class")
print(summary(predito))
```

```
   0    1
 841 1700
```

Qualidade do modelo: dados de treino

```
matconf <- confusionMatrix(reference = grad_treino$graduou, data=predito, positive="1" )
matconf
```

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0  797   44
         1  181 1519

               Accuracy : 0.9115
                 95% CI : (0.8997, 0.9222)
    No Information Rate : 0.6151
```

```
    P-Value [Acc > NIR] : < 2.2e-16

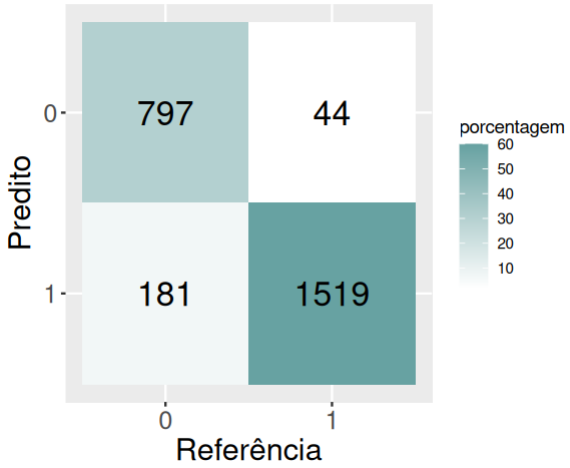
                  Kappa : 0.808

 Mcnemar's Test P-Value : < 2.2e-16

            Sensitivity : 0.9718
            Specificity : 0.8149
         Pos Pred Value : 0.8935
         Neg Pred Value : 0.9477
             Prevalence : 0.6151
         Detection Rate : 0.5978
   Detection Prevalence : 0.6690
      Balanced Accuracy : 0.8934

       'Positive' Class : 1
```

```
grafdim(5,4)
grafico_matriz_confusao(matconf)
```




```
res_modelo  <- c("modelo base")
res_acur_tr <- c(matconf$byClass["Balanced Accuracy"])
```

```
predito <- predict(mod_base, grad_valid, type="class")
print(summary(predito))
```

```
  0   1
182 362
```

Qualidade do modelo: dados de validação

```
matconf <- confusionMatrix(reference = grad_valid$graduou, data=predito, positive="1" )
matconf
```

```
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 168  14
```

```
         1  48 314

               Accuracy : 0.886
                 95% CI : (0.8563, 0.9115)
    No Information Rate : 0.6029
    P-Value [Acc > NIR] : < 2.2e-16

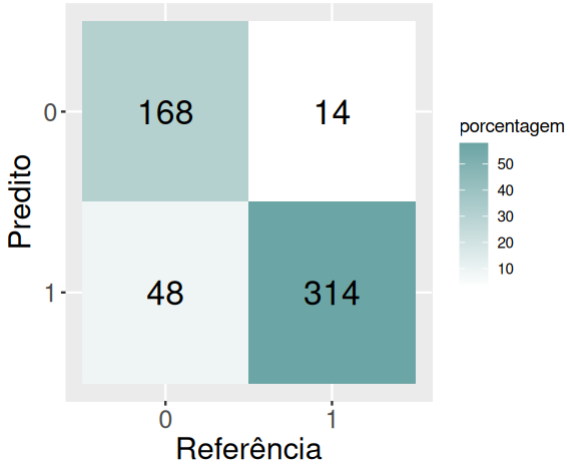
                  Kappa : 0.7554

 Mcnemar's Test P-Value : 2.777e-05

            Sensitivity : 0.9573
            Specificity : 0.7778
         Pos Pred Value : 0.8674
         Neg Pred Value : 0.9231
             Prevalence : 0.6029
         Detection Rate : 0.5772
   Detection Prevalence : 0.6654
      Balanced Accuracy : 0.8675

       'Positive' Class : 1
```

```
grafdim(5,4)
grafico_matriz_confusao(matconf)
```




```
res_acur_vl <- c(matconf$byClass["Balanced Accuracy"])
```

```
print(data.frame(modelo=res_modelo, acc_bal_treino=res_acur_tr, acc_bal_val=res_acur_vl))
```

```
                      modelo acc_bal_treino acc_bal_val
Balanced Accuracy modelo base      0.8933887   0.8675474
```

Modelo com peso

A quantidade de casos positivos (graduou) é muito diferente - cerca de 60% maior - que a dos casos negativos (evadiu):

```
print(table(grad_treino$graduou))
```

```
   0    1
 978 1563
```

Quando há um desbalanceamento da quantidade positivos/negativos, pode ocorrer uma maior sensibilidade do modelo em treinar para a classe dominante. Um modo de contornar tal problema seria atribuir pesos diferentes para observações de cada classe, de modo a compensar a diferença.

```
npos     <- sum(grad_treino$graduou == 1)
total    <- nrow(grad_treino)
nneg     <- total - npos
peso_pos <- nneg / total      # o peso dos positivos é a proporção dos negativos
peso_neg <- npos / total      # e vice-versa
paste("Peso positivo: ",peso_pos)
paste("Peso negativo: ",peso_neg)
pesos    <- numeric(nrow(grad_treino))
pesos    <- ifelse(grad_treino$graduou == 1, peso_pos, peso_neg)
```

'Peso positivo: 0.384887839433294'

'Peso negativo: 0.615112160566706'

```
mod_peso <- rpart(graduou ~ ., data=grad_treino, method="class",
                  weights=pesos,
                  control=rpart.control(cp=0.0025,minsplit=2, minbucket=1))
```

```
printcp(mod_peso)
```

```
Classification tree:
rpart(formula = graduou ~ ., data = grad_treino, weights = pesos,
    method = "class", control = rpart.control(cp = 0.0025, minsplit = 2,
        minbucket = 1))

Variables actually used in tree construction:
[1] aprov_2sem   cumprid_1sem cumprid_2sem graduad_2sem idade_matric
[6] navalid_2sem pagto_em_dia qualif_mae   tx_desemp

Root node error: 601.58/2541 = 0.23675

n= 2541

         CP nsplit rel error  xerror     xstd
1 0.6901252      0   1.00000 1.05327 0.028789
2 0.0539234      1   0.30987 0.32014 0.021142
3 0.0061889      2   0.25595 0.27375 0.019818
4 0.0033933      4   0.24357 0.25253 0.019151
5 0.0029389      6   0.23679 0.25227 0.019143
6 0.0028732      8   0.23091 0.25009 0.019072
7 0.0025000     10   0.22516 0.25163 0.019122
```

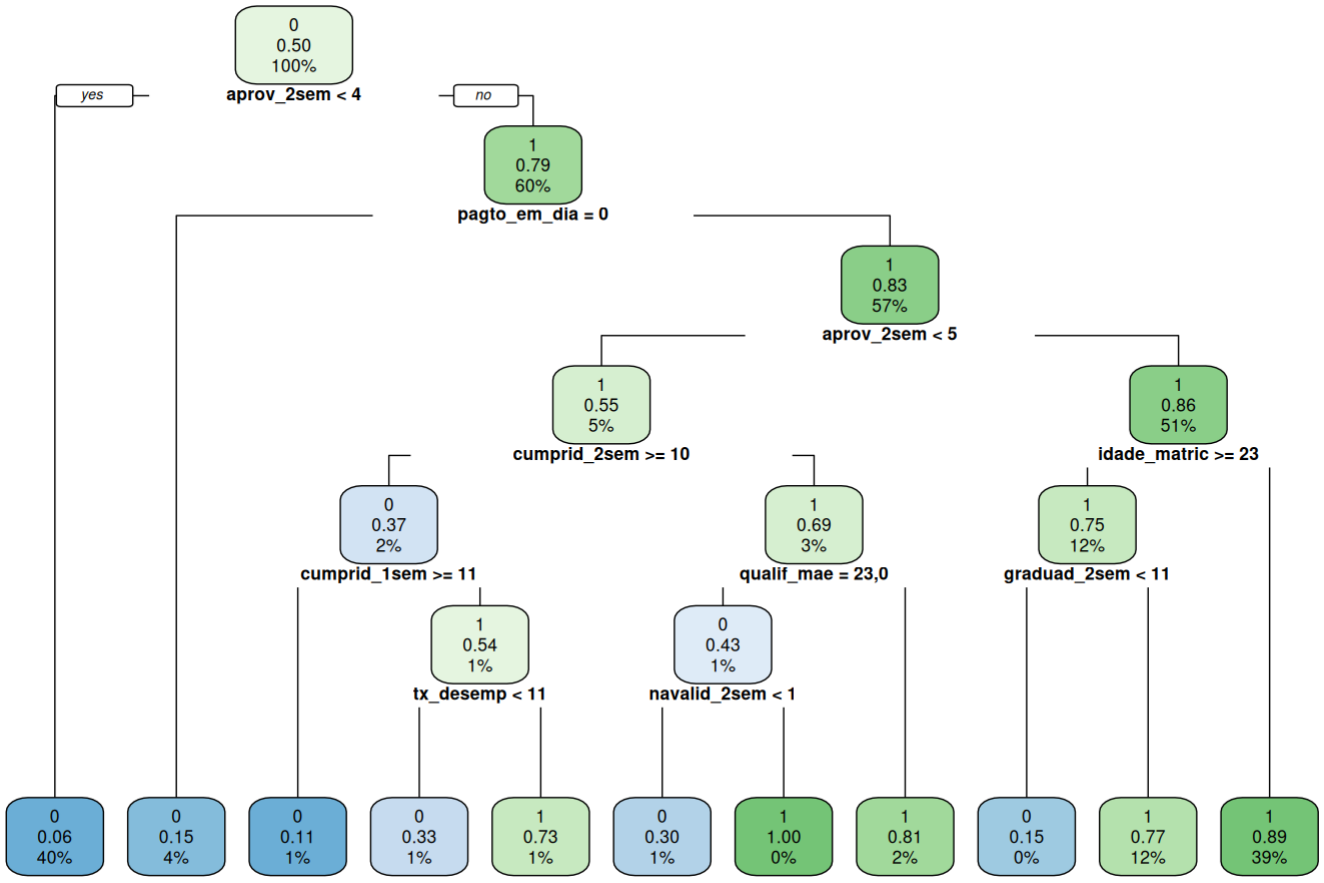
```
mod_peso
```

```
n= 2541
```

```
node), split, n, loss, yval, (yprob)
      * denotes terminal node

  1) root 2541 601.5797000 0 (0.50000000 0.50000000)
    2) aprov_2sem< 3.5 805  30.7910300 0 (0.93541437 0.06458563) *
    3) aprov_2sem≥3.5 1736 155.6234000 1 (0.21423568 0.78576432)
      6) pagto_em_dia=0 82   6.9279810 0 (0.85035193 0.14964807) *
      7) pagto_em_dia=1 1654 116.2562000 1 (0.17093562 0.82906438)
       14) aprov_2sem< 4.5 139  28.9102700 1 (0.44947594 0.55052406)
         28) cumprid_2sem≥9.5 56  10.3919700 0 (0.63188491 0.36811509)
           56) cumprid_1sem≥10.5 19   1.1546640 0 (0.89499678 0.10500322) *
           57) cumprid_1sem< 10.5 37   7.9964580 1 (0.46399945 0.53600055)
            114) tx_desemp< 10.95 16   2.6942150 0 (0.67264381 0.32735619) *
            115) tx_desemp≥10.95 21   2.4604490 1 (0.27327564 0.72672436) *
         29) cumprid_2sem< 9.5 83  11.0720200 1 (0.30679142 0.69320858)
           58) qualif_mae=23,0 22   4.6186540 0 (0.57114668 0.42885332)
            116) navalid_2sem< 0.5 17   2.6942150 0 (0.69540844 0.30459156) *
            117) navalid_2sem≥0.5 5   0.0000000 1 (0.00000000 1.00000000) *
           59) qualif_mae=1,13,22 61   4.9208970 1 (0.19434860 0.80565140) *
       15) aprov_2sem≥4.5 1515  87.3459300 1 (0.14184210 0.85815790)
         30) idade_matric≥22.5 341  36.2916200 1 (0.25058082 0.74941918)
           60) graduad_2sem< 10.5 9   0.7697757 0 (0.84833682 0.15166318) *
           61) graduad_2sem≥10.5 332  31.9858300 1 (0.22887169 0.77112831) *
         31) idade_matric< 22.5 1174  51.0543100 1 (0.10840317 0.89159683) *
```

```
grafdim(12,8)
rpart.plot(mod_peso, cex=0.85)
```



```
predito <- predict(mod_peso, grad_treino, type="class")
print(summary(predito))
```

```
   0    1
 948 1593
```

Qualidade do modelo: dados de treino

```
matconf <- confusionMatrix(reference = grad_treino$graduou, data=predito, positive="1" )
matconf
```

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0  831  117
         1  147 1446

               Accuracy : 0.8961
                 95% CI : (0.8836, 0.9077)
    No Information Rate : 0.6151
    P-Value [Acc > NIR] : < 2e-16

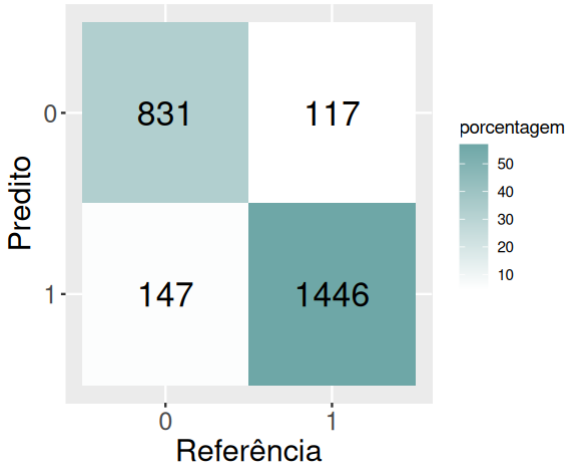
                  Kappa : 0.7793

 Mcnemar's Test P-Value : 0.07429

            Sensitivity : 0.9251
            Specificity : 0.8497
         Pos Pred Value : 0.9077
         Neg Pred Value : 0.8766
             Prevalence : 0.6151
         Detection Rate : 0.5691
   Detection Prevalence : 0.6269
      Balanced Accuracy : 0.8874

       'Positive' Class : 1
```

```
grafdim(5,4)
grafico_matriz_confusao(matconf)
```



```
res_modelo  <- c(res_modelo, "modelo com peso")
res_acur_tr <- c(res_acur_tr, matconf$byClass["Balanced Accuracy"])
```

```
predito <- predict(mod_peso, grad_valid, type="class")
print(summary(predito))
```

```
  0   1
216 328
```

Qualidade do modelo: dados de validação

```
matconf <- confusionMatrix(reference = grad_valid$graduou, data=predito, positive="1" )
matconf
```

```
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 182  34
         1  34 294

               Accuracy : 0.875
                 95% CI : (0.8442, 0.9016)
    No Information Rate : 0.6029
    P-Value [Acc > NIR] : <2e-16

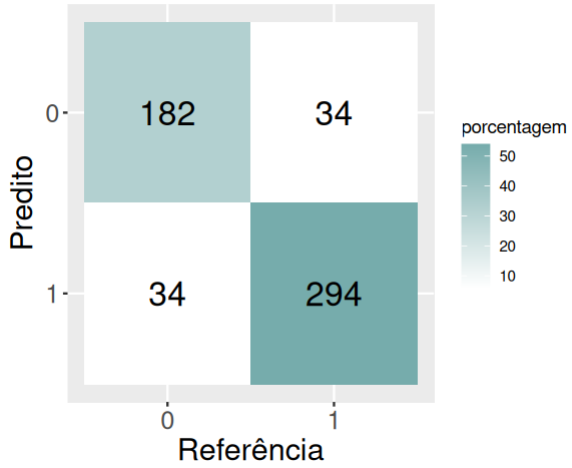
                  Kappa : 0.7389

 Mcnemar's Test P-Value : 1

            Sensitivity : 0.8963
            Specificity : 0.8426
         Pos Pred Value : 0.8963
         Neg Pred Value : 0.8426
             Prevalence : 0.6029
         Detection Rate : 0.5404
   Detection Prevalence : 0.6029
      Balanced Accuracy : 0.8695

       'Positive' Class : 1
```

```
grafdim(5,4)
grafico_matriz_confusao(matconf)
```




```
res_acur_vl <- c(res_acur_vl, matconf$byClass["Balanced Accuracy"])
```

```
print(data.frame(modelo=res_modelo, acc_bal_treino=res_acur_tr, acc_bal_val=res_acur_vl))
```

```
           modelo acc_bal_treino acc_bal_val
1     modelo base      0.8933887   0.8675474
2 modelo com peso      0.8874186   0.8694670
```

Um resultado da técnica de pesos é que a quantidade de falsos positivos e falsos negativos ficou próxima. O modelo base apresentava 4 vezes mais falsos positivos que falsos negativos; no modelo com pesos, apresentou apenas 25% a mais.

Modelo com custo

Pode ser mais importante para quem constroi o modelo considerar um tipo de classificação errada mais grave ou importante que outra. No caso do estudo de quem gradua e quem evade, o positivo é "graduou". Assim, classificar como evade (negativo) alguém que não vai evadir (vai graduar), um erro do Tipo II (falso negativo), é bem menos importante que classificar como quem vai graduar (positivo) alguém que vai de fato evadir (erro Tipo I - falso positivo), sempre considerando que é importante identificar estudantes propensos a evadir e agir para mantê-los estudando.

A alteração da função de custo uniforme é feita fornecendo ao `rpart` uma matriz de custo que tem os pesos das classificações incorretas.
A posição $m_{i,j}$ da matriz indica o custo de classificar i por j quando obviamente $i \neq j$. Como consequência, $m_{i,i} = 0$.

Tabela 13: Especificação do custo (*loss*)

	evadiu (`0`)	graduou (`1`)	explicação
predito como evadiu (`0`)	0	1	penalidade é `1` não evadir e dizer que evadiu
predito como graduou (`1`)	2	0	penalidade é `2` vezes maior ter alguém evadiu mas prever que graduou

```
custo <- matrix(c(0,1,2,0), byrow=TRUE, nrow=2)
print(custo)
```

```
     [,1] [,2]
[1,]    0    1
[2,]    2    0
```

O parâmetro `loss` porém espera os observados *nas linhas* e os preditos *nas colunas*. É necessário então transpor a matriz de custo:

```
custo <- t(custo)
print(custo)
```

```
     [,1] [,2]
[1,]    0    2
[2,]    1    0
```

```
mod_custo <- rpart(graduou ~ ., data=grad_treino, method="class",
                   control=rpart.control(cp=0.0025,minsplit=2, minbucket=1),
                   parms=list(loss=custo) )
```

```
printcp(mod_custo)
```

```
Classification tree:
rpart(formula = graduou ~ ., data = grad_treino, method = "class",
    parms = list(loss = custo), control = rpart.control(cp = 0.0025,
        minsplit = 2, minbucket = 1))

Variables actually used in tree construction:
 [1] aprov_1sem   aprov_2sem   cumprid_1sem cumprid_2sem devedor
 [6] graduad_2sem idade_matric matric_1sem  navalid_2sem pagto_em_dia
[11] qualif_mae   tx_desemp

Root node error: 1563/2541 = 0.61511

n= 2541

         CP nsplit rel error  xerror     xstd
1 0.6557901      0   1.00000 2.00000 0.031385
2 0.0409469      1   0.34421 0.33781 0.017548
3 0.0061847      2   0.30326 0.33141 0.017850
4 0.0051184      5   0.28471 0.29431 0.016619
5 0.0035189      6   0.27959 0.30134 0.016948
6 0.0031990     11   0.25976 0.30006 0.016981
7 0.0028791     14   0.25016 0.31030 0.017315
8 0.0025592     16   0.24440 0.30838 0.017269
9 0.0025000     22   0.22713 0.30454 0.017106
```

```
mod_custo
```

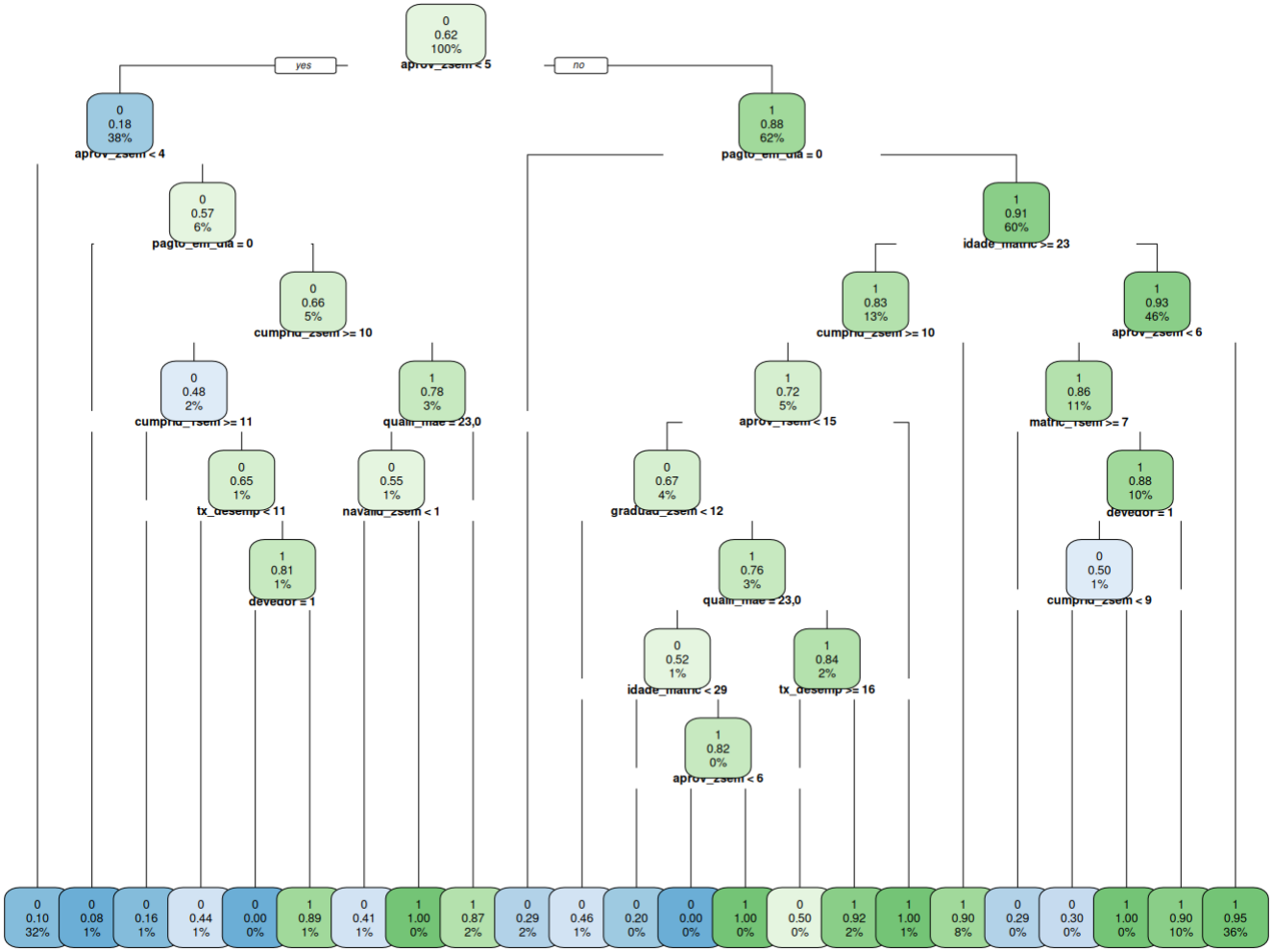
```
n= 2541

node), split, n, loss, yval, (yprob)
      * denotes terminal node

  1) root 2541 1563 0 (0.38488784 0.61511216)
    2) aprov_2sem< 4.5 970  174 0 (0.82061856 0.17938144)
      4) aprov_2sem< 3.5 805   80 0 (0.90062112 0.09937888) *
      5) aprov_2sem≥3.5 165   94 0 (0.43030303 0.56969697)
       10) pagto_em_dia=0 26    2 0 (0.92307692 0.07692308) *
```

```
    11) pagto_em_dia=1 139   92 0 (0.33812950 0.66187050)
      22) cumprid_2sem≥9.5 56   27 0 (0.51785714 0.48214286)
        44) cumprid_1sem≥10.5 19    3 0 (0.84210526 0.15789474) *
        45) cumprid_1sem< 10.5 37   24 0 (0.35135135 0.64864865)
          90) tx_desemp< 10.95 16    7 0 (0.56250000 0.43750000) *
          91) tx_desemp≥10.95 21    8 1 (0.19047619 0.80952381)
           182) devedor=1 2    0 0 (1.00000000 0.00000000) *
           183) devedor=0 19    4 1 (0.10526316 0.89473684) *
      23) cumprid_2sem< 9.5 83   36 1 (0.21686747 0.78313253)
        46) qualif_mae=23,0 22   12 0 (0.45454545 0.54545455)
          92) navalid_2sem< 0.5 17    7 0 (0.58823529 0.41176471) *
          93) navalid_2sem≥0.5 5    0 1 (0.00000000 1.00000000) *
        47) qualif_mae=1,13,22 61   16 1 (0.13114754 0.86885246) *
  3) aprov_2sem≥4.5 1571  364 1 (0.11584978 0.88415022)
    6) pagto_em_dia=0 56   16 0 (0.71428571 0.28571429) *
    7) pagto_em_dia=1 1515  284 1 (0.09372937 0.90627063)
     14) idade_matric≥22.5 341  118 1 (0.17302053 0.82697947)
       28) cumprid_2sem≥9.5 137   76 1 (0.27737226 0.72262774)
         56) aprov_1sem< 14.5 114   76 0 (0.33333333 0.66666667)
          112) graduad_2sem< 11.5 35   16 0 (0.54285714 0.45714286) *
          113) graduad_2sem≥11.5 79   38 1 (0.24050633 0.75949367)
            226) qualif_mae=23,0 21   11 0 (0.47619048 0.52380952)
              452) idade_matric< 28.5 10    2 0 (0.80000000 0.20000000) *
              453) idade_matric≥28.5 11    4 1 (0.18181818 0.81818182)
                906) aprov_2sem< 5.5 2    0 0 (1.00000000 0.00000000) *
                907) aprov_2sem≥5.5 9    0 1 (0.00000000 1.00000000) *
            227) qualif_mae=1,13,22 58   18 1 (0.15517241 0.84482759)
              454) tx_desemp≥15.85 10    5 0 (0.50000000 0.50000000) *
              455) tx_desemp< 15.85 48    8 1 (0.08333333 0.91666667) *
         57) aprov_1sem≥14.5 23    0 1 (0.00000000 1.00000000) *
       29) cumprid_2sem< 9.5 204   42 1 (0.10294118 0.89705882) *
     15) idade_matric< 22.5 1174  166 1 (0.07069847 0.92930153)
       30) aprov_2sem< 5.5 269   74 1 (0.13754647 0.86245353)
         60) matric_1sem≥6.5 7    2 0 (0.71428571 0.28571429) *
         61) matric_1sem< 6.5 262   64 1 (0.12213740 0.87786260)
          122) devedor=1 14    7 0 (0.50000000 0.50000000)
            244) cumprid_2sem< 8.5 10    3 0 (0.70000000 0.30000000) *
            245) cumprid_2sem≥8.5 4    0 1 (0.00000000 1.00000000) *
          123) devedor=0 248   50 1 (0.10080645 0.89919355) *
       31) aprov_2sem≥5.5 905   92 1 (0.05082873 0.94917127) *
```

```
grafdim(12,9)
rpart.plot(mod_custo, cex=0.6)
```




```
predito <- predict(mod_custo, grad_treino, type="class")
print(summary(predito))
```

```
   0    1
1015 1526
```

Qualidade do modelo: dados de treino

```
matconf <- confusionMatrix(reference = grad_treino$graduou, data=predito, positive="1" )
matconf
```

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0  872  143
         1  106 1420

               Accuracy : 0.902
                 95% CI : (0.8898, 0.9133)
    No Information Rate : 0.6151
    P-Value [Acc > NIR] : < 2e-16

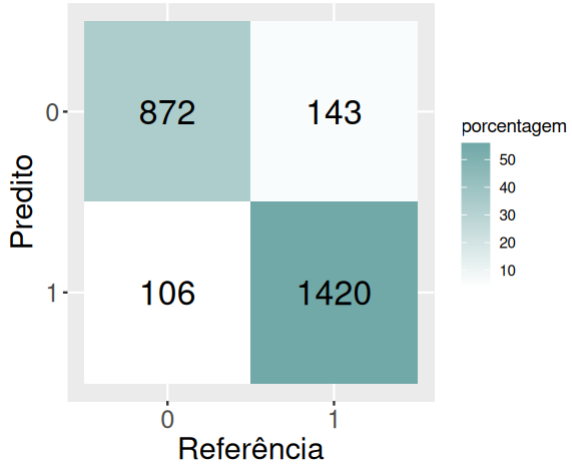
                  Kappa : 0.7945
```

```
 Mcnemar's Test P-Value : 0.02252

            Sensitivity : 0.9085
            Specificity : 0.8916
         Pos Pred Value : 0.9305
         Neg Pred Value : 0.8591
             Prevalence : 0.6151
         Detection Rate : 0.5588
   Detection Prevalence : 0.6006
      Balanced Accuracy : 0.9001

       'Positive' Class : 1
```

```
grafdim(5,4)
grafico_matriz_confusao(matconf)
```




```
res_modelo  <- c(res_modelo, "modelo com custo")
res_acur_tr <- c(res_acur_tr, matconf$byClass["Balanced Accuracy"])
```

```
predito <- predict(mod_custo, grad_valid, type="class")
print(summary(predito))
```

```
  0   1
233 311
```

Qualidade do modelo: dados de validação

```
matconf <- confusionMatrix(reference = grad_valid$graduou, data=predito, positive="1" )
matconf
```

```
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 190  43
         1  26 285

               Accuracy : 0.8732
```

```
               95% CI : (0.8422, 0.9)
    No Information Rate : 0.6029
    P-Value [Acc > NIR] : < 2e-16

                  Kappa : 0.7386

 Mcnemar's Test P-Value : 0.05408

            Sensitivity : 0.8689
            Specificity : 0.8796
         Pos Pred Value : 0.9164
         Neg Pred Value : 0.8155
             Prevalence : 0.6029
         Detection Rate : 0.5239
   Detection Prevalence : 0.5717
      Balanced Accuracy : 0.8743

       'Positive' Class : 1
```

```
grafdim(5,4)
grafico_matriz_confusao(matconf)
```



É possível observar que, mesmo com mais casos positivos, o modelo jogou o "custo" para o falso negativo, minimizando a ocorrência do falso positivo. Apesar dessa transferência ainda apresentou uma melhor acurácia balanceada que os demais modelos.

```
res_acur_vl <- c(res_acur_vl, matconf$byClass["Balanced Accuracy"])
```

```
print(data.frame(modelo=res_modelo, acc_bal_treino=res_acur_tr, acc_bal_val=res_acur_vl))
```

```
            modelo acc_bal_treino acc_bal_val
1      modelo base      0.8933887   0.8675474
2  modelo com peso      0.8874186   0.8694670
3 modelo com custo      0.9000624   0.8742660
```

Modelo com peso, custo e poda

Vamos construir uma árvore de decisão com:

- **ponderação**, para considerar um maior equilíbrio entre casos positivos e negativos;

- **custo**, para indicar a importância de um erro frente a outro e
- **poda**, para simplificar a complexidade da árvore de decisão final.

```
mod_pcp <- rpart(graduou ~ ., data=grad_treino, method="class",
                 control=rpart.control(cp=0.0025,minsplit=2, minbucket=1),
                 weights=pesos,
                 parms=list(loss=custo) )
```

```
printcp(mod_pcp)
```

```
Classification tree:
rpart(formula = graduou ~ ., data = grad_treino, weights = pesos,
    method = "class", parms = list(loss = custo), control = rpart.control(cp = 0.0025,
        minsplit = 2, minbucket = 1))

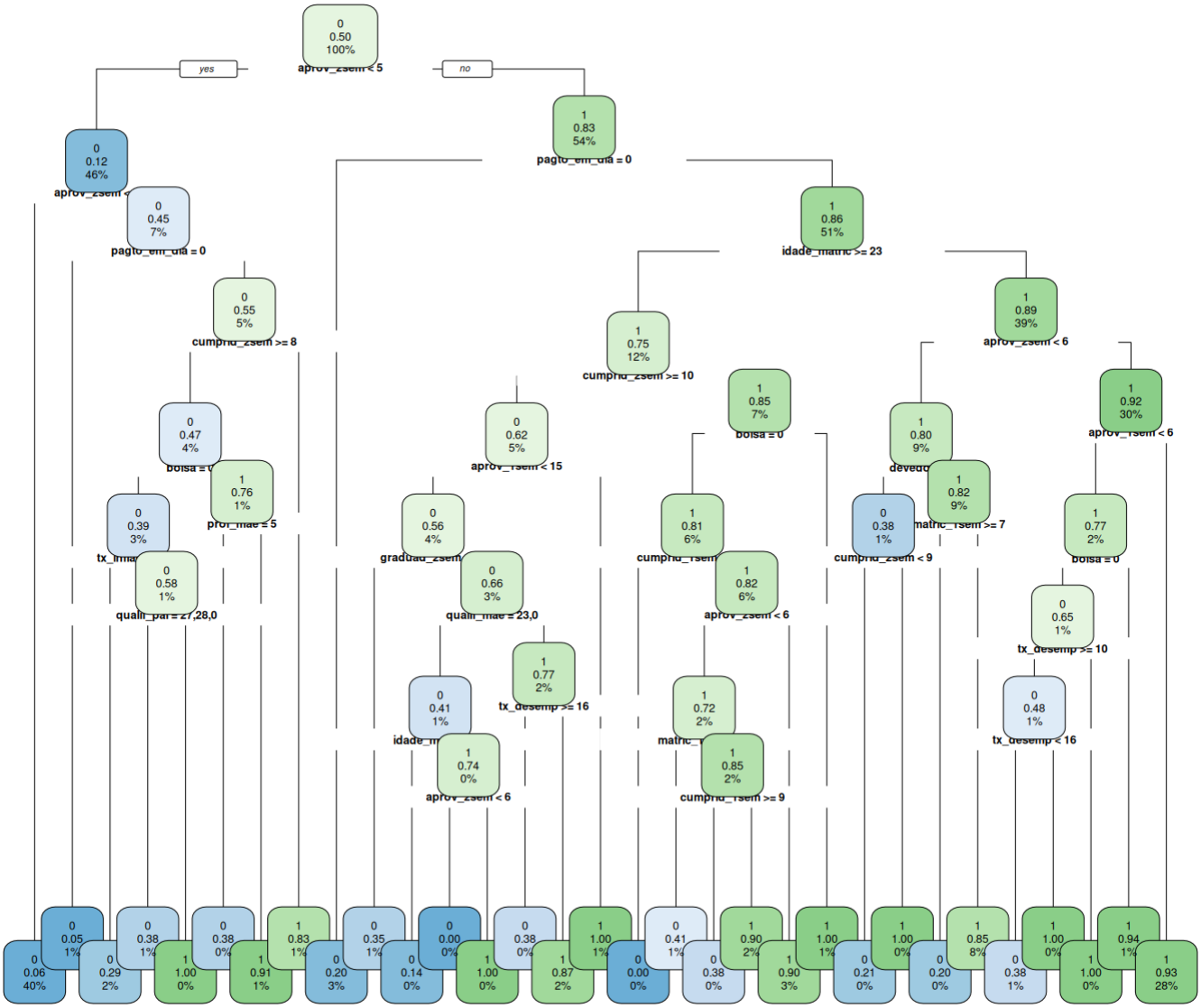
Variables actually used in tree construction:
 [1] aprov_1sem   aprov_2sem   bolsa        cumprid_1sem cumprid_2sem
 [6] devedor      graduad_2sem idade_matric matric_1sem  pagto_em_dia
[11] prof_mae     qualif_mae   qualif_pai   tx_desemp    tx_inflacao

Root node error: 601.58/2541 = 0.23675

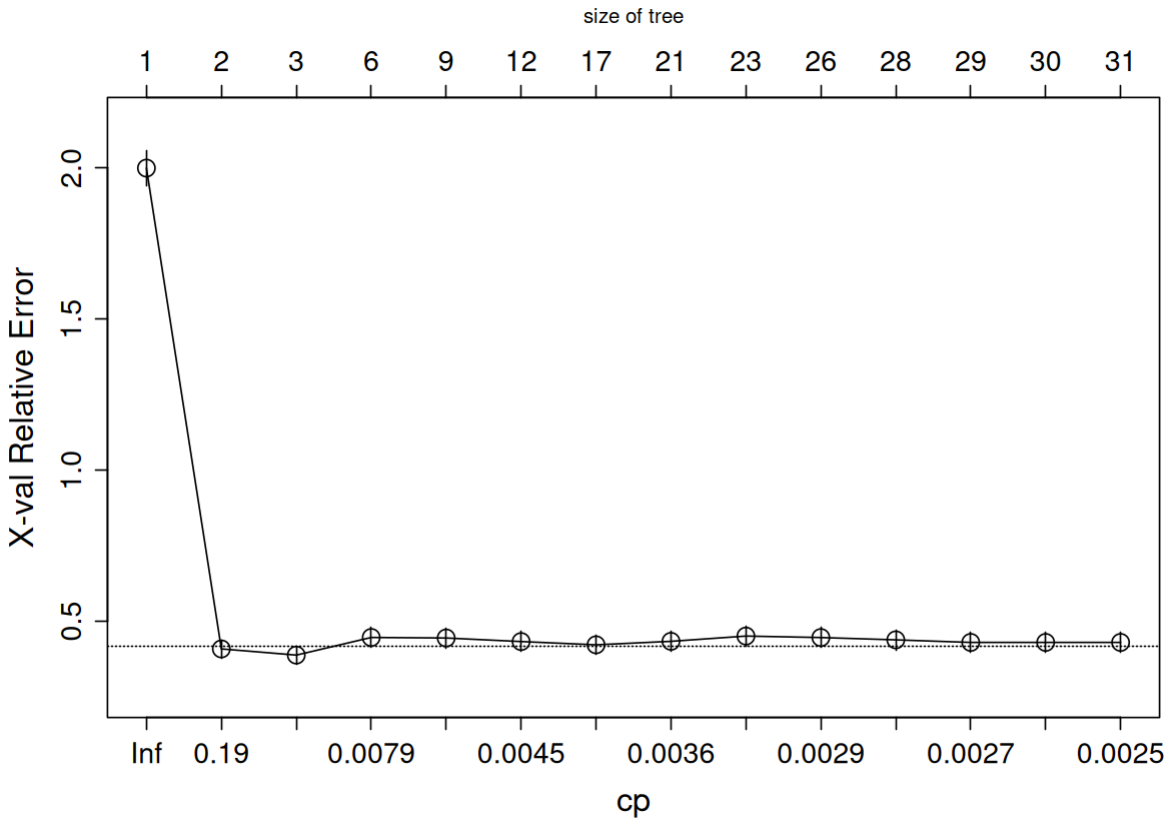
n= 2541

          CP nsplit rel error  xerror     xstd
1  0.5164875      0   1.00000 2.00000 0.057659
2  0.0715629      1   0.48351 0.40874 0.030178
3  0.0096951      2   0.41195 0.38832 0.030402
4  0.0064725      5   0.38286 0.44669 0.033293
5  0.0049182      8   0.36289 0.44490 0.033428
6  0.0040978     11   0.34411 0.43312 0.032683
7  0.0036729     16   0.32299 0.42237 0.032329
8  0.0035189     20   0.30829 0.43389 0.032791
9  0.0028941     22   0.30126 0.45129 0.033420
10 0.0028791     25   0.29257 0.44643 0.033258
11 0.0028104     27   0.28682 0.43875 0.032958
12 0.0025592     28   0.28401 0.43030 0.032544
13 0.0025592     29   0.28145 0.43030 0.032544
14 0.0025000     30   0.27889 0.43030 0.032544
```

```
grafdim(12,10)
rpart.plot(mod_pcp, cex=0.6)
```




```
grafdim(10,7)
plotcp(mod_pcp, cex=1.8, cex.lab=1.6, cex.axis=1.4, sub="")
```




```
cp     <- mod_pcp$cptable[,1]
xerror <- mod_pcp$cptable[,4]
xstd   <- mod_pcp$cptable[,5]
tem    <- character(length(cp))
tem[1] <- "-"
for (linha in 2:length(cp)) {
    if ( (xerror[linha]+xstd[linha]) < (xerror[linha-1]-xstd[linha-1]) ) {
        tem[linha] <- "___não_tem___"
    } else { tem[linha] <- "tem     "}
}
print(data.frame(cp, "xerr_menos_xstd"=xerror-xstd, xerror, "xerr_mais_xstd"=xerror+xstd, tem_intersecção=
```

```
            cp xerr_menos_xstd    xerror xerr_mais_xstd tem_intersecção
1  0.516487485       1.9423408 2.0000000      2.0576592               -
2  0.071562867       0.3785651 0.4087428      0.4389206   ___não_tem___
3  0.009695057       0.3579141 0.3883165      0.4187189        tem
4  0.006472530       0.4133936 0.4466870      0.4799804        tem
5  0.004918181       0.4114768 0.4449050      0.4783332        tem
6  0.004097830       0.4004370 0.4331198      0.4658026        tem
7  0.003672935       0.3900455 0.4223748      0.4547040        tem
8  0.003518874       0.4010997 0.4338911      0.4666824        tem
9  0.002894125       0.4178708 0.4512912      0.4847115        tem
10 0.002879079       0.4131721 0.4464299      0.4796878        tem
11 0.002810389       0.4057947 0.4387524      0.4717100        tem
12 0.002559181       0.3977594 0.4303035      0.4628477        tem
13 0.002559181       0.3977594 0.4303035      0.4628477        tem
14 0.002500000       0.3977594 0.4303035      0.4628477        tem
```

O gráfico recomenda um valor de `cp` menor que 0,19. Vamos considerar 0,18 para a poda.

```
mod_pcp <- prune(mod_pcp, cp=0.18)
```

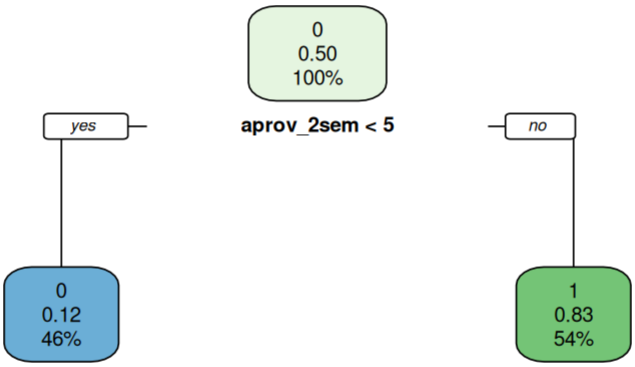
```
mod_pcp
```

```
n= 2541

node), split, n, loss, yval, (yprob)
      * denotes terminal node

1) root 2541 601.57970 0 (0.5000000 0.5000000)
  2) aprov_2sem< 4.5 970  66.97048 0 (0.8796793 0.1203207) *
  3) aprov_2sem≥4.5 1571 223.90080 1 (0.1731479 0.8268521) *
```

```
grafdim(12,8)
rpart.plot(mod_pcp)
```



Qualidade do modelo: dados de treino

```
predito <- predict(mod_pcp, grad_treino, type="class")
print(summary(predito))
```

```
   0    1
 970 1571
```

```r
matconf <- confusionMatrix(reference = grad_treino$graduou, data=predito, positive="1" )
matconf
```

```
Confusion Matrix and Statistics

          Reference
Prediction    0    1
         0  796  174
         1  182 1389

               Accuracy : 0.8599
                 95% CI : (0.8458, 0.8732)
    No Information Rate : 0.6151
    P-Value [Acc > NIR] : <2e-16

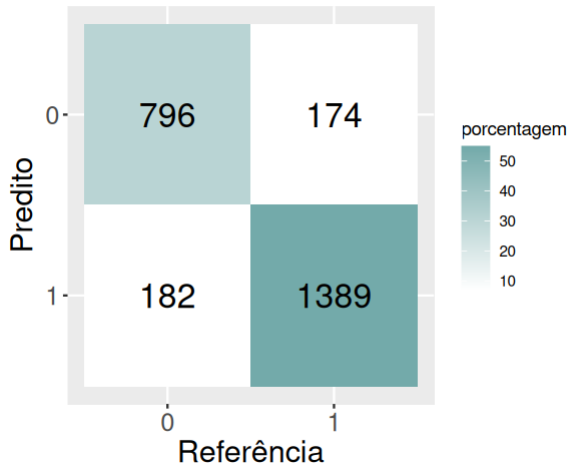
                  Kappa : 0.7037

 Mcnemar's Test P-Value : 0.7106

            Sensitivity : 0.8887
            Specificity : 0.8139
         Pos Pred Value : 0.8842
         Neg Pred Value : 0.8206
             Prevalence : 0.6151
         Detection Rate : 0.5466
   Detection Prevalence : 0.6183
      Balanced Accuracy : 0.8513

       'Positive' Class : 1
```

```r
grafdim(5,4)
grafico_matriz_confusao(matconf)
```




```r
res_modelo  <- c(res_modelo, "modelo com peso, custo e poda")
res_acur_tr <- c(res_acur_tr, matconf$byClass["Balanced Accuracy"])
```

```r
predito <- predict(mod_pcp, grad_valid, type="class")
print(summary(predito))
```

```
  0   1
231 313
```

Qualidade do modelo: dados de validação

```
matconf <- confusionMatrix(reference = grad_valid$graduou, data=predito, positive="1" )
matconf
```

```
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 181  50
         1  35 278

               Accuracy : 0.8438
                 95% CI : (0.8105, 0.8732)
    No Information Rate : 0.6029
    P-Value [Acc > NIR] : <2e-16

                  Kappa : 0.6775

 Mcnemar's Test P-Value : 0.1289

            Sensitivity : 0.8476
            Specificity : 0.8380
         Pos Pred Value : 0.8882
         Neg Pred Value : 0.7835
             Prevalence : 0.6029
         Detection Rate : 0.5110
   Detection Prevalence : 0.5754
      Balanced Accuracy : 0.8428

       'Positive' Class : 1
```

```
grafdim(5,4)
grafico_matriz_confusao(matconf)
```




```
res_acur_vl <- c(res_acur_vl, matconf$byClass["Balanced Accuracy"])
```

Avaliação final - modelo com custo: dados de teste

```
print(data.frame(modelo=res_modelo, acc_bal_treino=res_acur_tr, acc_bal_val=res_acur_vl))
```

```
                         modelo acc_bal_treino acc_bal_val
1                   modelo base      0.8933887   0.8675474
2                modelo com peso      0.8874186   0.8694670
3               modelo com custo      0.9000624   0.8742660
4 modelo com peso, custo e poda      0.8512908   0.8427620
```

Apesar de o modelo completo utilizar quase todas variáveis na sua construção, a poda fez com que a decisão da árvore fosse baseada apenas sobre `aprov_2sem`. Seu rendimento avaliado por meio da acurácia balanceada é o menor de todos os outros modelos, porém é claramente o mais simples e sabemos que considera como mais importante o erro tipo II (falso positivo).

Se formos considerar apenas a acurácia balanceada, o melhor modelo é aquele apenas com custo. Vamos elegê-lo melhor modelo e avaliar como se comporta com casos desconhecidos, do conjunto de teste.

```
predito <- predict(mod_custo, grad_teste, type="class")
print(summary(predito))
```

```
  0   1
246 298
```

Qualidade do modelo

```
matconf <- confusionMatrix(reference = grad_teste$graduou, data=predito, positive="1" )
matconf
```

```
Confusion Matrix and Statistics

          Reference
Prediction   0   1
         0 207  39
         1  20 278

               Accuracy : 0.8915
                 95% CI : (0.8623, 0.9164)
    No Information Rate : 0.5827
    P-Value [Acc > NIR] : < 2e-16

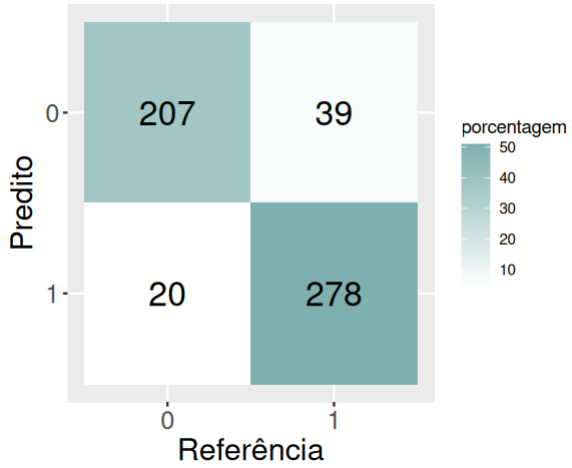
                  Kappa : 0.7796

 Mcnemar's Test P-Value : 0.01911

            Sensitivity : 0.8770
            Specificity : 0.9119
         Pos Pred Value : 0.9329
         Neg Pred Value : 0.8415
             Prevalence : 0.5827
         Detection Rate : 0.5110
   Detection Prevalence : 0.5478
      Balanced Accuracy : 0.8944
```

```
        'Positive' Class : 1
```

```
grafdim(5,4)
grafico_matriz_confusao(matconf)
```



A acurácia balanceada para o modelo de custo com os dados de teste foi de 0,894, bastante compatível com os resultados de acurácia balanceada para os dados de treino (0,900) e de validação (0,874), indicando que não houve superajuste (*overfitting*).

O modelo da árvore de decisão apenas com custo é bastante complexo. Talvez fosse recomendável podá-lo.